

Aplicación móvil para la navegación y estimación del tiempo en el transporte público de colectivos



Trabajo Final

Ingeniería de Sistemas

Facultad de Ciencias Exactas UNCPBA

Alumnos: Mauro Luis Longhi

Martin Bianco

Directores: Prof. Dr. Marcelo G. Armentano

Ing. Sebastián Vallejos

Capítulo 1: Introducción	5
1.1 Motivación	5
1.2 Objetivos	6
1.3 Organización del informe	7
Capítulo 2: Estado del Arte y marco teórico	8
2.1 Introducción	8
2.2 Sistemas de información geográficos (GIS)	8
2.2.1 Análisis y estudio del sistema de transporte.	10
2.2.2 Haversine	10
2.3 Aplicaciones existentes	12
2.3.1 Moovit	12
2.3.2 CualBondi	13
2.3.3 Cuando Llega	14
2.3.4 Google Maps	15
2.4 Impacto sobre la salud	16
2.4.1 Soluciones tecnológicas	18
2.5 Regresión lineal	19
2.5.1 Regresión lineal Simple	21
2.5.2 Regresión lineal Múltiple	23
2.5.3 Estimación por mínimos cuadrados	25
2.5.4 Métricas	26
2.5.4.1 Error absoluto medio (MAE)	27
2.5.4.2 Error cuadrático medio (RMSE)	28
2.5.4.3 R al cuadrado (R^2)	29
Capítulo 3: Enfoques de predicción	31
3.1 Introducción	31
3.2 Datos disponibles	31
3.3 Enfoque de Regresión lineal simple	34
3.3.1 Presentación de los datos	35
3.3.2 Reestructuración de la información	35
3.3.3 Refinamiento de los datos	41
3.4 Enfoque de Matriz	45
3.4.1 Marco Teórico	45
3.4.2 Preprocesamiento	47
3.4.2.1 Presentación de los datos	47
3.4.2.2 Reestructuración de la información	49
3.4.2.3 Refinamiento de los datos	50
3.4.3 Implementación	52
3.5 Enfoque de Regresión Múltiple	54
3.6 Resultados	56
3.6.1 Regresión lineal simple	56
3.6.2 Regresión lineal múltiple	59
3.6.3 Enfoque Matriz	60

3.6.4 Conclusiones	61
Capítulo 4: Desarrollo de la Propuesta	63
4.1 Introducción	63
4.2 Estructura de la Herramienta	64
4.3 Patrones Empleados	68
4.3.1 Enfoque Servidor	68
4.3.2 Enfoque Cliente	71
4.4 Requerimientos Funcionales	76
4.5 Escenarios de Calidad	78
4.6 Desarrollo del servidor	81
4.6.1 Arquitectura del servidor	81
4.6.2 Implementación de Modelos, Repositorios y Controladores	85
4.7 Enfoque aplicación móvil	88
4.7.1 Introducción	88
4.7.2 Definición de la arquitectura	89
4.7.3 Modelado de la arquitectura	91
4.7.3.1 La capa de presentación	92
4.7.3.2 La capa de la lógica de negocio	92
4.7.3.3 La capa de datos	93
4.7.3.4 Capa de inyección de dependencias	93
4.7.4 Implementación de la arquitectura	94
Capítulo 5: Interfaz Gráfica	103
5.1 Introducción	103
5.2 Estructura general de la Interfaz de Usuario	103
5.3 Análisis individual de las Vistas	111
5.3.1 Vista mapa principal	111
5.3.2 Vista de Selección líneas de colectivos	113
5.3.3 Vista del cuadro de selección de lugares	116
5.3.4 Selección de un lugar como referencia de punto.	119
5.3.5 Selección de una ruta con múltiple selección de líneas de colectivo	121
5.3.6 Vista del cálculo de tiempos e información de un recorrido	123
5.3.7 Vista Menú de configuraciones: Selección líneas de colectivos	125
5.3.8 Vista Menú de configuraciones: Remover todas las líneas guardadas	127
5.3.9 Vista Menú de configuraciones: Selección de algoritmos.	128
Capítulo 6: Conclusiones y trabajos futuros	129
6.1 Conclusiones	129
6.2 Trabajos Futuros	130
Apéndice 1: Tecnologías empleadas	132
Tecnologías Empleadas	132
1.1 Tecnología y bibliotecas utilizadas en el cliente	132
1.1.1 Kotlin	132
1.1.2 Postman	133

1.1.3 Retrofit	133
1.1.4 Plataforma de Mapas (Google Maps Platform). SDK de Mapas para Android	134
1.1.5 Material Design, librería para Android	135
1.2 Tecnología y bibliotecas utilizadas en el servidor	136
1.2.1 .Net 5	136
1.2.2 SQLite	136
1.2.3 LINQ	137
1.2.4 Docker	137
1.2.5 Entity Framework Core	138
1.2.6 Newtonsoft.Json	138
1.2.7 ML.NET	139
1.2.8 Swagger	139
1.3 Tecnología utilizadas general	140
1.3.1 Entorno de Desarrollo Integrado	140
1.3.2 GIT y GitHub: administración de código	141
1.4 Tecnologías utilizadas para la regresión lineal simple	143
1.4.1 Scikit-Learn	143
1.4.2 Jupyter Notebook	144
Apéndice 2: Videos demostrativos de los flujos de la aplicación	146
2.1 Demostración sobre el Mapa	146
2.1.1 Visualización de los controles nativos de los mapas de google utilizados	146
2.1.2 Simulación del recorrido base de las diferentes unidades de colectivos	146
2.1.3 Ventana para la selección de lugares sobre el mapa	146
2.1.3.1 Ubicación y selección de un sitio desde un marcador aleatorio en el mapa o marcador representativo de la unidad activa en el recorrido	146
2.1.3.2 Atajo visual para obtener ubicación actual del usuario	147
2.1.3.3 Selección manual lugar de origen y destino sobre el mapa	147
2.1.4 Flujos búsqueda de lugares	147
2.1.4.1 Búsqueda de recorrido con una sola linea de colectivo seleccionada	147
2.2 Demostración sobre el menú de configuración	147
2.2.1 Selección y visualización de rutas disponibles	148
2.2.2 Selección algoritmo para cálculo de búsquedas	148
2.2.3 Selección para remover las líneas de colectivos guardadas	148
Bibliografía	149

Capítulo 1: Introducción

En la ciudad de Tandil, el sistema de colectivos es el único medio de transporte masivo por lo que es utilizado por todos los sectores de la sociedad y de todas las edades. Este sistema de transporte cuenta con una flota de unidades y líneas de recorridos que permiten trasladarse desde cualquier punto de la ciudad a otro en poco tiempo. Esto significa un gran impacto sobre las actividades de la ciudad, ya sea a nivel económico, social o simplemente recreativo, por lo que un buen funcionamiento del sistema es crucial para toda la población. La utilización de este medio masivo de transporte requiere cada vez más del apoyo tecnológico para un uso más eficiente y sencillo para todos los usuarios. Este trabajo se centra en encontrar aquellas características o servicios adicionales que podrían aportar valor agregado a los actuales sistemas relacionados al transporte público, para posteriormente realizar el diseño e implementación de dichas características y complementar las diferentes plataformas existentes.

1.1 Motivación

Si bien la ciudad de Tandil cuenta con una aplicación vinculada al transporte de pasajeros, llamada GPS SUMO, la misma presenta ciertas falencias como son la falta de disponibilidad de herramientas específicas que faciliten la experiencia de los pasajeros y la ausencia de un mecanismo que personalice las necesidades de cada usuario en particular. Por lo tanto, tomando en cuenta esta problemática y el hecho de que las herramientas existentes no logran consolidar el vínculo entre el usuario y el sistema de colectivos, este trabajo apunta a crear una aplicación que surge de las dificultades mencionadas previamente y que ayuden a mejorar la experiencia del servicio.

La aplicación de GPS Sumo¹ permite visualizar en tiempo real el movimiento y la ubicación de los colectivos, el saldo de la tarjeta, entre otras funcionalidades. Nuestro sistema busca ser un complemento que puede ser importante para el día a día de los usuarios sin afectar el resto de las herramientas ya implementadas, facilitando y

¹ <http://www.gpssumo.com/>

mejorando la experiencia de usuario. Por lo tanto, el trabajo realizado no busca de ninguna forma ser un sustituto o reemplazo de GPS Sumo, sino un complemento.

1.2 Objetivos

El principal objetivo de este trabajo final consiste en diseñar y crear un sistema de software capaz de predecir el tiempo que demorará una persona en desplazarse de una ubicación específica a otra utilizando el sistema de colectivos públicos de la ciudad de Tandil. Complementariamente, se desarrollará un mecanismo para obtener los caminos posibles más recomendables correspondientes a un par de coordenadas de origen y destino en base a ciertos parámetros de entrada, buscando producir resultados confiables independientemente de los múltiples factores que afectan al movimiento del vehículo. El sistema propuesto, por lo tanto, no solo tendrá en cuenta aspectos relacionados al transporte público, tales como la línea y el recorrido, sino que también considerará factores externos como horario, zona céntrica, día de la semana, entre otros condicionantes.

Debido a la gran demanda en la utilización de los colectivos de forma diaria por parte de los ciudadanos, se buscará utilizar la información más reciente y precisa posible para poder otorgar un servicio que genere confianza en los usuarios. Con la información recopilada, y la utilización de un conjunto de gráficos y esquemas, será posible obtener una visión más clara y precisa del rendimiento del sistema de transporte. Adicionalmente la implementación de una serie de métricas, permitirá analizar matemáticamente la performance de los modelos propuestos.

Durante el desarrollo, se prestará especial atención a que los datos utilizados por el modelo y por la aplicación no comprometan la privacidad de los empleados del transporte público, de los consumidores, ni de ninguna otra persona. Asimismo, tampoco se almacenará o divulgará información de la infraestructura de los colectivos, garantizando por lo tanto un sistema totalmente confiable y seguro.

A nivel de estructura del sistema, se buscará generar una arquitectura que sea utilizable en el mayor número de dispositivos móviles posibles independientemente del modelo tecnológico. A su vez, se buscará diseñar una interfaz sencilla y fácil de utilizar, para que,

por un lado, se garantice que la experiencia pueda llegar a ser intuitiva, deseable y placentera y, por otro lado, permita atraer a nuevos usuarios a utilizarla.

1.3 Organización del informe

En esta sección se encarga de detallar cómo se estructura el presente informe, ofreciendo una descripción introductoria de las temáticas abordadas en cada uno de los capítulos que integran al trabajo de grado presentado. El informe cuenta con una serie de capítulos que abordan diferentes tópicos, como son: el estado del arte y el marco teórico, los enfoques propuestos, implementación de la herramienta, la interfaz de usuario y las conclusiones obtenidas.

El capítulo 2 representa la búsqueda, lectura, prueba, estudio de la bibliografía y diferentes herramientas y conocimientos relacionados a los sistemas de transporte públicos de colectivos. También se hace un análisis de los conceptos teóricos utilizados para la resolución de la problemática planteada.

El capítulo 3, se encuentra dirigido a explicar de manera detallada los diferentes enfoques propuestos y los algoritmos utilizados para el procesamiento del conjunto de datos histórico de los recorridos en la ciudad, el cálculo de las diferentes operaciones y métricas utilizadas en la aplicación. Se brinda un análisis de los resultados obtenidos, comparando la precisión y performance entre las diferentes técnicas empleadas.

El capítulo 4 y 5 reflejan el diseño, desarrollo e implementación de la herramienta. El primero de estos, trata los aspectos técnicos que fueron seleccionados para desarrollar la arquitectura e implementación del servidor y de la aplicación móvil. Seguido de esto, en el capítulo 5 se presenta el análisis y especificación de la interfaz de usuario de la aplicación móvil y los diferentes escenarios implementados. En este punto, se presenta el punto de acceso a las diferentes funcionalidades provistas y el diseño utilizado.

Por último el capítulo 6 presenta las conclusiones del trabajo de tesis, que incluyen los factores positivos y negativos de la herramienta implementada. Al final del documento se incluye un Anexo con información y material de interés y la bibliografía consultada.

Capítulo 2: Estado del Arte y marco teórico

2.1 Introducción

En esta sección se realizará un análisis de las diversas decisiones tomadas para la construcción de la aplicación presentada. Se realiza una descripción de las problemáticas que surgieron a lo largo del desarrollo y las medidas tomadas para abordarlas.

Inicialmente se realiza una introducción a los sistemas de información geográficos, describiendo brevemente qué son y sus características principales. Posteriormente se incluye información de las aplicaciones disponibles en la actualidad, que presentan una funcionalidad similar a la realizada en este trabajo indicando cuales son sus características principales y sus diferencias con la herramienta propuesta. Dichas aplicaciones no solo se limitan a la ciudad de Tandil sino que son utilizadas a nivel nacional o internacional y se encuentran disponibles en múltiples plataformas.

Complementariamente, se lleva a cabo un análisis del impacto de la temática estudiada en la salud de los individuos que utilizan el sistema de transporte. Se describe cómo los diversos factores que se relacionan con el uso de colectivos y su espera afectan, ya sea de manera positiva o negativa, en el estado de salud de las personas y en las áreas donde se desenvuelven.

2.2 Sistemas de información geográficos (GIS)

En la actualidad, la tecnología relacionada a los sistemas de información geográfica (SIG o GIS en inglés) es utilizada en todo el mundo en gran variedad de ramas científicas y tecnológicas. Los GIS, se pueden definir como una integración organizada entre hardware, software y datos geográficos, con el objetivo de capturar, analizar y

calcular la información geográficamente para solucionar problemas específicos y obtener información derivada sobre el espacio.

Gracias a dichos sistemas, procesos complejos de análisis de información relacionada al espacio que requerían gran cantidad de tiempo y recursos se han reducido, situación que ha sido aprovechada por otras ciencias, convirtiéndose en herramientas indispensables para el progreso en otras áreas como la arqueología, la sociología y cualquier otra disciplina que hagan uso de alguna variable relacionada al espacio geográfico. A modo de ejemplo, una de las áreas de aplicación es la socioeconómica, donde se pueden encontrar aplicaciones para la búsqueda y localización de servicios y negocios bajo ciertos parámetros geográficos.

Con la evolución de los GIS y su aplicación en diversas áreas de la planificación como transporte de pertenencias y productos, dichos sistemas se han centrado esencialmente en la movilidad de las personas en el entorno urbano.

A través de un GIS, los mapas pueden ser integrados y correlacionados fácilmente con múltiples datos. De hecho, mediante un campo común de referencia, cualquier información en una tabla puede visualizarse en un mapa instantáneamente, y cualquier problema representado en un mapa puede analizarse varias veces. Al contrario de lo que sucede con mapas tradicionales, los mapas en un GIS cambian dinámicamente en la medida que los datos alfanuméricos son actualizados. En la práctica, un GIS puede mapear cualquier información que esté almacenada en bases de datos o tablas que tengan un componente geográfico, lo cual posibilita visualizar patrones, relaciones y tendencias. Con un GIS se tiene una perspectiva nueva y dinámica en el manejo de la información, con el fin de ayudar a tomar mejores decisiones.

En la actualidad, los procesos de planeación, organización, gestión y evaluación del transporte reclaman sistemas eficientes de manejo y análisis de información, en términos de velocidad de procesamiento, capacidad de almacenamiento, versatilidad y confiabilidad. Para aspirar a cumplir con lo anterior, resulta indispensable disponer de los mecanismos para garantizar la generación de información certera y realista, para poder entregarle a los consumidores de estos transportes, un sistema funcional basado en las necesidades de los usuarios.

2.2.1 Análisis y estudio del sistema de transporte.

Debido a la demanda del sector de transporte, la disponibilidad para obtener información precisa y actualizada sobre la localización de las unidades y de sus atributos asociados, características y condiciones donde se desarrolla cada actividad determinará la variabilidad de la información para registrar el comportamiento del tráfico dentro del espacio geográfico de trabajo.

El marco geográfico que refleja la red vial, ya sea en dentro de la ciudad o en zonas interurbanas, permite a los especialistas en transporte y logística, la optimización de recursos y la demanda de transporte, así como la resolución del problema del viajante mediante la integración de la tecnología GPS que permite la visualización de los vehículos en la computadora ó sistema de localización automática vehicular (AVL - Automatic Vehicle Location) o los sistemas de gestión de tráfico de cada ciudad.

Los sistemas GIS se usan en transporte para realizar el inventario vial, la gestión de tráfico, o análisis del mercado, en este sentido se utiliza para registrar tramos de caminos, señalización, modelar los sistemas de circulación en función de las condiciones de tráfico como también la posibilidad de deducir el camino más corto en distancia o en tiempo entre dos puntos [1].

La disponibilidad de información actualizada, rápida y precisa puede ayudar a una herramienta a ser muy eficaz, entregando a los usuarios información confiable y que impacte visualmente para tomar decisiones y planificar el uso de las distintas alternativas de transporte.

Para alcanzar el objetivo central del proyecto, se hizo uso de receptores GPS (Sistema de Posicionamiento Global) para la captura de información geográficamente referenciada en los datos del transporte y de sus atributos asociados, características y condiciones de la misma para el procesamiento posterior de los datos y programación de la interfaz para usuario final. Esta información se podrá encontrar detallada en la Sección 3.2, que incluye los datos disponibles para la desarrollar en los distintos enfoques de predicción.

2.2.2 Haversine

Una de las grandes dificultades que se presenta cuando se dispone de información geográfica es a la hora de calcular la distancia entre dos coordenadas. Dados dos puntos, medir la distancia en un eje plano es relativamente sencillo, pero sin embargo al ubicar dichos puntos en la esfera terrestre se dificulta el cálculo entre dos posiciones. En otras palabras, la complejidad se debe a que en el cálculo de la distancia entre ambas posiciones debemos contemplar la curvatura terrestre.

Una de las alternativas para resolver dicho problema es emplear la fórmula de Haversine², la cuál se trata de una ecuación empleada para la navegación astronómica, en cuanto al cálculo de la distancia de círculo máximo entre dos puntos de un globo sabiendo su longitud y su latitud. Sin entrar en demasiados detalles en términos científicos, la fórmula de Haversine hace uso, además de las dos posiciones (las cuales disponen de un valor de latitud y longitud), del radio de la Tierra. Resulta que este valor es relativo a la latitud, debido a que la Tierra no es perfectamente redonda, el valor del radio ecuatorial es de 6378 km mientras que el polar es de 6357 km pero para llevar a cabo nuestras pruebas realizamos un valor intermedio acorde a nuestra ubicación geográfica.

Para hacer uso de la fórmula de Haversine se debe realizar la conversión de la latitud y longitud desde grados a radianes. Posteriormente el próximo cálculo que se debe realizar es obtener la diferencia entre las latitudes y longitudes de ambas posiciones.

Matemáticamente la fórmula de Haversine se representa en la figura 2.1.

$$a = \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos \varphi_1 \cdot \cos \varphi_2 \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right)$$

$$c = 2 \cdot \operatorname{atan2}(\sqrt{a}, \sqrt{1-a})$$

$$d = R \cdot c$$

Figura 2.1 Fórmula de Haversine.

En donde:

- R representa el radio de la tierra utilizado.
- φ_1 y φ_2 representa las latitudes de los puntos 1 y 2 respectivamente.

² https://es.wikipedia.org/wiki/F%C3%B3rmula_del_semiverseno

- $\Delta\varphi$ representa la diferencia entre las latitudes.
- $\Delta\lambda$ indica la diferencia entre las longitudes.

Si bien esta fórmula es utilizada ampliamente para realizar el cálculo de distancias entre dos puntos, hay que tener en cuenta varias consideraciones.

La fórmula da por sentado que la Tierra es completamente redonda, lo cual no es cierto completamente con lo que cabe esperar una tasa de error que se podría llegar a asumir.

2.3 Aplicaciones existentes

El tiempo que conlleva realizar un viaje utilizando el transporte público, es un problema común en todo el mundo y por eso se han implementado diversas aplicaciones y sistemas para contrarrestar esta problemática. Dichas aplicaciones no solo están disponibles para dispositivos móviles, sino que también consisten en aplicaciones web. Además, tampoco se limitan solamente al sistema de colectivos, sino que también se adaptan a otros medios, cómo por ejemplo el tren, subte, entre otros. A continuación, se procede a mencionar algunos de las aplicaciones existentes más utilizadas en la Argentina:

2.3.1 Moovit

Moovit³ consiste en una aplicación web que permite la localización a través de un punto origen y destino, la visualización de los recorridos y notificación de alertas en el tráfico.

Cada línea de colectivo detalla la información de cada trayecto y el tiempo de arribo del próximo colectivo en tiempo real trayecto a trayecto. Para la selección de una dirección, se especifican el punto de partida y el destino elegido. La búsqueda muestra las rutas sugeridas que indican no sólo las líneas de colectivos, sino también el recorrido completo que se deben realizar para llegar. Como se puede observar en la figura 2.2, cada ruta tiene un tiempo de recorrido además del camino necesario para abordar el colectivo y una vez descendido del colectivo, muestra el camino al punto de destino elegido.

³ <https://moovitapp.com/>

En cada destino seleccionado, la recomendación sugerida permite visualizar la frecuencia, el tiempo y demoras de los próximos arribos de los colectivos, aunque no ofrece una visualización en tiempo real de la ubicación de cada móvil.

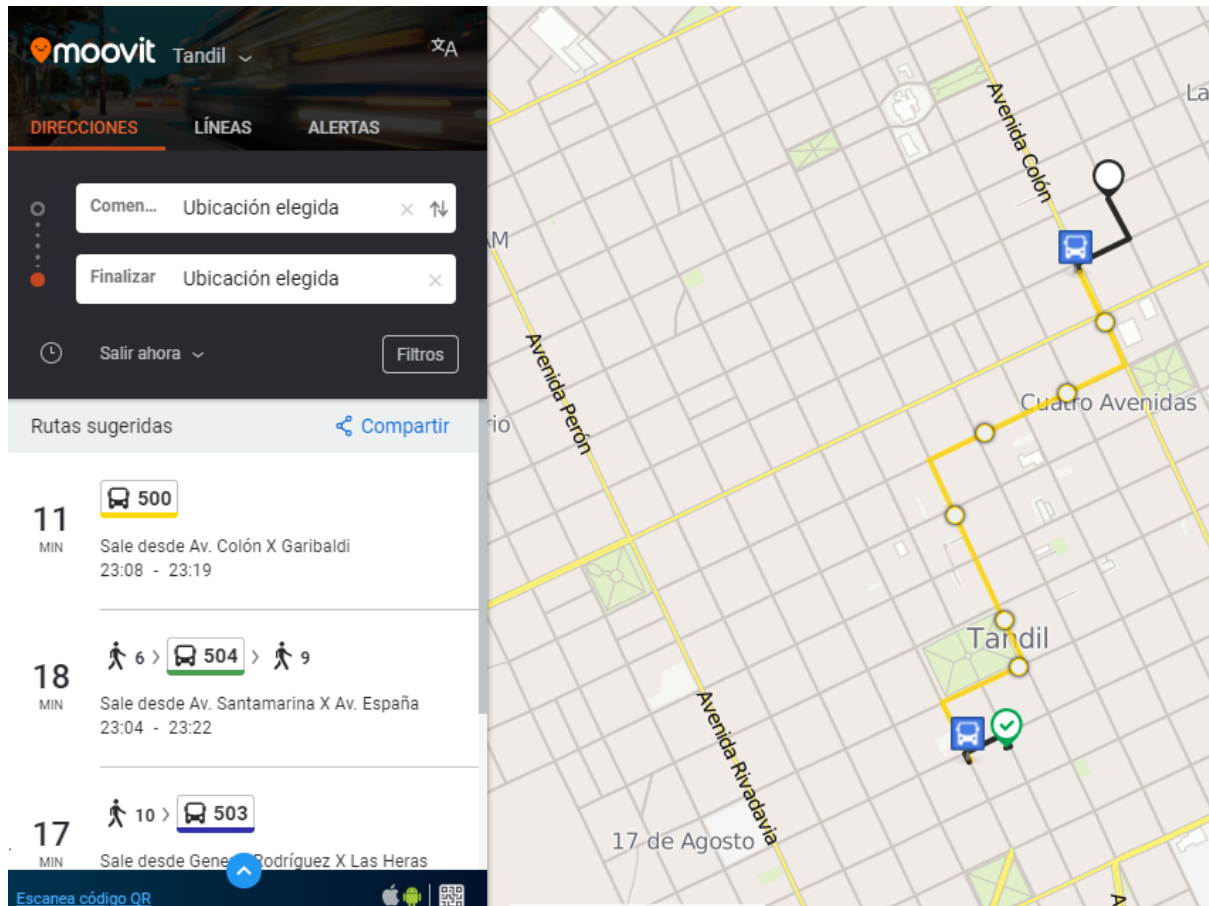


Figura 2.2 Descripción de un recorrido de la aplicación Moovit

2.3.2 CualBondi

CualBondi⁴ se trata de una aplicación web que permite visualizar solo una serie de mapas correspondientes a un número limitado de ciudades importantes, que lamentablemente (en la fecha de realización de este trabajo) no incluye a Tandil.

A partir del ingreso del punto de origen y destino donde se quiere desplazar el usuario, se muestran las alternativas de colectivos que un usuario puede utilizar y se visualiza el recorrido correspondiente. Dichas alternativas se pueden ver en la figura 2.3 las cuales

⁴ <https://cualbondi.com.ar/mapa/buenos-aires/>

muestran un ejemplo en la ciudad de Buenos Aires. En la imagen se observa el listado de las líneas de colectivos que están incluidos entre las ubicaciones seleccionadas, y la distancia total del recorrido según diferentes modos de transporte. A nivel de funcionalidad, presenta características básicas, que no informan el tiempo o distancia en ese trayecto y cuenta con poca interacción con el usuario. Tampoco informa algún detalle sobre la distancia desde la parada de inicio hasta el punto final.

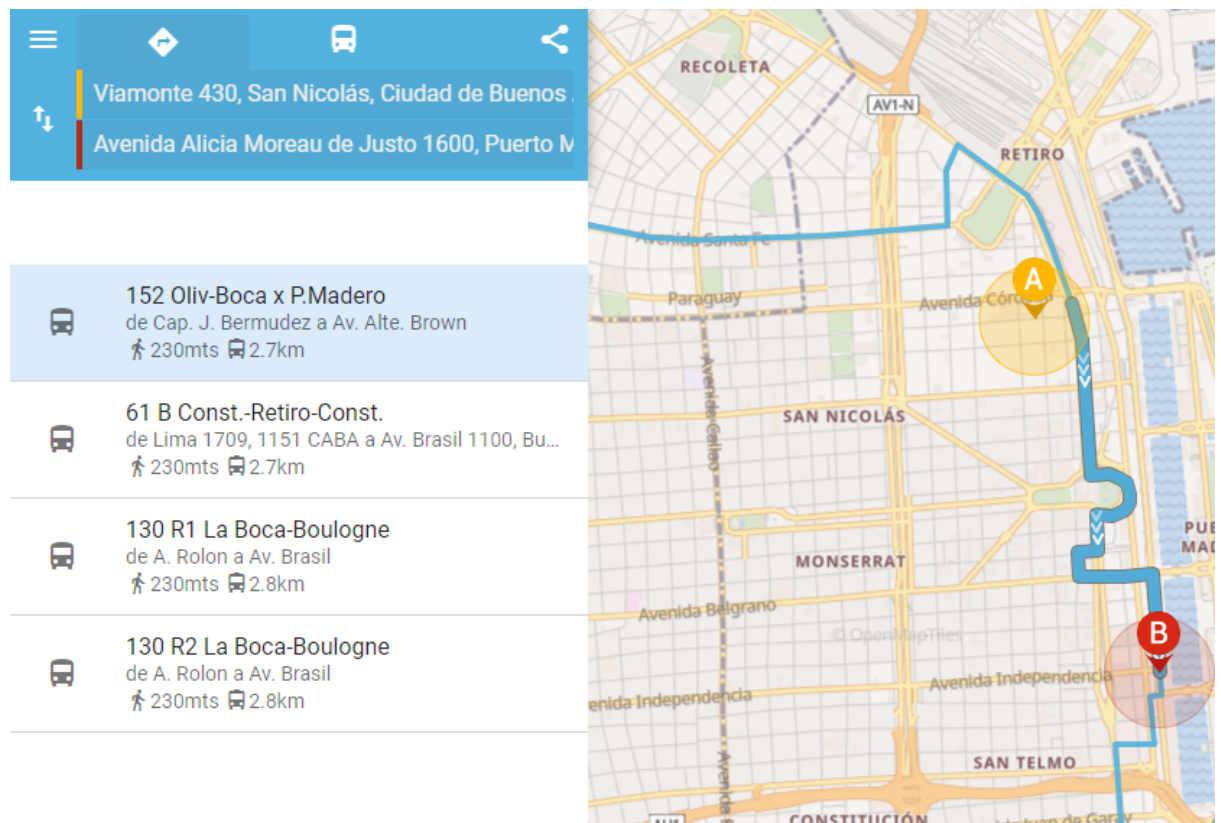


Figura 2.3 Descripción de un recorrido de la aplicación CualBondi

2.3.3 Cuando Llega

Cuando Llega⁵ es una aplicación orientada a brindar toda la información necesaria para el transporte urbano de la ciudad de Mar del Plata. Dentro de las características, se pueden ver los recorridos de las diferentes líneas de colectivos, puntos de recarga e información para ir de un punto seleccionado.

⁵ https://apps1.mardelplata.gob.ar/app_cuando_llega/web/cuando.php

Para seleccionar un destino es necesario especificar una línea, punto de partida y fin, cómo se puede apreciar en la figura 2.4. Dentro de la consulta se puede observar el recorrido de esa línea y el correspondiente tiempo de arribo al punto de destino elegido. No ofrece seguimiento por horarios de los colectivos durante el recorrido, ni información en tiempo real de demoras o próximos arribos, solo ofrece el tiempo de llegada a un punto específico, indicando en cada ramal, el tiempo de llegada.

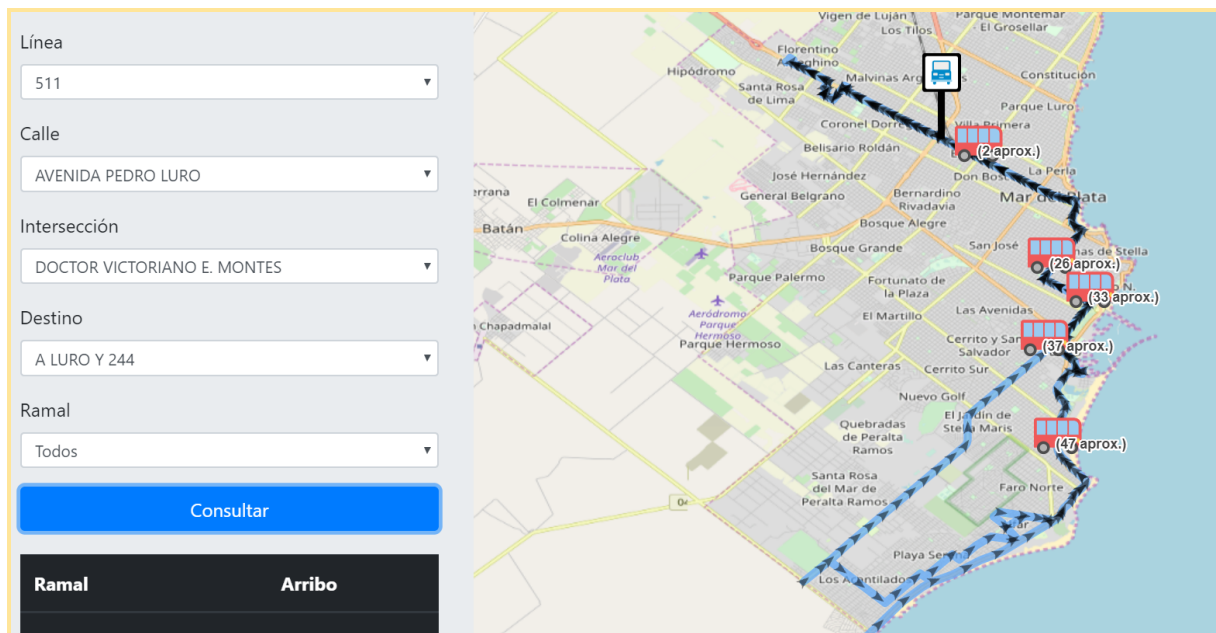


Figura 2.4 Descripción de un recorrido de la aplicación Cuando llega

2.3.4 Google Maps

Google Maps⁶ es un servidor de aplicaciones de mapas en la web que pertenece a Alphabet Inc. Ofrece imágenes de mapas desplazables, así como fotografías por satélite del mundo e incluso la ruta entre diferentes ubicaciones o imágenes a pie de calle con Google Street View, condiciones de tráfico en tiempo real (Google Traffic) y un calculador de rutas a pie, en coche, bicicleta (beta) y transporte público y un navegador GPS, Google Maps Go.

Entre las funcionalidades principales que provee Google Maps, se destacan la posibilidad de permitir a los usuarios conocer las horas de llegada y salida de las

⁶ <https://www.google.com.ar/maps/>

distintas unidades, así como también recibir alertas de los distintos servicios de transporte público -vinculadas a traslados de paradas o eventos imprevistos que afectan una estación o una ruta- y actualizaciones de viaje, tales como retrasos, cancelaciones o cambios de recorridos, limitado solo al área del Gran Buenos Aires y con un límite reducido de líneas incorporadas.

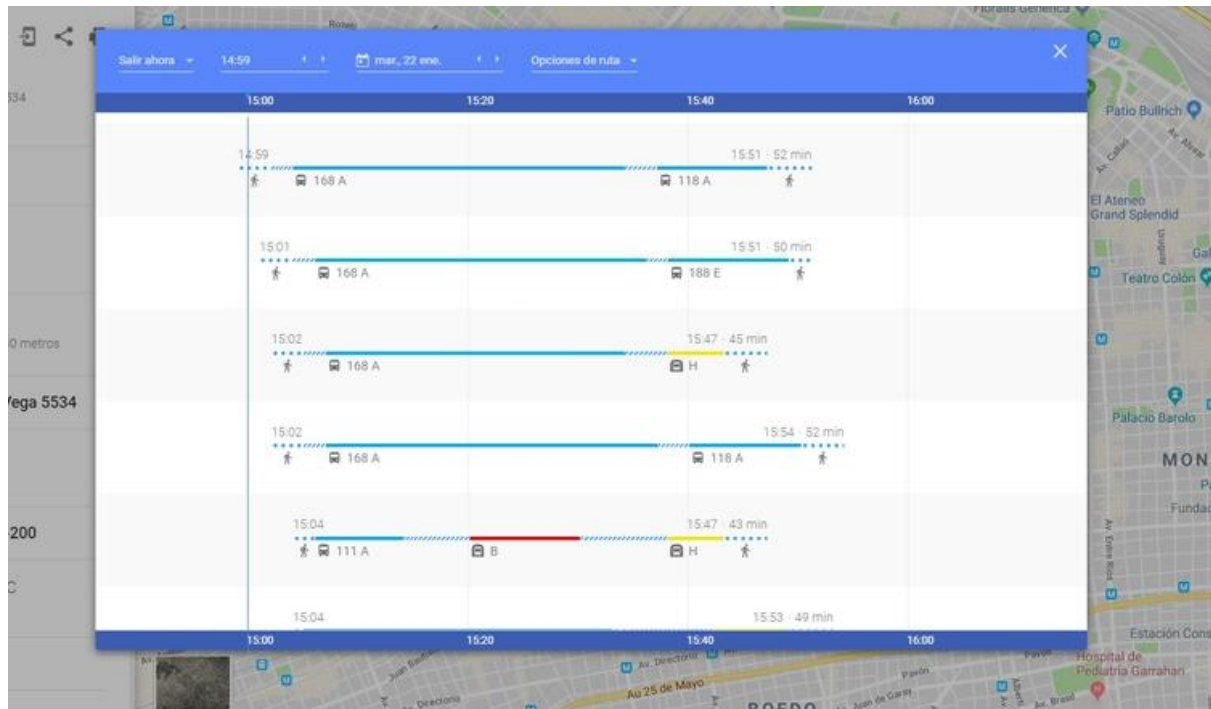


Figura 2.5 Descripción del tráfico y la afluencia de colectivos en Google Maps.

2.4 Impacto sobre la salud

Una de las principales motivaciones que se plantearon al inicio de este trabajo de tesis fue que el consumidor tenga que esperar el menor tiempo posible al arribo de una unidad de transporte público. En el estudio de cualquier problema que conlleve ocasiones de espera, es indudable que a cualquier persona no le agrada esperar. Dicha preocupación se incrementa en aquellas ocasiones donde el clima dificulta la movilidad, por ejemplo con situaciones de clima con mucho calor o mucho frío o en caso de lluvia, generando sensación de fastidio.

A veces suele considerarse que los costos asociados a este problema son de carácter económico, impactando en el ámbito laboral por ejemplo, pero también existen

problemas de carácter psicológico. Comúnmente se considera que, luego de realizar una espera de cierto tiempo, el estrés y la ansiedad empiezan a acumularse causando efectos negativos en el individuo [2]. Esto se debe a dos razones particulares. En primer lugar por la sensación de desperdicio de tiempo que podría ser utilizado para otra cuestión. En segundo lugar, por la incertidumbre que conlleva la situación de esperar y cuándo se va a concluir. Todo esto genera un efecto negativo en el humor y en la salud de la persona impactando tanto en su desempeño diario como en la comunicación con otras personas. Si bien el tiempo de espera que hay entre los colectivos en algunos casos puede ser fija, la manera en que las personas hacen uso de este tiempo varía por lo que el impacto no es semejante para todos. Sin embargo, a medida que el transporte público no arriba a horario, esto produce un aumento en la ansiedad, concentrando a las personas en el tiempo de espera y generando mucha frustración inclusive si el colectivo sólo se encuentra demorado solo unos segundos.

Otra cuestión que empeora la espera de los pasajeros consiste en que no todas las esquinas de la ciudad en donde hay una parada de colectivos poseen la estructura para realizar la espera con comodidad. Por ejemplo, no se dispone de bancos o asientos donde quizás una persona mayor o embarazada pueda descansar durante la espera. En muchos casos tampoco se provee un lugar techado donde en situaciones de lluvia pueda resguardar a varias personas que aguardan la llegada del colectivo correspondiente.

En un estudio realizado en el 2002 [3], se discute el concepto de urgencia horaria y examinan las diferencias entre aquellos usuarios que disponen de horario flexible y los que no lo tienen (los esquemas de horario flexible posibilitan a los trabajadores o estudiantes elegir, dentro de límites, las horas en las que comienzan y finalizan la jornada). El objetivo de esta investigación fue examinar los niveles de estrés de los viajeros con horarios flexibles en comparación con los trabajadores con un horario de trabajo fijo. Según dicho estudio, la urgencia del tiempo es un concepto de personalidad relacionado con la percepción que uno tiene del tiempo, y las personas que tienen urgencia de tiempo experimentaron niveles más altos de estrés como resultado de los plazos de llegada y la presión. Dado que los horarios de las jornadas flexibles reducen en gran medida las presiones de desplazamiento, se determinó que los viajeros de horario flexible experimentan menos estrés, menos urgencia de tiempo y por consiguiente niveles más altos de satisfacción en el uso de transporte público diariamente.

Si bien en la actualidad se carece de investigaciones específicas relacionadas a los entornos de las paradas o puntos de arribo de colectivos, se han realizado estudios en el área de psicología y marketing donde se analiza cómo el ambiente influye en la percepción del tiempo. En psicología, Droit-Volet y Gil [4] explican cómo el estado emocional de la persona y el ritmo del ambiente influye en la percepción del tiempo. La investigación de mercados y marketing [5] también establece una influencia considerable por parte del entorno de servicio en el tiempo de espera del cliente a través de la emoción, así como la relación negativa entre el tiempo de espera y la satisfacción del servicio. Por otra parte, Hornik [6] muestra que la gente que dispone de estados de ánimo positivos tienden a subestimar la duración de las actividades, pero en cambio aquellos usuarios con estados de ánimo neutrales o negativos tienden a sobrestimar de forma no saludable. Diferentes investigaciones realizan un análisis de aquellos componentes o factores ambientales que afectan la percepción del tiempo de las personas, cómo por ejemplo la iluminación, la temperatura, la música, la decoración, la organización de los servicios prestados, las distracciones disponibles mientras esperan y las interacciones sociales. Un estudio realizado por Pruyn y Smidts [7] también encontraron que aquellos entornos de espera influyen en las percepciones del tiempo de espera de los clientes y sugieren que los entornos de espera atractivos inducen un estado de ánimo positivo y mejoran la percepción del tiempo de espera.

Por fortuna, hay diversas alternativas que las empresas de transporte público pueden hacer para que la espera impacte en menor medida [8]. La primera opción lógica es ampliar el número de vehículos la cual implica un costo muy alto tanto para las unidades como los empleados. Otra opción válida son los paneles de llegada en tiempo real que pueden aliviar un poco la ansiedad. La posibilidad más accesible para reducir el estrés de los usuarios puede resolverse a través de una aplicación.

2.4.1 Soluciones tecnológicas

La necesidad de desarrollar aplicaciones informáticas que faciliten la realización de tareas a los usuarios se ha convertido en un factor determinante para la mayoría de los diseñadores/desarrolladores. Muchas organizaciones y compañías han incluido en sus proyectos requisitos de usabilidad en sus especificaciones de requisitos de software,

pues han identificado la importancia que representa desarrollar productos "usables" que los ayuden a atraer la mayor cantidad de usuarios a sus aplicaciones.

Es así que la usabilidad es considerada como un atributo de calidad que evalúa qué tan fácil se utiliza un producto y que tan intuitiva es una interfaz gráfica. La palabra usabilidad también se refiere a los métodos para mejorar la facilidad de uso durante el proceso de diseño. Dentro de los factores que determinan la usabilidad podemos mencionar la accesibilidad, legibilidad, navegabilidad, facilidad de aprendizaje, velocidad de utilización y eficiencia del usuario.

Este término además posibilita la medición de la calidad en la experiencia que tiene un usuario cuando interactúa con un producto o sistema. Otra definición del término es que se trata de una medida de la calidad de la experiencia que tiene un usuario cuando interactúa con un producto o sistema. Esto es medido a través del estudio de la relación que se produce entre las herramientas (entendidas por ejemplo, en un sitio Web como el conjunto integrado por el sistema de navegación, las funcionalidades y los contenidos ofrecidos) y quienes las utilizan, para determinar la eficiencia en el uso de los diferentes elementos ofrecidos en las pantallas y la efectividad en el cumplimiento de las tareas que se pueden llevar a cabo a través de ellas.

“La usabilidad se refiere a la capacidad de un software de ser comprendido, aprendido, usado y ser atractivo para el usuario, en condiciones específicas de uso”. [9].

La usabilidad depende no sólo del producto sino también del usuario. Por ello un producto no es en ningún caso intrínsecamente usable, sólo tendrá la capacidad de ser usado en un contexto particular y por usuarios particulares.

“Usabilidad es la eficiencia y satisfacción con la que un producto permite alcanzar objetivos específicos a usuarios específicos en un contexto de uso específico”. [ISO/IEC 9241]

Se extiende al concepto de calidad en el uso, es decir, se refiere a la forma en que el usuario realiza las tareas específicas en escenarios específicos con efectividad.

2.5 Regresión lineal

Dentro de las disciplinas del campo de la inteligencia artificial se encuentra el aprendizaje automático, que se refiere a la capacidad de los sistemas para identificar

soluciones a los problemas de forma independiente a partir de predicciones. El concepto de Machine learning [10] incluye a un conjunto de algoritmos que posibilitan la detección de patrones a partir de un conjunto de datos, y crear estructuras (conocidos como modelos) que permiten representarlos. Cabe destacar que machine learning trabaja y aprende sobre patrones que se encuentran presentes en los datos que se disponen, por lo tanto solo pueden identificar escenarios similares, lo cual en el futuro no siempre ocurre.

La regresión lineal [11] es una técnica de aprendizaje supervisado que se utiliza en Machine Learning. Consiste en un método estadístico que permite resumir y estudiar la relación que existe entre una variable dependiente y una o más variables independientes. Este procedimiento engloba a un conjunto de métodos estadísticos que usamos cuando tanto la variable de respuesta como la(s) variable(s) predictiva(s) son continuas y se busca predecir valores de la primera en función de valores observados de las segundas. En otras palabras consiste en ajustar un modelo a los datos, estimando coeficientes a partir de las observaciones, con el fin de predecir valores de la variable de respuesta a partir de una (regresión simple) o más variables (regresión múltiple) predictivas o explicativas.

Una de las ventajas que presenta la regresión es la amplia variedad de situaciones donde puede adaptarse y aplicarse con el fin de resolver ciertas problemáticas, para ejemplificar dichos usos podemos mencionar:

- Análisis de mercados, con el fin de determinar inversiones eficaces o predecir el número de ventas en el futuro.
- En el ámbito de la física se utiliza para calibrar medidas y caracterizar la relación entre variables de diferentes productos o componentes.
- En la medicina, se han realizado estudios para vincular la reducción de peso de una persona en función del número de días/semanas que ha realizado una dieta determinada.

En resumen la regresión cumple un rol muy importante a la hora de identificar cuáles son las variables predictivas relacionadas con una variable de respuesta, describir qué

forma tiene la relación entre dichas variables y conducir a una función matemática óptima que modele la relación.

2.5.1 Regresión lineal Simple

Dentro del modelo de regresión lineal se encuentra la regresión lineal simple la cuál se basa en la predicción de una variable de respuesta cuantitativa a partir de una única variable predictora cuantitativa, relacionando mediante la siguiente fórmula matemática:

$$Y_i = \alpha + \beta X_i + \epsilon_i \quad i = 1, 2, \dots, N$$

En donde:

- **Y** representa la variable dependiente, lo que significa qué es la variable que estamos interesados en calcular.
- **X** es la variable predictora o independiente, cuyo valor no depende del de otra variable.
- **α** es el punto de corte en el eje de ordenadas. El problema planteado en particular a priori no tiene un impacto real.
- **β** es la pendiente o gradiente de la recta, que son los coeficientes de regresión. En otras palabras corresponde a la cantidad que varía la variable dependiente cuando la variable independiente aumenta en una unidad. Geométricamente indica la inclinación de la recta.
- **ϵ_i** corresponde al término de residuos, que representa la diferencia entre el valor observado y el estimado.

- El subíndice i denota el número de observaciones. En general, el subíndice i será empleado cuando la muestra contenga datos de sección cruzada y el subíndice t cuando tengamos observaciones correspondientes a series temporales.
- N indica el tamaño muestral, número de observaciones que se disponen para realizar el estudio.

Para complementar la información relacionada al ϵ_i , Este error se introduce por varios motivos, entre los cuales se destacan:

- Errores de especificación ocasionados por la omisión de alguna variable explicativa o por las posibles ausencias de linealidad en la relación entre X e Y .
- Errores de medida producidos a la hora de obtener datos sobre las variables de interés.
- Efectos impredecibles, originados por las características de la situación económica o del contexto de análisis, y efectos no cuantificables derivados de las preferencias y los gustos de los individuos o entidades económicas.

La ecuación matemática surge debido a que la relación entre dos variables no siempre es perfecta o nula, de hecho, generalmente no se es ni una ni otra. Un ejemplo visual puede apreciarse en la figura 2.6, en donde se muestra cómo se relacionan las variables precio y superficie. Como se puede observar en dicha figura, a medida que el valor de la superficie (visible en el eje X) se incrementa, el precio de la propiedad tiende a elevarse también (en el eje Y).

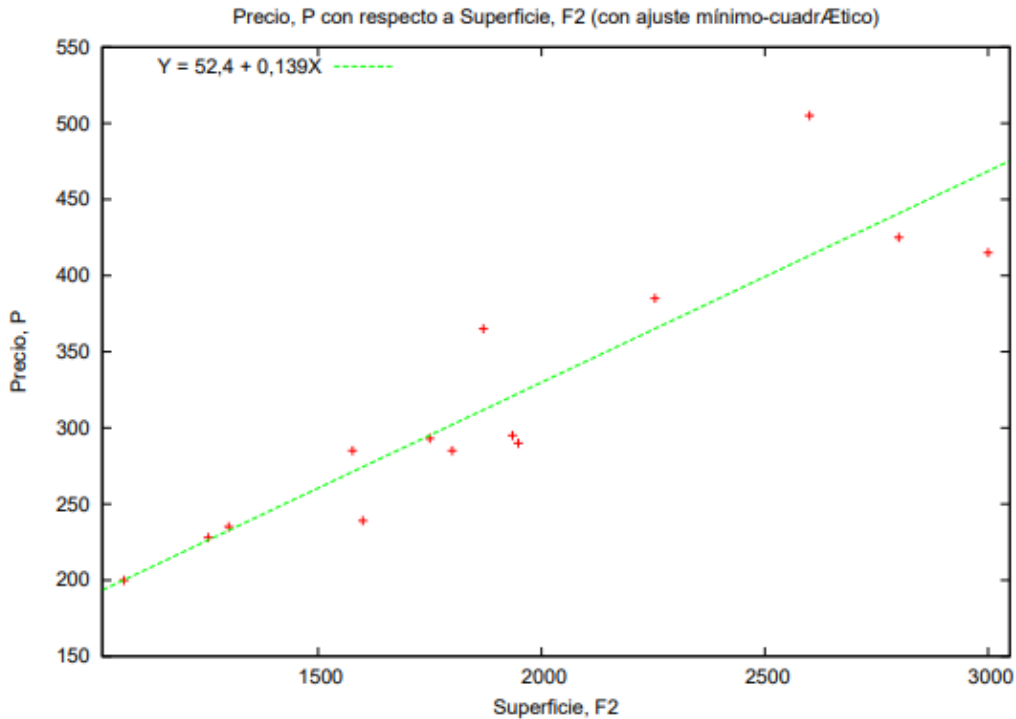


Figura 2.6 Diagrama de dispersión precio-superficie de vivienda (San Diego 1990).

En una situación ideal (aunque prácticamente imposible de suceder en la realidad) en la que todos los puntos de la variable independiente se disponen en una línea recta, no habría necesidad de calcular la recta que mejor resuma los puntos, ya que simplemente uniendo los puntos entre se logra conseguir la recta con mejor ajuste [12]. En un contexto más realista, hay diferentes posibles rectas, pero no todas logran ajustarse de la misma manera a los puntos. El problema radica en encontrar aquella recta capaz de convertirse en la mejor representante del conjunto de valores utilizados.

2.5.2 Regresión lineal Múltiple

Dentro de las regresiones lineales, también se encuentra la regresión lineal múltiple. Al igual que la regresión lineal simple, se dispone de una única variable dependiente, pero en la regresión múltiple, puede haber más de una variable independiente. Dichas variables pueden adoptar dos formas generales [13]:

- Continuas: Consisten en números reales (que pueden tener o no decimales) y siendo de utilidad cuando su rango no sea desde $-\infty$ hasta

$+\infty$. Se trata de variables cuantitativas (cómo por ejemplo la edad o el peso) pero también pueden ser consideradas continuas variables cualitativas cuando pueden ordenarse y tienen un número elevado de elementos.

- Discretas: Las variables discretas son aquellas que se presentan como categorías independientes. Suelen ser factores cualitativos que indican alguna característica del individuo (como el color, idioma, género, entre otros). En el caso de que las características solo dispongan de dos valores se suelen llamar dicotómicas.

En la regresión lineal múltiple se utiliza más de una variable de entrada, lo que ofrece la ventaja de poder hacer uso de más información en la construcción del modelo y por consiguiente conseguir estimaciones más precisas. La fórmula correspondiente a la regresión múltiple es la siguiente:

$$y_j = b_0 + b_1x_{1j} + b_2x_{2j} + \dots + b_kx_{kj} + u_j$$

En donde y es la variable dependiente, x las variables independientes (también conocidos como predictores), u los residuos y b los coeficientes estimados del efecto marginal entre cada x e y . Los coeficientes son determinados de forma que la suma de cuadrados entre los valores observados y los pronosticados sea mínima, lo que significa, que se minimiza el error.

En el modelo lineal múltiple es requerido que los predictores sean independientes, por lo que no debe haber colinealidad alguna entre ellos. Dicha colinealidad ocurre en los casos donde un predictor se encuentra relacionado con uno o varios de los demás predictores que dispone el modelo, o cuando se trate de una combinación lineal de otros predictores. Una de las consecuencias de la colinealidad es que no se permite distinguir de forma exacta el impacto individual que dispone cada una de las variables sobre la variable resultado, lo que implica un aumento en la varianza de los coeficientes de regresión calculados. Además, ante cualquier cambio pequeño en los datos, se provocan grandes cambios en las estimaciones obtenidas.

2.5.3 Estimación por mínimos cuadrados

Existen distintos métodos para ajustar una simple función, donde se busca minimizar una medida diferente del grado de ajuste. Uno de los procedimientos más utilizados es el método de mínimos cuadrados [14], en donde se busca la recta que hace mínima la suma de los cuadrados de las distancias verticales entre cada punto y la recta. Esto implica que, ante múltiples rectas posibles, solamente una posibilita que las distancias verticales entre cada punto y la recta sean mínimas. Este método garantiza encontrar el valor de máxima verosimilitud de los parámetros del modelo lineal para un conjunto de datos, al minimizar la magnitud de la diferencia entre los puntos y la recta. Partiendo de un conjunto de datos muestrales, se cuenta con observaciones Y_i como variables compuestas por dos componentes. El primero de ellos que depende del valor que posea el regresor X_i , cuyo valor es observado. El segundo componente es el valor residual o error que no es tenido en consideración. Teniendo un número N de muestras con una estructura similar.

$$Y_1 = \alpha + \beta X_1 + u_1$$

$$Y_2 = \alpha + \beta X_2 + u_2$$

•

•

$$Y_n = \alpha + \beta X_n + u_n$$

Este sistema puede representarse de manera gráfica cómo se indica en el gráfico 2.7

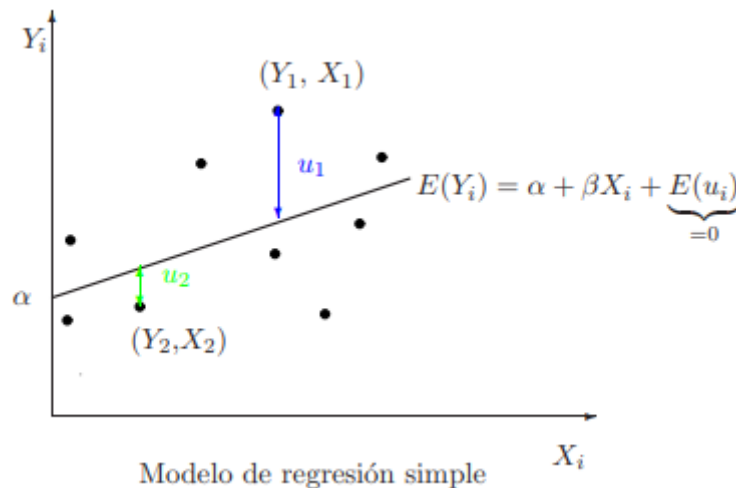


Figura 2.7 Gráfico ejemplo de un conjunto de muestras al azar.

El par (X_i, Y_i) se ubican o distribuyen alrededor de la recta $\alpha + \beta X_i$. Las desviaciones que cada uno de los puntos tenga respecto a esta recta viene dada por el valor que el término de error posea (indicados en el gráfico con u_1 y u_2). Una aclaración que vale la pena mencionar es que las distancias se elevan al cuadrado debido a que al manejar distancias positivas y negativas, se anulan entre sí al realizar la suma. Por ejemplo, en el caso de la primer observación adjunta, u_1 el error es positivo debido a que el valor Y_1 se encuentra por encima de la recta central. En cambio la segunda observación correspondiente al punto (X_2, Y_2) se encuentra de manera inferior a la recta óptima, por lo tanto el error tomaría un valor negativo.

Por lo tanto, la recta central es aquella que se consigue cuando el valor de los errores o perturbaciones es cero. Considerando que la perturbación tiene un promedio de cero (caso muy poco probable), lo que significa que no tiene impacto sobre la variable Y , la recta central posee el mismo comportamiento medio de la variable de interés. La estimación de un modelo de regresión tiene como objetivo calcular y obtener una aproximación a la recta central buscando minimizar la suma de los cuadrados de estos errores.

2.5.4 Métricas

La performance de la regresión puede interpretarse conociendo los valores de error de las predicciones del modelo. Complementariamente, se puede analizar el rendimiento observando de qué manera la línea de regresión calculada puede comprobar qué bien o no se ajusta al conjunto de datos analizados. Un buen modelo de regresión se corresponde cuando la diferencia entre los datos actuales o observados y los valores predichos por el algoritmo, posee valores pequeños.

Para realizar una evaluación de la precisión del modelo se dispone de una serie de métricas específicas [15][16] que permite analizar la performance de un modelo. En primer lugar, uno de los coeficientes más calculados es la correlación de Pearson [17]. Dicho índice mide el grado en que dos variables están relacionadas y se utiliza para medir la fuerza y la dirección de la relación lineal que hay entre dos variables. Este coeficiente se calcula dividiendo la covarianza de las variables por el producto de sus desviaciones estándar y tiene un valor entre +1 y -1, donde 1 es una correlación lineal positiva perfecta, 0 no es una correlación lineal y -1 es una perfecta correlación lineal negativa. En la Figura 2.8 se expone una tabla en donde se indica el grado de intensidad presente entre las variables según cuál es el valor resultante de la correlación.

Valor	Intensidad
1	Perfecta
0,81-0,99	Alta
0,61-0,80	Medio-alta
0,41-0,60	Media
0,21-0,40	Medio-baja
0,01-0,20	Baja
0	Nula

Figura 2.8 Grado de intensidad según la correlación obtenida.

2.5.4.1 Error absoluto medio (MAE)

El error absoluto medio o “MAE” (también conocido como Desviación absoluta media, MAD), es la media de la diferencia absoluta entre los puntos de la salida predicha y los datos reales. Dispone de la misma escala que los datos que se están evaluando. En la figura 2.9 se muestra la fórmula correspondiente a dicha métrica.

Lógicamente cuanto menor sea el valor MAE, mejor va a resultar la performance del modelo. Se trata de una puntuación lineal, lo que implica que todas las diferencias particulares son ponderadas de la misma manera en el promedio.

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

Figura 2.9 Fórmula del Error absoluto medio (MAE)

Vale destacar que este es un método de evaluación más subjetivo que objetivo. Esto se debe a que el valor de error obtenido calculado va a ser mejor o peor en función al rango de valores que disponga nuestro conjunto de datos. Haciendo uso de un ejemplo sencillo, si obtenemos un error de 100 (independientemente de la escala utilizada) el modelo va a ser performante si el conjunto de datos tiene valores entre 1.000 y 1.000.000 por ejemplo. Pero si en cambio disponemos datos entre 1.000 y 1.500 el error es demasiado alto.

Una desventaja de la métrica descrita es que si se realiza y se obtiene una predicción muy mala, el error empeorará aún más y puede sesgar la métrica para analizar el modelo. Este es un comportamiento particularmente problemático si tenemos datos ruidosos (es decir, los datos que por cualquier motivo no son del todo confiables) ya que esta métrica penaliza aquellos errores grandes. Por lo tanto, no es demasiado sensible a valores pocos habituales o atípicos a diferencia del error cuadrático medio.

2.5.4.2 Error cuadrático medio (RMSE)

El error cuadrático medio es una métrica que toma la raíz cuadrada del promedio de las diferencias al cuadrado entre las predicciones del modelo y el valor actual. Expresado en palabras más sencillas consiste en la raíz cuadrada de MSE.

En la figura 2.10 se expone la fórmula correspondiente al RMSE.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Figura 2.10 Fórmula del Error cuadrático medio (RMSE)

Ambas métricas pertenecen a un rango de valores positivos que parte desde 0 y no posee límite superior (infinito positivo), por lo tanto a medida que el valor aumenta, se reduce la performance del modelo, lo que se resume que a menor valor de RMSE, mejor es el modelo.

Dicha raíz cuadrada se introduce para hacer que la escala de los errores sea igual a la escala de los objetivos. Entre las similitudes entre ambos errores, son similares en términos de sus minimizadores, cada minimizador de MSE es también un minimizador para RMSE y viceversa, ya que la raíz cuadrada es una función que no disminuye. Por ejemplo, si tenemos dos conjuntos de predicciones, A y B, y decimos que el MSE de A es mayor que el MSE de B, entonces podemos estar seguros de que RMSE de A es mayor que RMSE de B. Y también funciona en la dirección opuesta.

2.5.4.3 R al cuadrado (R^2)

Utilizando las métricas mencionadas anteriormente, puede resultar difícil identificar si el modelo es bueno o malo analizando los resultados de RMSE o MSE. Por dicha razón el coeficiente de determinación (o R^2), es una métrica que podemos hacer uso para evaluar y analizar el modelo y está fuertemente relacionada con el MSE, pero dispone de la ventaja de no utilizar escala, por lo que no es de importancia si los valores de obtenidos son muy grandes o muy pequeños, debido a que R^2 siempre estará entre $-\infty$ y 1. En caso de disponer de un valor de R^2 negativo, implica que el modelo es peor que predecir la media. En la figura 2.11 se observa la fórmula correspondiente al R^2 y sus componentes.

Coeficiente de Determinación $\rightarrow R^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}$

Suma de Cuadrados Total $\rightarrow SST = \sum (y - \bar{y})^2$

Regresión de Suma de Cuadrados $\rightarrow SSR = \sum (y' - \bar{y}')^2$

Error de Suma de Cuadrados $\rightarrow SSE = \sum (y - y')^2$

Figura 2.11 Fórmula de R^2 y sus componentes con las ecuaciones correspondientes.

Capítulo 3: Enfoques de predicción

3.1 Introducción

En este capítulo se describe cómo se diseñaron e implementaron los algoritmos utilizados para calcular el tiempo del sistema de transporte. Es importante inicialmente dejar en claro cuál es el objetivo a alcanzar, definiendo qué se pretende predecir o calcular, especificando cuáles son los datos que se disponen y cuáles son necesarios conseguir. En este caso, el objetivo a alcanzar es estimar la cantidad de tiempo necesaria para desplazarse entre dos puntos concretos de la ciudad haciendo uso del sistema de transporte público.

Para la resolución de la problemática planteada, se idearon tres alternativas que si bien disponen de conceptos en común, difieren en algunos aspectos. El primer enfoque consiste en iniciar desde los datos de entrada disponibles, modelarlos y llevar a cabo una regresión lineal simple, la cual a lo largo de esta sección se describen todos los detalles teóricos y prácticos correspondientes. El segundo enfoque consiste brevemente en considerar a la ciudad como una matriz y realizar los cálculos necesarios en función a su ubicación dentro de dicha estructura. El tercer enfoque propuesto consiste también en una regresión lineal pero con múltiples parámetros de entrada, en donde se consideran una serie de variables previamente no utilizadas.

Para las diferentes alternativas se proveerá un análisis completo, justificando su elección y la implementación realizada adjuntando los resultados obtenidos.

3.2 Datos disponibles

Para la realización de este trabajo, se parte de un dataset provisto por el ISISTAN en donde se disponía el registro de los viajes llevados a cabo por la línea de colectivos número 500 (correspondiente al color amarillo). Dicho conjunto de datos se

corresponde a datos con fecha de diversos meses entre los años 2017 y 2019, dentro del horario laboral que tiene el sistema de colectivos de Tandil, el cual corresponde generalmente entre las 7 am hasta las 23 pm (sujeto a modificaciones por parte del municipio). Los datos presentes en el dataset disponen de las siguiente propiedades:

- Línea de colectivo: Expresado en formato numérico. Durante el desarrollo de este trabajo la ciudad de Tandil dispone de seis líneas representadas por un determinado color y número entre 500 y 506. A la hora de realizar las operaciones este valor es siempre constante debido a que se opera con una sola línea.
- Id: Indicando a la unidad de colectivo de la línea correspondiente. También de tipo numérico dentro de un rango entre 1 y 13.
- Tiempo del request: Marca temporal en el que se desea recabar la información del servicio. Indicando la fecha y horario correspondiente pero en un formato que más adelante especificaremos.
- Tiempo del colectivo: Marca temporal registrada por el colectivo. Mismo formato que la propiedad anterior.
- Velocidad: Velocidad instantánea de la unidad (medida en km/h) correspondiente a la marca temporal registrada.
- Posición: Par de coordenadas de latitud y longitud señalando la ubicación en el mapa en un instante determinado. Ambas propiedades se encuentran en tipo float, en donde a mayor número de dígitos mayor resulta la precisión a la hora de ubicar.

```
[
  {
    "linea":500,
    "id":1,
    "tiempo_request":1508079422879,
    "tiempo_colectivo":1508079360000,
    "velocidad":38.0,
    "latitud":-37.29446,
    "longitud":-59.150047
  },
  {
    "linea":500,
    "id":1,
    "tiempo_request":1508079426821,
    "tiempo_colectivo":1508079360000,
    "velocidad":42.0,
    "latitud":-37.292927,
    "longitud":-59.151985
  },
  { }
]
```

Figura 3.1: Ejemplo extraído del dataset original para visualizar los datos utilizados

Es importante remarcar que el tiempo se disponía en formato timestamp, lo cual representa una secuencia de caracteres, que se corresponden a una hora y fecha en la cual sucedió el evento. Más precisamente el timestamp se refiere a la cantidad de segundos transcurridos desde el estándar UNIX Epoch⁷ (00:00:00 UTC del 1 de enero de 1970).

En primer lugar, antes de cualquier operación sobre el conjunto de datos, se optó por organizar los datos bajo un criterio en específico, es por eso que se llevó a cabo un ordenamiento completo del dataset por el campo del Tiempo del request. Esto permite tener un registro secuencial de la sucesión de datos a lo largo del tiempo. Esta medida es muy importante para prevenir resultados incoherentes e inexactos con la realidad. En la figura 3.2 se expone un histograma en donde se observa la cantidad de datos disponibles agrupados según el valor de fecha que disponga diferenciados por año y mes.

⁷ https://es.wikipedia.org/wiki/Tiempo_Unix

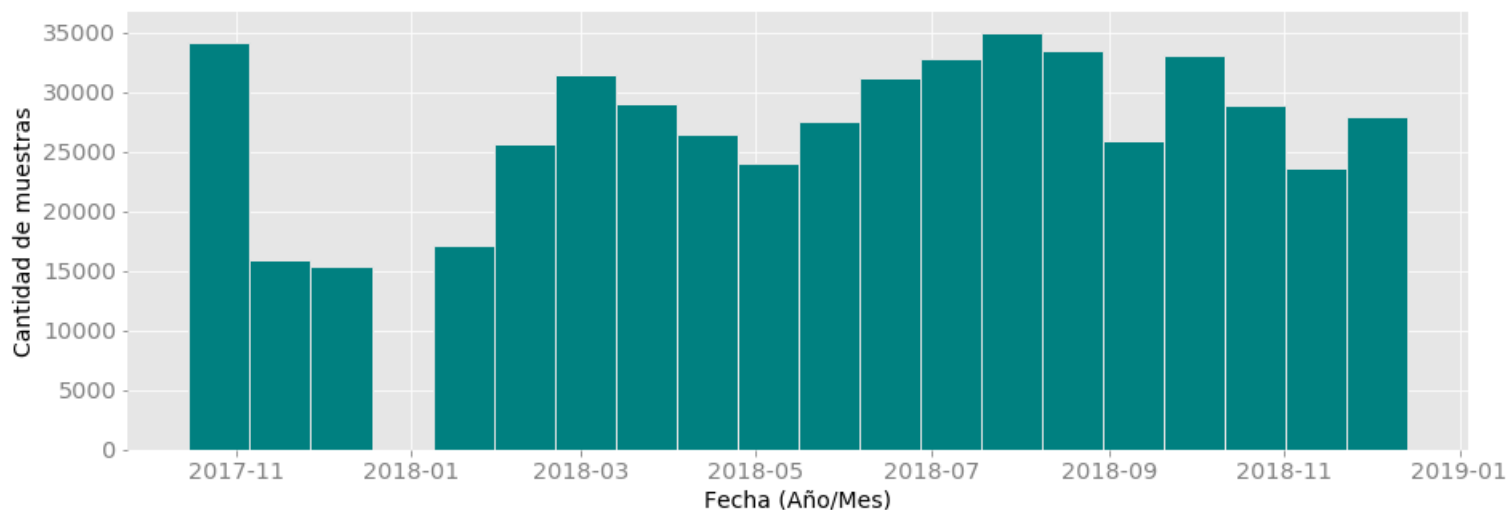


Figura 3.2 Diagrama de la distribución de los datos por fecha de la línea 500, unidad 1.

3.3 Enfoque de Regresión lineal simple

Una de las principales ventajas del modelo escogido es la sencillez a la hora de realizar la implementación y su flexibilidad para trabajar. Para trabajar con esta técnica es preciso definir cuáles iban a ser las variables a utilizar y cuál iba a ser el rol de cada una de ellas.

Recordando la fórmula descrita anteriormente:

$$Y_i = \alpha + \beta X_i + \epsilon_i \quad i = 1, 2, \dots, N$$

En este primer caso de prueba se decidió seleccionar como variable independiente la distancia y como variable dependiente el tiempo. A modo de resumen, el modelo busca predecir la cantidad de tiempo aproximada que va a tomar en este caso la unidad de colectivo en realizar una cierta longitud.

- Y representa la variable dependiente, en este caso el tiempo.
- X es la variable predictora o independiente, cuyo valor representa la distancia.

Vale destacar que las unidades a utilizar no están definidas aún en esta etapa, sino que van a ser determinadas posteriormente durante la implementación del algoritmo.

Lógicamente representa una relación lineal simple debido a que al aumentar la distancia que se debe recorrer, esto implica un incremento en menor o mayor medida en el tiempo que conlleva realizar dicho recorrido. Cómo se mencionó anteriormente este modelo supuso un inicio donde se busco obtener resultados de la manera más sencilla e ignorando los factores reales que afectan a las variables mencionadas. Es por eso que en este modelo no se tienen en cuenta por ejemplo, las zonas donde se transita, reconociendo que no es lo mismo realizar la misma distancia en la zona céntrica que en zonas de las afueras de la ciudad. Tampoco se tuvo en cuenta el día de la semana del dataset, aclarando que la frecuencia de los colectivos no es igual en días semanales que en días sábados y domingos.

En la siguiente sección se realiza una descripción de los pasos realizados para la preparación y configuración de la información disponible con el objetivo de implementar el modelo descrito en la etapa anterior. Detallando la justificación de las decisiones tomadas y mencionando los inconvenientes que surgieron a lo largo de esta etapa y qué acciones se efectuaron para resolverlos.

3.3.1 Presentación de los datos

Para la implementación de este caso de prueba se decidió trabajar con tres conjuntos de datos del mismo formato (mismas columnas y mismo tipo de datos correspondiente) pero de diferente unidad. Se disponen tres dataset correspondientes a las unidades 1, 2 y 3 de la línea 500. Esto permite analizar y comparar los resultados obtenidos entre recorridos iguales pero realizados desde diferente transporte.

3.3.2 Reestructuración de la información

Una vez establecida y definida la estructura seleccionada del dataframe a utilizar en la regresión especificando sus variables independientes y dependientes, el siguiente paso dentro de la etapa de preprocesamiento es la limpieza de datos para la definición del modelo de análisis.

Debido a que en el proceso de la regresión lineal simple solo se dispone de dos variables (una dependiente y otra independiente) se debió tomar la decisión de qué datos serían utilizados para la implementación y cuáles iban a ser aquellos que no se iban a tener en cuenta. Al no disponer de las variables buscadas de manera directa, se implementaron una serie de medidas sobre el conjunto de datos con el fin de procesar los mismos de la manera más eficiente y correcta posible.

La representación del dataset inicialmente obtenida da el contexto del problema a analizar, pero para lograr una mejor aproximación a un modelo listo para ser estudiado, ejecutado y que brinde la mejor información, es necesario evaluar individualmente cada variable y así definir qué valores generan una disociación de los datos. Una revisión manual del conjunto de datos es importante para evitar errores en el análisis de los datos y en el proceso de modelado, y se realizan en base a la exploración y comprensión del contexto y dominio del problema.

A pesar de los grandes esfuerzos en la recolección de un gran número de muestras, sujetas a diversas variables, existen algunos factores exentos de su distinción previos a su captura.

Alguno de esos casos son aquellos datos capturados durante:

- Horario de descanso o pausa de la unidad.
- Circulación de la unidad de transporte en una zona fuera de su recorrido habitual y preestablecido.
- Trackeo de status de la unidad fuera del horario laboral definido.
- Pérdida de información durante el cálculo o almacenamiento de valores.
- Corrupción en la transmisión o el almacenaje.

Para la implementación de la regresión se aplicó una serie de medidas para obtener los datos para el modelo. El primer paso consistió en eliminar del dataset inicial la columna relacionada a la línea de colectivo utilizada. La justificación es que se buscó aplicar el algoritmo inicialmente a una línea de colectivos única para poder implementar el modelo y realizar un análisis completo de manera más sencilla para luego una vez hecho esto, poder replicarlo en resto las otras líneas del sistema de transporte de ciudad independientemente del recorrido de cada una de ellas.

La segunda decisión tomada con respecto al refinamiento de los datos consistió en la eliminación de la columna que representaba la unidad de transporte correspondiente a la tupla de datos. Las razones para disponer esta regla es que se buscó implementar un modelo simple y general sin utilizar una unidad en específico. También se consideró que el sistema de colectivos de la ciudad dispone una serie de unidades que son muy similares o iguales entre sí por lo tanto no habría diferencia a nivel de vehículo a la hora de realizar el recorrido, por lo que en teoría no habría impacto en los resultados a calcular.

La tercera medida implementada consistió en eliminar el campo relacionado a la velocidad. Esto se debe principalmente a dos razones.

- La velocidad indicada en el dataset corresponde a la velocidad que poseía el colectivo en ese instante, lo cual no es representativa ya que por ejemplo si el valor cero podría ser por qué el colectivo estaba en un semáforo o con ascenso/descenso de personas, o en otro caso no se encontraba en servicio. Esto significa que el dato en singular no era representativo ni útil, sino que la velocidad debe calcularse teniendo en cuenta los movimientos pasados y futuros.
- La velocidad es una propiedad que no va a ser utilizada de manera directa o indirecta en la implementación de la regresión lineal simple, por qué no debe ser incluida en los datos a analizar.

La cuarta medida corresponde a qué tiempo considerar, debido a que el dataset dispone del tiempo de request y el tiempo del colectivo. A partir del análisis se decidió descartar el tiempo del colectivo debido a que los valores indican que dicha variable no se actualizaba con la frecuencia correspondiente. A modo de ejemplo, en muchas ocasiones, en la información histórica para el mismo tiempo se disponía de tres ubicaciones distintas, cómo se puede observar en la figura 3.3. Es por eso que se decidió por considerar el tiempo de request para realizar los cálculos pertinentes, realizando la conversión de timestamp a formato fecha previamente mencionada.

```
[
  {
    "linea": 500,
    "id": 1,
    "tiempo_request": 1508079506867,
    "tiempo_colectivo": 1508079480000,
    "velocidad": 25.0,
    "latitud": -37.291195,
    "longitud": -59.15708
  },
  {
    "linea": 500,
    "id": 1,
    "tiempo_request": 1508079521863,
    "tiempo_colectivo": 1508079480000,
    "velocidad": 11.0,
    "latitud": -37.291874,
    "longitud": -59.158077
  },
  {
    "linea": 500,
    "id": 1,
    "tiempo_request": 1508079546821,
    "tiempo_colectivo": 1508079480000,
    "velocidad": 24.0,
    "latitud": -37.291145,
    "longitud": -59.158997
  },
  {
    "linea": 500,
    "id": 1,
    "tiempo_request": 1508079561821,
    "tiempo_colectivo": 1508079480000,
    "velocidad": 24.0,
    "latitud": -37.291145,
    "longitud": -59.158997
  }
]
```

Figura 3.3 Ejemplo de valores de tiempo repetidos, pero distinta ubicación.

Cómo se mencionó previamente, una de las variables a utilizar es la distancia, pero dicho dato no se encuentra de manera explícita en el dataset provisto, sin embargo, para obtenerlo podemos hacer uso de las coordenadas registradas. Para llevar a cabo esto se toman un par de datos consecutivos y se calcula la diferencia en metros entre las coordenadas registradas. Pero una de las grandes dificultades que se presentó a la hora de realizar la estructuración de los datos se originó en el momento de calcular la distancia entre dos coordenadas pero para resolver dicho problema se empleó la fórmula de Haversine previamente explicada en la sección 2.2. Otra cuestión muy

importante es que resulta que las coordenadas presentes en el dataset, no siguen un recorrido lineal. La fórmula nos otorga la distancia lineal entre dos puntos, pero en el uso diario, el recorrido del colectivo contempla giros recurrentes en muchas esquinas de la ciudad, lo que implica que la ecuación no contemple dichos casos. Un ejemplo sencillo puede apreciarse en la Figura 3.4.

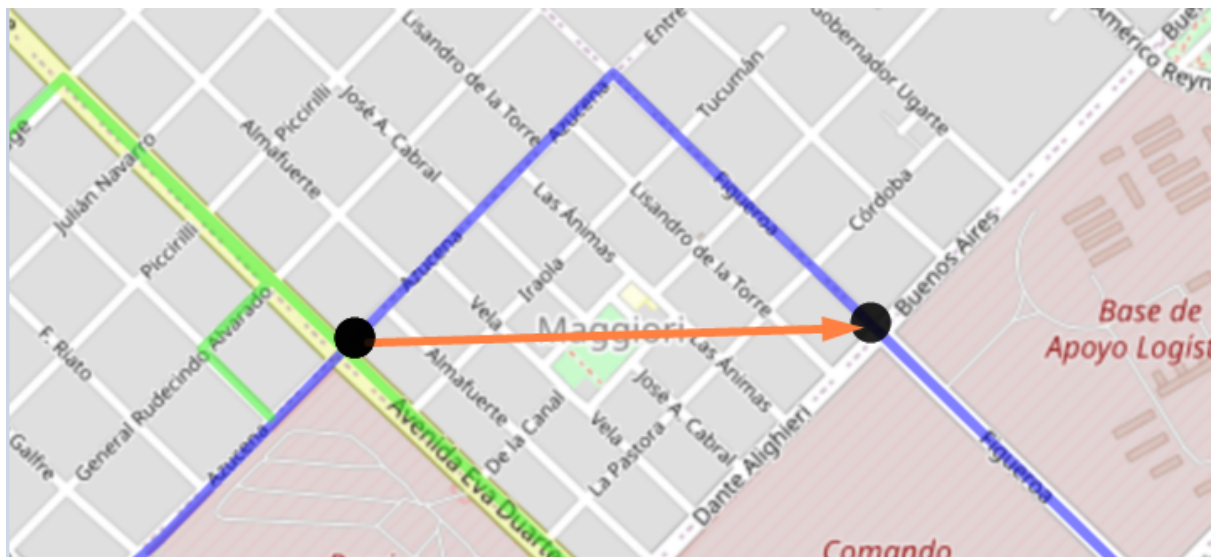


Figura 3.4 Imagen correspondiente a una porción de recorrido de la línea 500.

Los puntos en negro representan valores del dataset, la flecha de color naranja indica la distancia calculada por la fórmula empleada, cuando realmente la distancia recorrida por el colectivo se corresponde con el recorrido destacado con color azul. Visualmente puede apreciarse cómo las distancias no coinciden. Es por eso que la precisión del modelo puede verse comprometida circunstancialmente.

De manera similar, con respecto al tiempo, se toman un par de datos consecutivos y se calcula la diferencia entre las fechas.

Finalmente, luego de los cambios realizados previamente, el dataset resultante consiste de dos columnas:

- **Tiempo:** valor numérico expresado en unidad de segundos.
- **Distancia:** valor numérico en unidad de metros.

Luego de aplicar las medidas previamente descritas sobre los conjuntos de datos, se realizó un análisis relacionado a la tendencia central, la dispersión y la forma de la

distribución del conjunto de datos. En la figura 3.5 se expone la cantidad y el formato de datos utilizados para cada una de las unidades utilizadas.

Tiempo Distancia			Tiempo Distancia			Tiempo Distancia		
0	3.942	242.024550	0	19.985	127.610640	0	110.166	249.499070
1	20.125	34.774185	1	25.219	127.060720	1	15.125	4.006848
2	25.031	43.153088	2	20.234	108.874810	2	29.937	119.470930
3	14.969	71.638800	3	29.578	146.830860	3	19.985	97.137344
4	20.016	168.257690	4	5.047	83.653305	4	45.109	117.375810
...	...	...	...	...	...	...	...	...
518229	40.050	129.990020	440387	20.641	162.068390	388539	9.969	166.764710
518230	45.019	119.647896	440388	24.469	184.411120	388540	25.469	27.052847
518231	25.018	72.594765	440389	19.937	201.415680	388541	19.578	29.085768
518232	15.017	148.752000	440390	20.000	30.372343	388542	85.359	27.272861
518233	3868.710	74.580830	440391	3.942	53.548862	388543	65.133	137.034410

Unidad 1, 518234 datos

Unidad 2, 440392 datos

Unidad 3, 388543 datos

Figura 3.5 Dataset utilizados especificando la cantidad de datos.

Con respecto a las unidades 1, 2 y 3 estudiadas, se dispone una gran cantidad de elementos para implementar la regresión. Posteriormente se construyeron una serie de gráficos para observar la distribución y dispersión que presentan los datos. A modo de ejemplo, la figura 3.6 expone la dispersión de los datos de una de las unidades utilizadas.

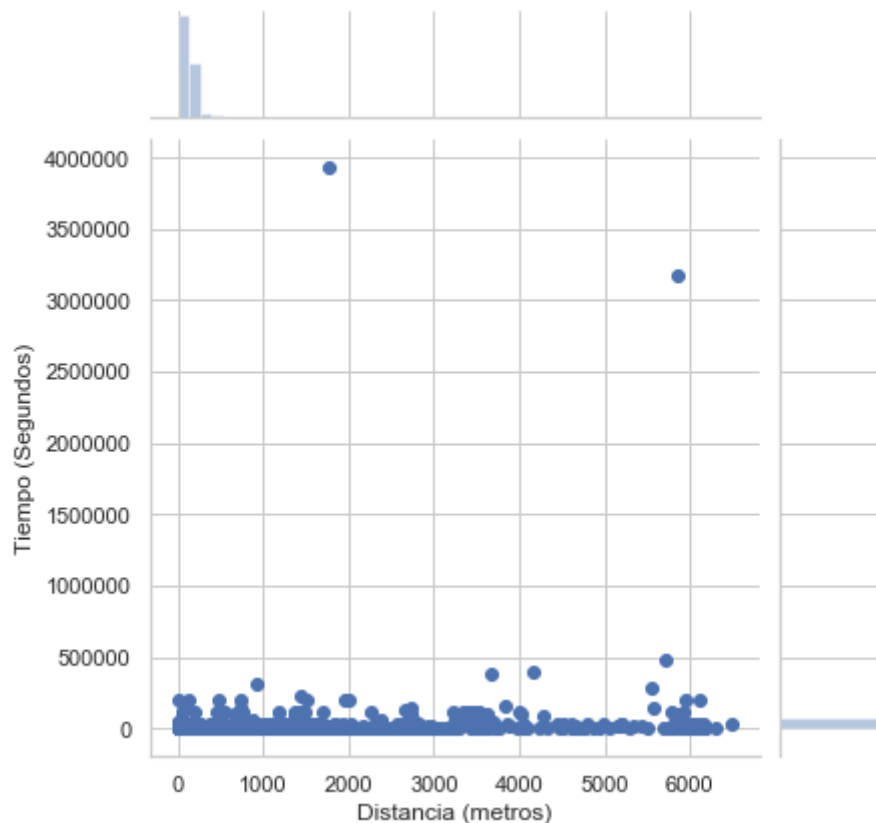


Figura 3.6 Distribución de los datos por tiempo y distancia de la unidad 2, línea 500.

Cómo se puede apreciar en la Figura 3.6, se halló una serie de valores inconsistentes en todas las unidades analizadas que se consideró que su inclusión en el modelo de regresión podría afectar negativamente en los resultados, debido a que sus valores no coinciden a algo cotidiano en el sistema de transporte y que probablemente consista en un valor capturado durante la finalización de un turno y el inicio del siguiente luego de una pausa o algún inconveniente en el aparato utilizado para registrar la información. A modo de ejemplo, se dispone una tupla que presenta un tiempo de alrededor de 4000000 segundos (más de 40 días) lo que claramente indica una ausencia de medición de datos durante ese periodo.

Dichos valores atípicos representan una minoría con respecto a los datos restantes, es por eso que se tomó la decisión de excluirlos completamente para este caso de pruebas.

3.3.3 Refinamiento de los datos

Con el fin de obtener unos resultados más precisos y coherentes, se realizó la implementación de una serie de filtros sobre el conjunto de datos para eliminar aquellos que contengan datos erróneos e inverosímiles y puedan provocar un análisis incorrecto. Para esto se pensó en trabajar con las regiones con mayor concentración de datos en distintos conjuntos de regiones. La separación en regiones permite lograr una separación por una característica definida y así tener para cada una de ellas dos tipos de análisis diferentes. También se eliminó completamente aquellos datos particulares que tuvieran alguna propiedad con ausencia de valor para evitar posibles problemas futuros a la hora de la implementación.

El primer paso para la definición de los conjuntos es descartar los valores que presentan la relación valor mínimo y máximo por tupla, es decir, aquel valor que sea demasiado pequeño y su valor para la otra variable sea demasiado grande. Para ello se procede a listar los filtros realizados con su justificación:

1. Distancia igual a cero metros

Esto representa cuando el colectivo no se ha desplazado en una cantidad de tiempo, por lo tanto se encuentra detenido. Es por eso que se optó por no incorporar estos valores ya que no representa utilidad en este problema.

2. Distancia mayor a un kilómetro (1000 metros)

Estos valores se optó por ignorarlos ya que al realizar una distancia importante incrementa considerablemente las probabilidades de error en los cálculos.

3. Tiempo mayor a 1 hora (3600 segundos).

Este tiempo representa datos inválidos para nuestro uso, ya que el objetivo es realizar una aproximación dentro de una continuidad de un recorrido. Por otro lado, este dato representa los tiempos de parada y descanso, datos que no se tomarán en cuenta para la regresión.

Luego de aplicar los filtros previamente mencionados se puede apreciar una distribución muy dispersa de la información, que se puede apreciar en la figura 3.7, que luego se validará con las métricas correspondientes.

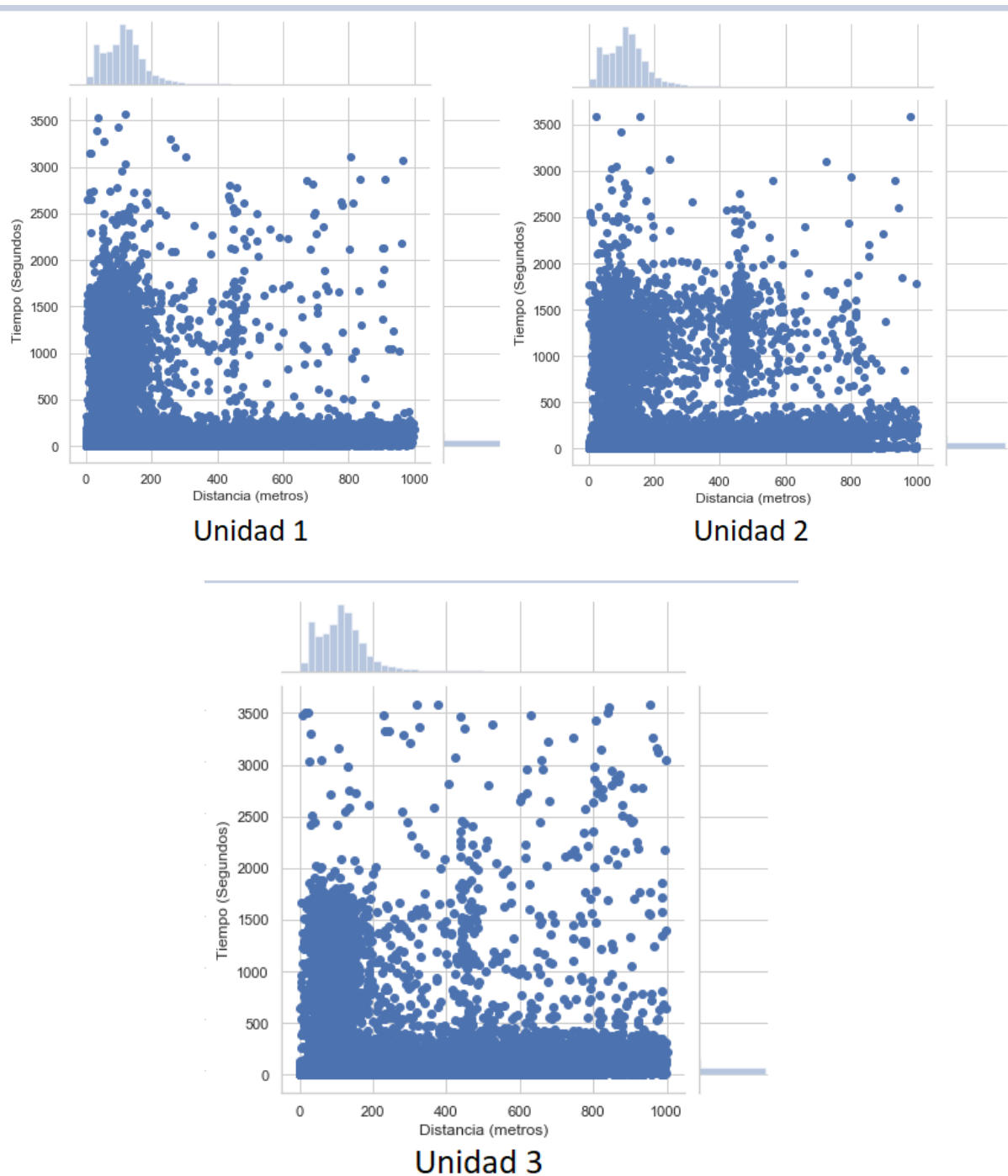


Figura 3.7: Diagrama de la distribución de los datos de las unidades trabajadas.

Para analizar la distribución del dataset de entrada, se separó la información en cuartiles. Los cuartiles [18] son valores que dividen al conjunto de datos en cuatro partes de igual tamaño. Haciendo uso de esta técnica, se puede realizar una

evaluación y analizar de manera sencilla la dispersión y la tendencia central de los datos, en donde los cuartiles se corresponden a:

- Primer cuartil (Q1): 25% de los datos es menor que o igual a este valor.
- Segundo cuartil (Q2): La mediana. El 50% de los datos es menor que o igual a este valor.
- Tercer cuartil (Q3): 75% de los datos es menor que o igual a este valor.
- Rango intercuartil: La distancia entre el primer primer cuartil y el tercer cuartil ($Q3-Q1$); de esta manera, se abarca el 50% central de los datos.

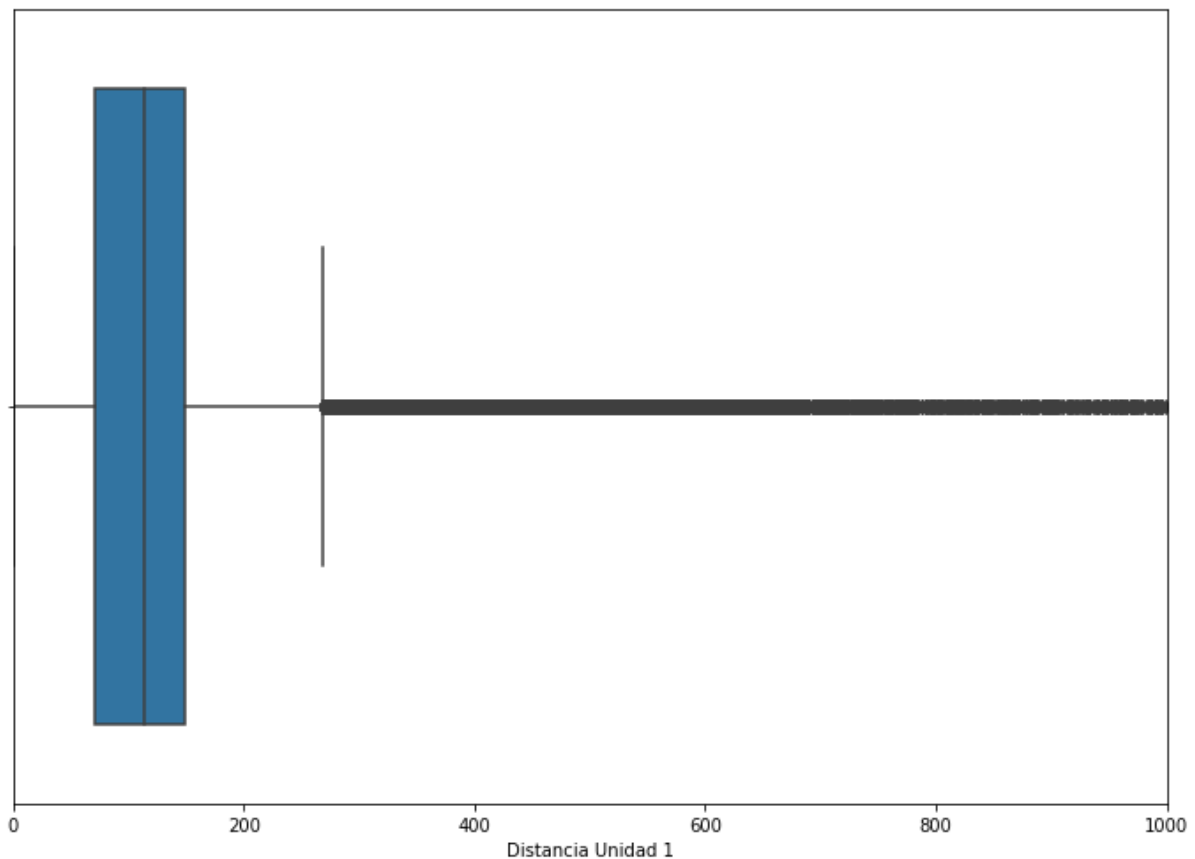


Figura 3.8 Cuartiles Distancia Unidad 1.

Cómo se puede apreciar en la Figura 3.8, cada caja representa los cuartiles en los que se encuentran los valores de la distancia. Los extremos representan el grueso de la distribución y por último, los puntos detectan los outliers, que son aquellos datos con números muy bajos o elevados y que quedan fuera del rango de valores más comunes del dataset. Una vez listo este primer paso en la limpieza de los datos, se continuó con la

implementación en posteriormente se presentan los resultados obtenidos y las conclusiones obtenidas.

3.4 Enfoque de Matriz

Para el segundo enfoque propuesto se decidió optar por la utilización de una matriz en donde representaremos todo el mapa de la ciudad de Tandil, permitiendo analizar los datos desde una perspectiva diferente.

A lo largo de esta sección se detallarán las decisiones tomadas para llevar a cabo la implementación del caso de prueba y sus correspondientes análisis estadísticos para realizar una evaluación de la performance de dicho método. También se hará una descripción acerca de las diferencias con respecto al caso de prueba inicial, indicando cuales son los aspectos a mejorar y cuales son para tener en cuenta nuevamente.

3.4.1 Marco Teórico

Realizando un análisis de la metodología utilizada en el caso de prueba anterior, podemos destacar que ciertas decisiones tomadas no fueron las apropiadas o ideales para el problema y que era necesario un cambio de enfoque. Esto quedó evidenciado con los resultados de las métricas obtenidas. Uno de los problemas principales del primer caso de pruebas planteado, radica en el uso de la función que a partir de dos coordenadas concretas, retorna los metros de distancia que hay entre dichos puntos. En resumen, esto se debe a que la precisión de los metros es muy fina y al realizar algún redondeo o algún truncamiento en algún dígito de las coordenadas, se pierde la exactitud y al tratarse de distancias tan pequeñas, el impacto de las consecuencias es mayor. También hay que considerar que dicha fórmula no se adapta al recorrido fijo que posee la línea, lo cual implicaba un impacto importante en los cálculos. Esto se puede apreciar claramente en la Figura 3.9.

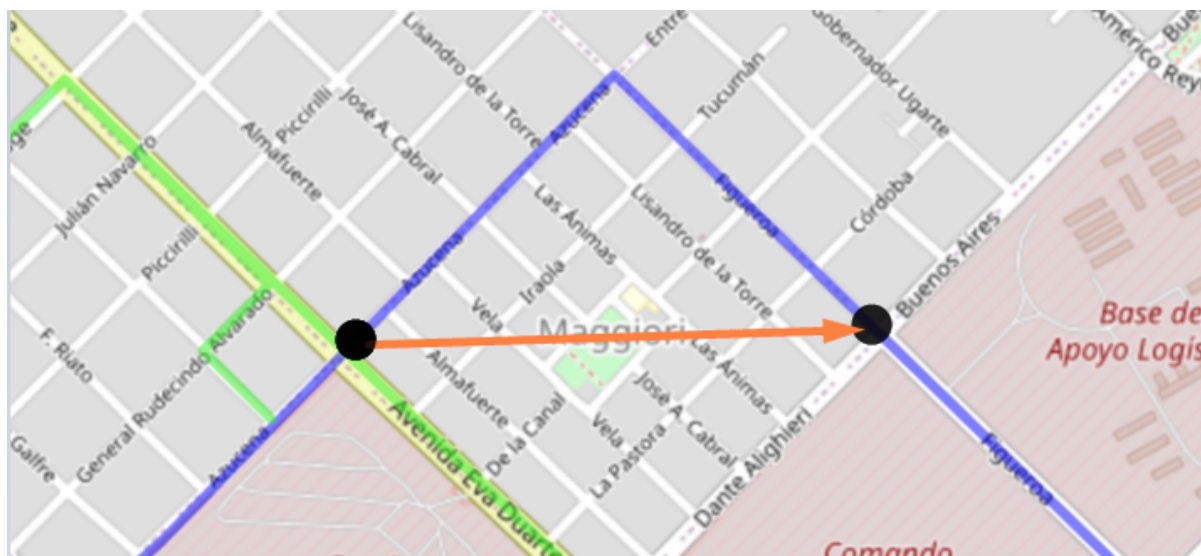


Figura 3.9 Imagen correspondiente a una porción de recorrido no lineal de la línea 500.

Cómo se puede apreciar, el resultado obtenido, no tiene en consideración las esquinas y los cambios de sentido que se realizan durante el trayecto.

Para este caso de pruebas, se optó por dividir la ciudad en una matriz, de tal manera que cada celda sea aproximadamente de 120-130 mts x 120-130 mts. Con el objetivo de que cada celda se ajuste y represente de manera aproximada a una manzana de la ciudad ya que en la mayoría de las situaciones disponen de ese tamaño. En dicha matriz, se incluyen no solamente todas las calles donde el sistema de colectivos circula sino también la mayoría de Tandil.

Si bien la matriz considera regiones donde no hay tránsito, ya que no hay calles ni avenidas presentes particularmente en algunos sectores por fuera de la zona céntrica de Tandil, se decidió realizar un trazado extenso debido a que la ciudad se encuentra en continuo crecimiento y expansión. Entonces de tal manera no habría inconvenientes en el futuro por si hay apertura de nuevos caminos en alguna de las zonas en particular.

Para este caso de pruebas, también se utilizó el mismo conjunto de datos que el primero, un histórico de la información disponible correspondiente a la línea 500. Debido a que esa línea dispone de dos recorridos, donde ciertos tramos del camino coinciden y otros no, por lo que se procedió a separar nuestro dataset según el recorrido que corresponda. En otras palabras consiste en realizar un trackeo de los puntos de la ciudad donde circula la línea de colectivo, registrándolos como una lista de coordenadas. Como la ciudad de Tandil dispone para una misma línea, al menos dos recorridos

distintos (con ciertos puntos en común claro), se procedió a clasificarlos de manera separada. A partir de la utilización de los recorridos bases, nos permitió separar nuestro dataset e identificar a qué recorrido pertenecen.

3.4.2 Preprocesamiento

En esta sección se realizará una descripción de las elecciones tomadas para llevar a cabo el caso de prueba. Partiendo desde el conjunto de datos inicial y las medidas adoptadas para implementación del predictor de tiempo. Justificando el modelado de la información y la selección de datos para luego evaluar el rendimiento a través de una serie de estimadores.

3.4.2.1 Presentación de los datos

La primera medida adoptada, fue la de dividir el conjunto de datos en diferentes segmentos según el tiempo que posea. Adicionalmente se tuvo en cuenta los días del fin de semana, donde el flujo de colectivos se reduce. Cabe destacar, que al momento de realizar este trabajo, en el horario nocturno posterior a las 23 horas, el servicio de transporte de la ciudad no circula. Esto permitió realizar una buena distribución de los datos a través de franjas horarias, de dos horas de rango en caso de días de semanas y de cinco horas los fines de semanas. Las franjas consideradas son las siguientes:

De lunes a viernes:

- Desde las 0 hs. hasta las 5 hs.
- Desde las 5 hs. hasta las 7 hs.
- Desde las 7 hs. hasta las 9 hs.
- Desde las 9 hs. hasta las 11 hs.
- Desde las 11 hs. hasta las 13 hs.
- Desde las 13 hs. hasta las 15 hs.
- Desde las 15 hs. hasta las 17 hs.
- Desde las 17 hs. hasta las 19 hs.

- Desde las 19 hs. hasta las 21 hs.
- Desde las 21 hs. hasta las 24 hs.

Sábado y Domingo:

- Desde las 0 hs. hasta las 5 hs.
- Desde las 5 hs. hasta las 12 hs.
- Desde las 12 hs. hasta las 17 hs.
- Desde las 17 hs. hasta las 24 hs.

Si bien en algunas franjas no se dispone de información, se las considera por si en algún futuro hay datos correspondientes a ese horario en particular.

Cómo se mencionó previamente, al igual que el caso de pruebas número uno, se decidió trabajar con los datos correspondientes a única línea y unidad ya que permite realizar las pruebas con mayor sencillez.

A continuación escogimos cuatro puntos de la ciudad para tomar como vértices de la región, de tal manera que formen un rectángulo que incluya dentro todos los puntos posibles de la ciudad donde circulen (o podría circular en un futuro) los colectivos. Los puntos considerados son los siguientes:

- Vértice superior izquierdo: -37.285083, -59.207464.
- Vértice inferior izquierdo: -37.359192, -59.207464.
- Vértice superior derecho: -37.285083, -59.069919.
- Vértice inferior derecho: -37.359192, -59.069919.

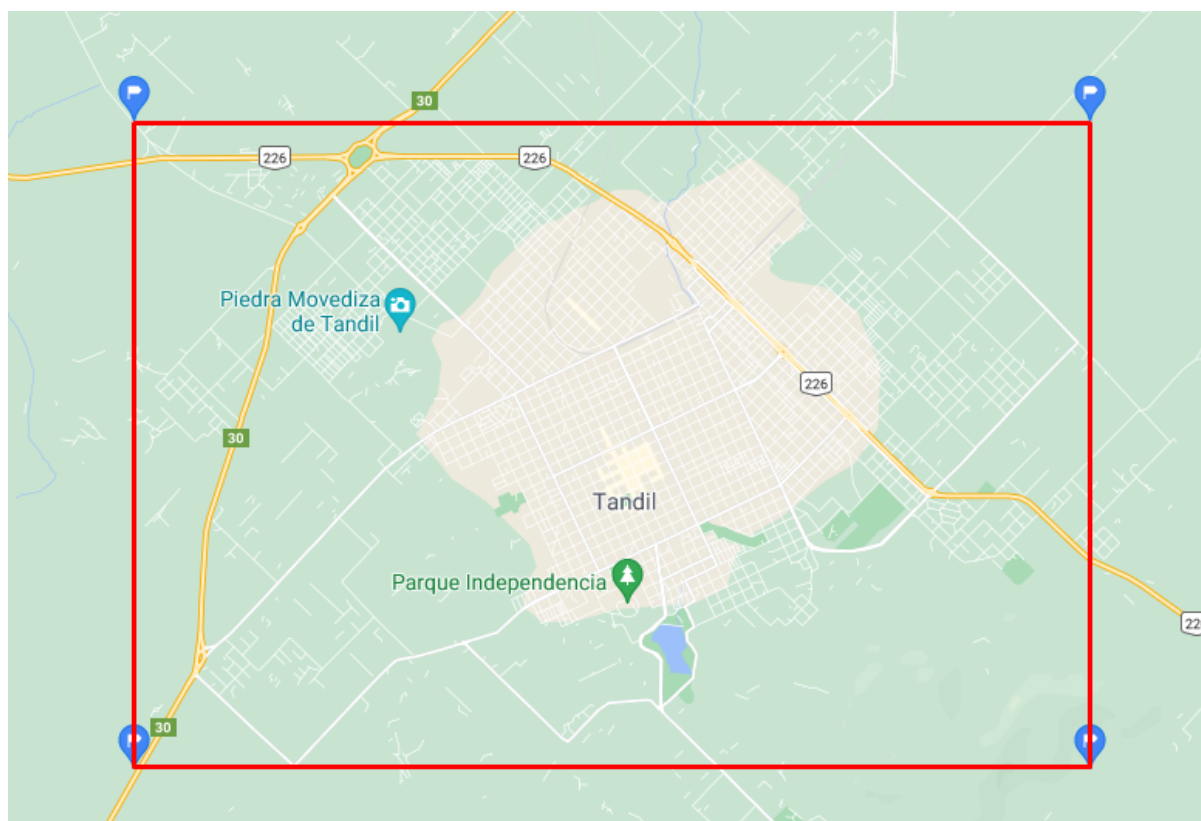


Figura 3.10 Área de la ciudad de Tandil contemplada para el caso de prueba

A partir de esos cuatro vértices conformamos una zona rectangular que engloba todas las calles de la ciudad. Posteriormente se dividió la región en forma de matriz con filas y columnas tomando una cierta medida de distancia entre cada una, de aproximadamente de 120-130 metros de cada lado. Esto permite tener la ciudad dividida en zonas de igual tamaño, representando una celda de la matriz. Es importante aclarar que las calles de la ciudad no están predispuestas de manera simétrica, si no que varía bastante entre cada barrio de la ciudad y además tampoco tenemos en cuenta casos de peajes o rotondas.

3.4.2.2 Reestructuración de la información

Una de las primeras medidas tomadas, consiste en ubicar cada una de las esquinas (representadas a través de una coordenada) que componen los recorridos a analizar, en su correspondiente celda de la matriz. Esto permitió obtener cuales son las celdas, y por lo tanto qué partes de la ciudad, que el colectivo recorre cada vez que realiza a lo largo de sus viajes. De esta manera, se generó un conjunto de listas que contienen las ubicaciones por las que circula un recorrido y una línea en particular. A

dichas listas, las llamaremos recorridos bases. Las listas disponen una sucesión de pares <Latitud, Longitud> en donde los valores incluidos disponen de una distancia de aproximadamente de entre 100 y 200 metros entre sí, asegurando una continuidad similar a la que realiza día a día el colectivo cuadra a cuadra. A partir de la documentación de dichos valores, esto nos permite generar las listas con el par <Fila, Columna> por donde cada línea de colectivo transita diariamente.

Cómo se mencionó anteriormente, es importante destacar que hay partes de los recorridos que se encuentran presentes en ambos recorridos de la misma línea, lo que conlleva a tener celdas compartidas entre ambos. De hecho, también hay ciertas líneas que comparten tramos específicos durante su recorrido.

3.4.2.3 Refinamiento de los datos

Al tener diferenciado los puntos de los posibles recorridos de la línea a utilizar, se prosiguió en separar el dataset, dependiendo justamente de qué recorrido pertenecen. Para llevar a cabo esto, se ubico a cada entrada del conjunto de datos original, en la celda de la matriz que pertenece dependiendo su ubicación geográfica, haciendo uso de sus coordenadas. Al tener la celda de la matriz correspondiente se determinó a cuál recorrido pertenece. A pesar de que para ciertos casos la metodología cumple sin inconvenientes surgen problemas para resolver en aquellas celdas que son compartidas por más de un recorrido por lo que debimos pensar en una solución. Es por eso que para aquellos valores de matriz compartidos consideramos aquellos valores anteriores y posteriores contiguos calculados del dataset para poder establecer la dirección del recorrido.

Finalmente se optó por realizar el almacenamiento de las información en una estructura mucho más adecuada y apta para realizar los cálculos pertinentes. Es por eso que se realizó un esquema de base de datos que dispone de varias tablas que permiten almacenar la información de manera más legible y extensible posible.

- Recorrido: Indicador relacionado al recorrido base que realiza.
- Unidad: Indica que los datos se corresponden a una unidad de colectivos específica, ya que cómo se mencionó anteriormente una línea dispone de varias.

- Celda de Origen: Índice de la celda de la matriz correspondiente al inicio de la medición del tiempo.
- Celda de Destino: Índice de la celda de la matriz donde concluye ese tramo
- Tiempo: El tiempo promedio correspondiente que conlleva ir desde la celda de origen a la celda destino
- Cantidad de muestras: número de datos presentes correspondientes al tramo analizado.
- Fecha: Indica dentro de qué rango horario se encuentra.

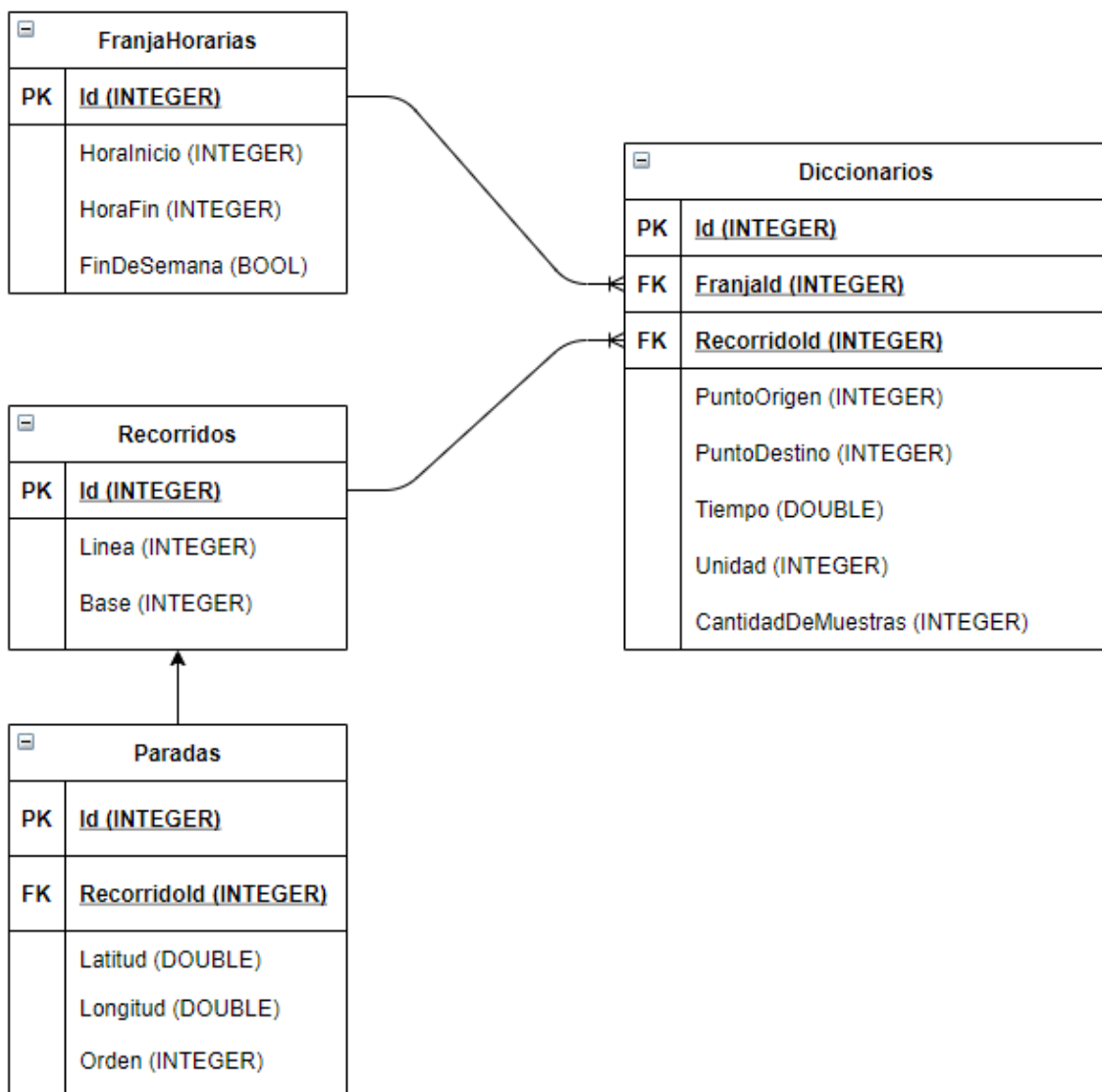


Figura 3.11 Esquema utilizado para la base de datos.

El esquema de base de datos presentado en la Figura 3.11 permite almacenar toda la información necesaria para realizar los cálculos de manera sencilla. Al disponer de los datos diferenciados según las franjas horarias, las líneas y sus diferentes recorridos en tablas separadas, se provee flexibilidad para agregar nueva información diferente sin impactar en los datos de tiempo preexistentes. Complementariamente se dispone de una tabla donde se persisten las paradas correspondientes a cada uno de los recorridos disponibles. De esta manera, se logra que ante cualquier modificación del camino que recorre una línea en particular, no impacte en el resto del esquema de tablas.

3.4.3 Implementación

Para realizar el cálculo de cuánto tiempo conlleva trasladarse desde un punto específico a otro punto en particular, se diseñó e implementó un algoritmo propio que se encuentra descrito a continuación y en donde se recibe de entrada una serie de parámetros provistos tanto por el usuario como el sistema. Dichos parámetros son:

- Las coordenadas correspondientes a la ubicación de origen y la de destino, mediante el par latitud longitud.
- Línea y trayecto a consultar.
- Fecha y hora en la que se quiere trasladar.
- Unidad del colectivo, aunque este último parámetro es de carácter opcional ya que no siempre es conocido su valor.

Una vez que los datos son recibidos por la API Rest, se procede a una serie de métodos para obtener el tiempo correspondiente. Inicialmente a partir de la hora y el día ingresado se obtiene a qué franja horaria corresponde la consulta. De manera similar se adquiere el identificador de recorrido a partir de la línea y el trayecto ingresado, en función del colectivo seleccionado por el usuario. Entre los parámetros recibidos se encuentra la unidad, pero al ser opcional, en caso de no ser enviado se consideran todas las unidades presentes en la base de datos. A partir de dichos datos obtenidos previamente, se obtienen todos los diccionarios correspondientes para luego ser utilizados para el cálculo del tiempo.

Complementariamente, entre los parámetros de entrada recibidos se encuentran el par de coordenadas al origen y al destino del viaje. Dichas coordenadas son utilizadas para el cálculo para obtener a qué celda de la matriz de la ciudad corresponde.

Haciendo uso de la lista de celdas de recorrido base mencionadas previamente para el recorrido escogido, se obtienen los índices a partir de las celdas de origen y destino correspondientes a las celdas previamente calculadas. Dicho de una manera más sencilla, se ubicaron las coordenadas recibidas dentro del recorrido para analizar qué porción del camino base se desea consultar. Esto nos permite validar que los índices se corresponden a una parte correcta del trayecto, y no que vaya en un sentido opuesto o simplemente no se correspondan con los caminos que realiza la línea. En caso de no cumplirlo, o si simplemente algunos otros de los parámetros ingresados no son válidos, la aplicación muestra un mensaje de error, informando el motivo del problema.

Pero en caso de que todos los parámetros ingresados correspondan a un pedido correcto, se procede a buscar en la base de datos los diccionarios de información histórica asociada. Dichos diccionarios se encuentran filtrados por franja horaria, recorrido, y solo consideramos los puntos que se encuentran entre los índices de las coordenadas calculadas previamente, limitando así al fragmento de la ciudad que nos interesa calcular.

Cómo se menciono previamente dicho diccionario se encuentra conformado por los siguientes campos:

- Unidad
- Punto de Origen
- Punto de destino
- Segundos (se dispone de una lista, con todos los tiempos históricos)
- Cantidad De muestras

Para obtener el tiempo relacionado simplemente calculamos la suma de los puntos contiguos que conforman el trayecto buscado. Para ejemplificar, suponiendo que se busca calcular el tiempo entre el punto de origen número 10 y el número 15. Para obtener el resultado se realiza la suma entre el tiempo propio entre 10 y 11, más el tiempo entre 11 y 12, y así sucesivamente hasta 14 y 15. A partir del análisis de los diccionarios presentes en la base de datos, se observó que si bien en la mayoría de los

casos la diferencia entre los puntos de origen y destino es de uno, (lo que significa que se trata de información de celdas contiguas de la matriz) en ciertas oportunidades dicha diferencia es mayor. Por lo que a la hora de calcular el tiempo, se decidió promediar los cálculos contiguos con dichos valores, preponderando la cantidad de muestras que disponen cada uno.

A modo de ejemplo, dicha metodología puede apreciarse en la Figura 3.12. Los valores en color verde se corresponden a la cantidad de muestras disponibles y los numeros en negro al extremo de cada línea corresponde desde donde (a la izquierda) hasta qué parte (derecha) del recorrido tiene en consideracion. Para este caso, el tiempo entre 10 y 12, se calcula el tiempo entre 10 y 11 más el tiempo entre 11 y 12 pero también se va a utilizar el tiempo de 10 a 12, pero dándole mayor o menor peso según la cantidad de muestras disponibles.

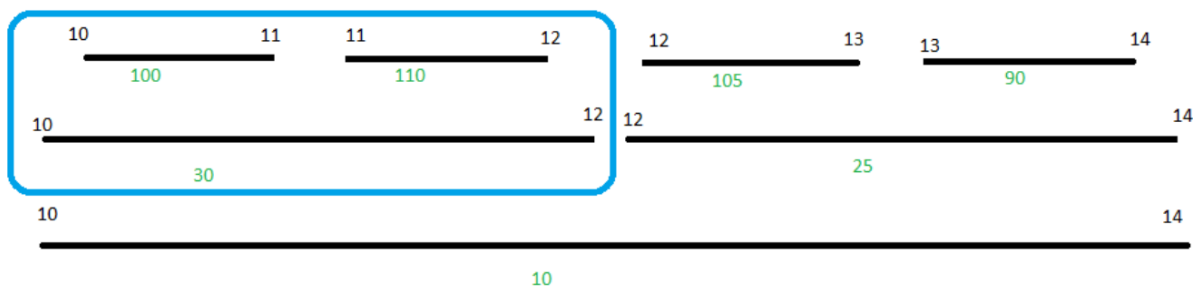


Figura 3.12: Cálculo del tiempo basado en partes.

Entre los valores retornados, se incluye además, la distancia que hay entre las coordenadas ingresadas, proporcionando la cantidad de metros a recorrer.

3.5 Enfoque de Regresión Múltiple

Analizando la estructura de los diccionarios y teniendo en cuenta que debido a diferentes factores, cómo reparación de calles o cambio de recorridos, los caminos base por los que transitan los colectivos pueden modificarse a lo largo del tiempo, se diseñó un método alternativo para solventar esa cuestión. Dicho algoritmo hace uso de los datos de los diccionarios previamente mencionados y del método de regresión lineal, pero a diferencia del caso de pruebas inicial, se dispone de más de un parámetro de

entrada. A diferencia del método de la matriz previamente descrito, esta regresión se recomienda utilizar cuando no se disponen datos sobre una porción del recorrido en específico, ya que ignora por qué sector de la ciudad circula sino que considera la distancia a desplazarse a la hora de realizar los cálculos.

Cómo se explicó previamente en la sección 2.5.2 se dispone de una serie de variables independientes. Para este algoritmo, la variable dependiente a calcular continúa siendo el tiempo expresado en unidad de segundos. En cambio las variables independientes a utilizar para el cálculo, son la franja horaria (considerando la misma división previamente utilizada), línea, unidad, recorrido, y la diferencia de celdas a recorrer. Siendo la diferencia de celdas una variable de tipo continua y todas las restantes de tipo discretas. Dicha diferencia de celdas no es un parámetro explícito, sino que se calcula a partir de las coordenadas seleccionadas por el usuario y ubicando dichos puntos dentro de los recorridos base que realiza el colectivo escogido y obteniendo la diferencia de celdas entre ambos puntos.

La implementación de este algoritmo no requiere realizar ninguna estructura nueva, debido a que realiza los cálculos utilizando los datos de los diccionarios previamente calculados y almacenados en la base de datos SQL. Al igual que la regresión lineal simple, los datos se dividen en dos, por un lado los datos que se utilizan para entrenar al modelo y los restantes para realizar el testeo de la performance de la técnica.

Complementariamente al disponer de variables discretas, se requiere realizar una transformación a variables numéricas, haciendo uso de una función de codificación propia del framework utilizado, la cual entraremos en detalle más adelante.

A la hora de realizar los cálculos correspondientes para la regresión múltiple se reciben una serie de parámetros que coinciden con los utilizados en la técnica de la matriz, dichos parámetros de entrada son los siguientes:

- Las coordenadas correspondientes a la ubicación de origen y la de destino, mediante el par latitud longitud.
- Línea y trayecto a consultar.
- Fecha y hora en la que se quiere trasladar.
- Unidad del colectivo.

A partir de las coordenadas ingresadas, se verifican que sean válidos al recorrido y trayecto correspondiente y se procede a ubicarlos dentro del recorrido base. En caso de ser correcto, se calcula la diferencia de celdas que se deben realizar desde el origen al destino marcado. Luego simplemente se hace uso de la regresión múltiple y se ingresan los parámetros restantes para obtener el tiempo correspondiente y se lo retorna para ser visualizado desde el lado de la aplicación móvil. Si bien el número de parámetros utilizados cambia, se puede hacer uso de las mismas métricas utilizadas previamente en la regresión simple para comparar la performance de cada una de ellas.

3.6 Resultados

A lo largo de esta sección se expone un análisis de los resultados obtenidos en los enfoques propuestos, observando y comparando los valores de las metodologías propuestas. Inicialmente se exponen de manera individual los resultados de cada uno de dichos enfoques para finalmente comparar los resultados y generar una conclusión general.

3.6.1 Regresión lineal simple

En primer lugar, la regresión lineal simple, en donde el input consiste en la variable distancia y la variable a calcular es el tiempo. Como se mencionó previamente, para la implementación de este caso de prueba se decidió trabajar con tres conjuntos de datos, correspondiente a la unidad uno, dos y tres. Para cada uno de ellos, se separó el 70% de los datos históricos para realizar el training del modelo y el 30% restante de los datos para realizar los test pertinentes. Después se procedió con el algoritmo y a realizar las predicciones correspondientes. Para ello, hicimos uso del conjunto de datos de testeo, y se observó con qué precisión nuestro algoritmo predice los valores. Siguiendo con los criterios normales dentro de la distancia normal de recorrido, obtuvimos la predicción para 100, 150, 250, 500, 1000 metros, con los siguientes resultados disponibles en la figura 3.13.

Distancia Recorrida (m) / Tiempo Estimado (seg)	Unidad 1	Unidad 2	Unidad 3
100	28,88	27,03	29,05
150	31,78	35,11	37,4
250	37,61	51,25	54,09
500	52,17	91,6	95,81
1000	81,29	172,32	179,25

Figura 3.13: Predicciones obtenidas para diversas distancias.

Además, otra manera de visualizar la comparación resultante es utilizando el gráfico de dispersión cómo se observa en la figura 3.14.

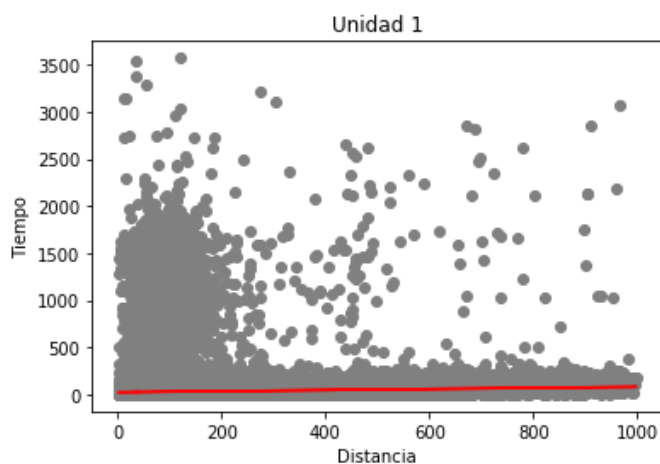


Figura 3.14: Regresión lineal obtenida para la unidad 1.

Y finalmente se calculó las métricas explicadas en la sección 2.5.4 y se puede observar los resultados obtenidos en la figura 3.15.

Metrica / Unidad	Unidad 1	Unidad 2	Unidad 3
Correlacion	0,051	0,135	0,162
Error absoluto medio (MAE)	18,18	18,42	20,77
Error cuadrático medio (RMSE)	87,59	88,57	97,67
R al cuadrado (R^2)	0,0027	0,01	0,014

Figura 3.15: Métricas obtenidas para los datos evaluados.

Basándonos en las métricas obtenidas a lo largo del proceso, se puede deducir que los resultados de aplicar esta metodología no fueron los esperados. En el momento inicial de estructuración de los datos, se intuía que al trabajar con información muy sensible se requiere estar pendientes y atentos a la hora de hacer los cálculos, por ejemplo los valores de las coordenadas, donde ante cualquier mínima variación impacta en el resultado.

Examinando visualmente los gráficos adjuntos en el informe donde se puede apreciar cómo los datos se encuentran distribuidos se deduce que los valores se encuentran dispersos y son demasiado variados para realizar una implementación de la regresión lineal. Buscar hallar la recta ideal que se adapte de la mejor manera a todos los datos disponibles resulta difícil de calcular sin penalizar severamente los errores de esta metodología.

Al analizar los valores de correlación entre las variables tiempo y distancia, se obtiene que la relación entre dichas variables es baja, lo cual no es un buen indicador a priori. Unos de los resultados más negativos se encuentran en los valores obtenidos para la métrica de R^2 . Si bien esta métrica no utiliza la escala del modelo, al tener valores muy cercanos a cero y lejanos del valor 1 ideal, nos indica que el modelo no es muy bueno para predecir.

Los valores de MAE, oscilan entre 18 y 21 segundos, esto representa que al realizar un cálculo de cuánto tiempo va a tomar realizar una distancia determinada, se va a tener un margen de dicho tiempo de error. Visualmente representa la distancia horizontal promedio entre cada punto y la recta ideal.

A modo de conclusión podemos determinar que si bien los resultados de las métricas definidas y utilizadas no son los ideales para el modelo definido, tampoco son demasiados erróneos como para descartar completamente esta técnica. La principal razón por la cual los resultados no son los esperados se debe a que la técnica empleada no resultó ser la más apropiada para el conjunto de datos utilizados. A través de los diferentes filtros especificados, se removieron los valores atípicos y se limitó el conjunto de datos para evitar incoherencias a la hora de la implementación. Pero al disponer de un dataset con gran cantidad de elementos y demasiados datos dispersos, lo cual hace imposible encontrar una predicción que resulte representativa para todos los valores posibles y no tener errores demasiados altos. El modelo aun así puede realizar predicciones que si bien no son lo suficientemente precisas, puede interpretarse y

utilizarse como base de referencia para analizar la performance de los siguientes casos de pruebas que realizaremos a continuación.

3.6.2 Regresión lineal múltiple

A la hora de realizar los cálculos correspondientes para la regresión múltiple se reciben una serie de parámetros que coinciden con los utilizados en la técnica de la matriz, dichos parámetros de entrada son los siguientes:

- Las coordenadas correspondientes a la ubicación de origen y la de destino, mediante el par latitud longitud.
- Línea y trayecto a consultar.
- Fecha y hora en la que se quiere trasladar.
- Unidad del colectivo.

A partir de las coordenadas ingresadas, se verifican que sean válidos al recorrido y trayecto correspondiente y se procede a ubicarlos dentro del recorrido base. En caso de ser correcto, se calcula la diferencia de celdas que se deben realizar desde el origen al destino marcado. Luego simplemente se hace uso de la regresión múltiple y se ingresan los parámetros restantes para obtener el tiempo correspondiente y se lo retorna para ser visualizado desde el lado de la aplicación móvil. Si bien el número de parámetros utilizados cambia, se puede hacer uso de las mismas métricas utilizadas previamente en la regresión simple para comparar la performance de cada una de ellas.

Metrica	Regresion Simple	Regresion Diferencia de Celdas
Correlacion	0.116	0.656
Error absoluto medio (MAE)	19.123	18.714
Error cuadrático medio (RMSE)	91.277	33.791
R al cuadrado (R^2)	0.009	0.438

Figura 3.16 Comparación de las métricas de las regresiones.

Cómo puede observarse en la figura 3.16, hay varias mejoras para destacar. En primer lugar, la correlación obtenida indica que la relación entre las variables de las diferencias

de celdas y el tiempo es de intensidad media/fuerte con sentido positivo, lo que significa que si una de las variables aumenta su valor, la otra probablemente también lo hará. En segundo lugar, el error absoluto medio es bastante similar en ambos casos, aunque el error cuadrático medio ya denota una diferencia significativa, siendo más severo en la regresión lineal simple. Por otro lado, la métrica R al cuadrado también posee mejores resultados para la regresión múltiple basada en diferencia de celdas, alcanzando un valor de 0.438, comparado con casi cero del otro caso.

3.6.3 Enfoque Matriz

Para evaluar la precisión del enfoque propuesto, se escogieron de manera aleatoria una cantidad específica de muestras de diferentes unidades de la línea 500 y se comparó los tiempos históricos reales con los tiempos calculados por el algoritmo. Para llevar a cabo dicha evaluación, se hizo uso de 6000 muestras de porciones de viajes (con un inicio y fin determinado) de diversos horarios, recorridos y distancia realizada. Como se puede apreciar en la figura 3.17, se dispone de una gran cantidad de datos de diferente distancia recorrida.

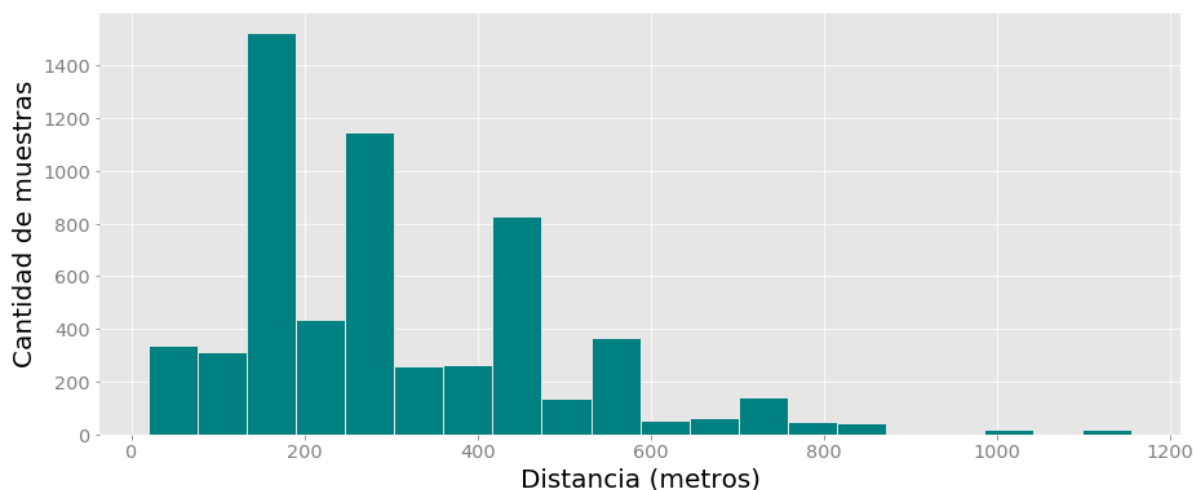


Figura 3.17: Distribución de los datos utilizados para evaluar la precisión del enfoque matriz.

Para cada uno de los datos históricos considerados, se calculó el tiempo haciendo uso del algoritmo propuesto para el mismo origen, destino, horario, trayecto, línea y unidad

específica. Esto nos permite medir el tiempo en la misma franja horaria, para una unidad en particular realizando un trayecto determinado y en una región de la ciudad en concreto y compararlo con un valor real.

De esta manera, se puede evaluar el nivel de precisión que dispone el enfoque propuesto con respecto a los datos reales. A partir de la comparación de los valores retornados por el algoritmo y los valores históricos, se calcularon las métricas empleadas descritas en la sección 2.5.4, en donde se obtuvo una serie de valores que nos permiten observar el rendimiento de los tres enfoques propuestos.

Métrica	Regresión Simple	Regresión Múltiple	Enfoque Matriz
Error absoluto medio (MAE)	19.123	18.714	18.876
Error cuadrático medio (RMSE)	91.277	33.791	29.182
R^2	0.009	0.438	0.400

Figura 3.18: Métricas obtenidas para los tres enfoques.

3.6.4 Conclusiones

A lo largo de esta sección, se expuso tres enfoques diferentes con el objetivo de resolver la problemática planteada, exponiendo para cada uno sus principales ventajas y desventajas, desarrollando cómo fue el proceso de diseño e implementación y exponiendo los resultados obtenidos.

Inicialmente se comenzó por la aplicación de una regresión lineal simple, en donde se calcula el tiempo en función de la distancia en metros a recorrer. Pese a la sencillez de su implementación, se encontraron una serie de desventajas cómo la ausencia de distinción de horario o sector de la ciudad, las que provocan que no sea de mucha utilidad. Al poseer una gran cantidad de datos los cuales poseen una dispersión demasiado amplia, provocan que la correlación obtenida y la precisión del algoritmo no sea muy eficaz, lo cual se avala analizando las métricas obtenidas en las diferentes unidades estudiadas.

Posteriormente se expuso un segundo caso de pruebas basado en dividir a la ciudad de Tandil cómo si se tratase de una matriz, en donde cada celda representa de manera aproximada a una manzana, aunque sin considerar peajes o cómo se disponen en el mapa. Dicho algoritmo requirió la definición de una nueva estructura de diccionario para almacenar la información e incluye nuevos parámetros a la hora de realizar los cálculos. En este caso, se tuvo en cuenta la unidad, el horario, el recorrido, la línea y la región de la ciudad donde se lleva a cabo el viaje. Esto implicó un incremento grande en la complejidad, pero al analizar los valores obtenidos en las diferentes métricas analizadas, se observó una gran mejoría en los resultados conseguidos, destacando un margen de error menor para los cálculos.

El tercer enfoque, combina los diccionarios previamente definidos y con la técnica de regresión para implementar una regresión lineal múltiple, la cual basa su cálculo en la diferencia de celdas, pero considerando también los mismos parámetros de entrada de la técnica de la matriz. Dicha técnica además puede utilizarse en porciones de recorrido nuevos o donde no se dispone de información histórica para estudiar. Analizando las métricas obtenidas (disponibles en la figura 3.18) se deduce que si bien el método de regresión lineal múltiple posee resultados más precisos que la regresión simple, el nivel de precisión es muy similar al de la técnica de la matriz, en donde se consiguió métricas parecidas y cercanas.

En la aplicación presentada, se optó por proveer la funcionalidad para la técnica de la matriz y para la regresión lineal múltiple. Para ambos casos se provee la funcionalidad individual adecuada que recibe los parámetros previamente mencionados y retorna el tiempo predicho junto a otra información adicional la cual se detalla en las secciones posteriores relacionadas a la implementación de la aplicación móvil.

Capítulo 4: Desarrollo de la Propuesta

4.1 Introducción

A lo largo de este capítulo se describen los detalles acerca de las diferentes decisiones que fueron tomadas para llevar a cabo la propuesta presentada, haciendo foco en las tecnologías y características principales que conforman la estructura de la solución desarrollada. Debido a que la aplicación consta de varios aspectos, en este apartado comenzamos desde un enfoque de la visión general de la herramienta implementada para posteriormente entrar en un nivel de detalles mayor.

A causa de la gran complejidad y los diferentes enfoques que dispone el proyecto planteado, es crucial dividir al enfoque general de toda la aplicación y cada uno de sus componentes. Por lo tanto se abordan los aspectos de la herramienta propuesta, dividiéndola, a grandes rasgos, en un apartado dedicado al servidor y en otro al cliente.

Se explicarán las diferentes tecnologías utilizadas, y la justificación de su elección. La aplicación llevada a cabo puede separarse, a grandes rasgos, en dos grandes secciones que si bien se encuentran relacionadas, actúan de manera independiente y difieren en muchos aspectos entre sí. En primer lugar, el apartado del cliente, que dispone de la capa visual con la que el usuario va a interactuar directamente mediante el dispositivo móvil. Y por otro lado, la sección del servidor que proporciona los métodos y los algoritmos para realizar los cálculos pertinentes.

Referido a la parte visual, también llamado front-end, consta de una aplicación móvil Android, donde se presentan diversas cuestiones como: la visualización e interacción sobre los mapas, la ubicación de lugares y puntos a elección del usuario, la simulación de recorridos de colectivos andando, la visualización de la información específica para el usuario. Entre las tecnologías empleadas, se puede mencionar: Android Studio, Kotlin, la

API de mapas de Google Maps Platform⁸ y Material Design⁹ para el desarrollo de los componentes de la interfaz gráfica.

En cuanto al apartado de servidor, más conocido como back-end, se enumeran las tecnologías usadas para que el usuario pueda calcular los tiempos y el análisis de los caminos posibles correspondientes al pedido solicitado. También se hace mención a conceptos de patrones de diseño y API Rest, componentes claves para el diseño de toda la funcionalidad del sistema. Entre las diversas librerías o herramientas utilizadas, se puede destacar Net 5¹⁰, Swagger¹¹, y Entity Framework Core¹².

Es importante destacar el uso de la herramienta llamada Git¹³, que proporciona un sistema de control de versiones, alojamiento del código desarrollado facilitando el trabajo colaborativo.

Complementariamente se realiza una descripción de aquellos requerimientos que se encuentran implementados en el proyecto para proporcionar una visión de la funcionalidad que dispone la herramienta.

4.2 Estructura de la Herramienta

Inicialmente, luego del análisis de la problemática planteada con las respectivas necesidades de los usuarios se optó por la realización de una aplicación móvil (concretamente para teléfonos y celulares inteligentes también conocidos como smartphones) y a partir de ahí surgieron diferentes inquietudes para realizar la selección de las tecnologías a utilizar.

La primera decisión tomada para este trabajo consistió en seleccionar el tipo de plataforma de desarrollo de la aplicación, de acuerdo al sistema operativo, funcionalidades y características de los dispositivos en donde se ejecutarán las mismas. La elección de realizar una aplicación nativa, por sobre otras como una aplicación multi-plataforma, híbrida o web, permite tener un producto desarrollado y optimizado específicamente para el sistema operativo determinado y la plataforma de desarrollo del

⁸ "Google Maps Platform | Google Developers." <https://developers.google.com/maps/documentation>.

⁹ "Design - Material Design." <https://material.io/design>.

¹⁰ <https://devblogs.microsoft.com/dotnet/introducing-net-5/>

¹¹ <https://swagger.io/>

¹² <https://docs.microsoft.com/en-us/ef/core/>

¹³ <https://git-scm.com/>

fabricante (Android). Es por estos motivos que, en primer lugar se obtiene un rendimiento mayor y una optimización superior tomando ventaja de las capacidades que posee el dispositivo a utilizar. En segundo lugar está relacionado al aspecto visual, las transiciones y los efectos pueden ser más precisos y performantes ajustándose a las capacidades del sistema operativo que se dispone, obteniendo una mejor experiencia de uso. Por otro lado, se dispone de experiencia en el desarrollo de aplicaciones nativas de Android, que nos permite minimizar el tiempo de aprendizaje de la tecnología y lenguajes aplicados. Es por estas razones que se optó por crear una aplicación nativa por encima de otras alternativas que podrían demandar mayor tiempo de aprendizaje y desarrollo cómo por ejemplo haciendo uso de HTML5 en complemento con CSS y JavaScript desplegando dentro de un contenedor nativo cómo Cordova¹⁴ o como el uso de Flutter¹⁵, un framework de desarrollo móvil de Google que proporciona una manera rápida y expresiva para los desarrolladores crear aplicaciones nativas tanto en iOS y Android, pero con la utilización de un lenguaje de desarrollo distinto al conocido (Dart), y la necesidad del conocimiento de los widgets de ambas plataformas.

Posteriormente, luego de seleccionar el tipo de aplicación, se debió decidir entre qué tipos de sistemas operativos se iba a desarrollar la aplicación, teniendo la opción de escoger entre iOS o Android, ya que si bien se trata de los dos sistemas operativos para móviles más utilizados en todo el mundo, la implementación a nivel de programación difiere completamente ya que se trabaja con librerías y herramientas diferentes. Luego de analizar las ventajas y desventajas de cada una de las alternativas, finalmente se optó por los dispositivos android principalmente por dos motivos. Por un lado es el sistema más utilizado y accesible no solo en la ciudad sino que también en Argentina. Por otro lado, se dispone de experiencia en el desarrollo en dicha plataforma en el ámbito laboral lo cual es un motivo crucial para la elección.

Luego de decidir de qué manera se iba a proceder desde la perspectiva visual de la herramienta, se prosiguió con definir qué tecnologías y metodologías iban a ser utilizadas para desarrollar desde el lado de back-end. Para la implementación desde el lado del servidor, se desarrolló un servicio web particular y se optó por hacerlo con una arquitectura de tipo Transferencia de Estado Representacional (REST) por encima de otras alternativas cómo SOAP (del inglés Simple Object Access Protocol). El motivo de

¹⁴ <https://cordova.apache.org/>

¹⁵ <https://flutter.dev/>

dicha decisión se debe a que REST es más performante y además no hay necesidad de mantener un estado de la información entre diferentes solicitudes, siendo independientes entre sí, lo que implica mayor facilidad para la codificación y posterior desarrollo que tienen estos servicios.

Definida la arquitectura REST se continuó con la definición de un lenguaje de programación que soportara y permitiera facilitar su desarrollo. Entre los distintos lenguajes disponibles en la actualidad se analizó que para C# existían múltiples frameworks y bibliotecas que ayudan a la implementación. Principalmente, la utilización de un lenguaje utilizado por los desarrolladores aceleró el proceso de desarrollo de la herramienta, complementándolo con la utilización del framework Net 5, el cual entraremos en detalle más adelante.

Es importante aclarar que la aplicación fue creada y es independiente en su totalidad con respecto a GPS Sumo, la cual muestra la ubicación de los colectivos en tiempo real dentro de la ciudad entre otras funciones.

A modo de resumen, la herramienta desarrollada consiste en una aplicación móvil en la que, a partir de ciertos parámetros provistos por el usuario, se calculan ciertos tiempos y valores de respuestas que son visualizados a través de la aplicación celular.

Entre los objetivos propuestos, se incluye lograr una aplicación en la cual los usuarios se sientan cómodos y puedan utilizarla diariamente sin ningún tipo de inconveniente o dificultad, siendo lo más intuitiva posible y para el cual se estudiará punto por punto las ventajas y desventajas a la hora de su implementación. Además brindando funcionalidad que hoy en día no se dispone en la herramienta Sumo y en la mayoría de aplicaciones en el mercado.

En el siguiente esquema (figura 4.1), se puede visualizar a grandes rasgos un ejemplo del flujo que posee la herramienta desarrollada, simulando un pedido relacionado al mapa y los componentes involucrados. En donde el usuario hace uso del dispositivo móvil a través del cual interactúa con la herramienta.

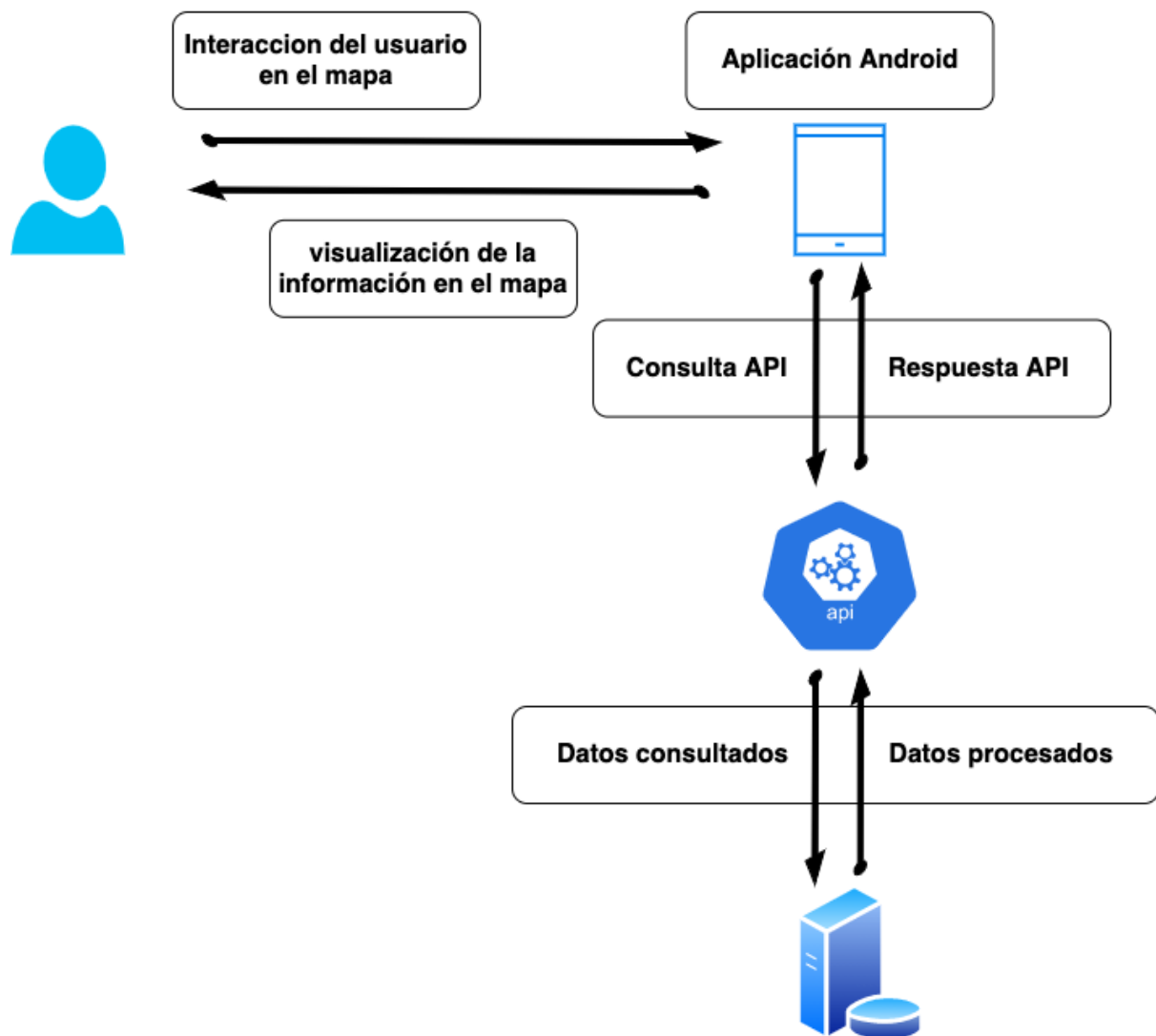


Figura 4.1. Descripción del flujo de información

Se dispone una API Rest que provee un conjunto de endpoints para que la capa visual consulte y obtenga toda la información necesaria a través de pedidos HTTP. Desde el lado de backend mucha información necesaria para los cálculos de los algoritmos se encuentra almacenada en una base de datos, cuyas características se abordarán posteriormente.

En las siguientes secciones se entra en detalle acerca de las especificaciones y las decisiones tomadas para llevar a cabo cada uno de los componentes de la herramienta.

4.3 Patrones Empleados

Durante la fase de desarrollo se afrontan diversos problemas recurrentes y cotidianos que pueden ser solucionados mediante la implementación de patrones de diseño. Estos básicamente son modelos que sirven como guía para que los programadores trabajen sobre ellos. Permiten crear una aplicación más extensible y robusta, siendo más sencilla de mantener en el futuro ante posibles cambios ya que se evita perder tiempo en búsqueda de soluciones a problemas ya conocidos o resueltos. También los patrones permiten estandarizar el lenguaje del código, haciendo que el diseño sea más legible y comprensible para otros programadores. Vale aclarar, que si bien dichas reglas son muy recomendables, no son obligatorias, aun así para esta aplicación decidimos implementar algunos patrones particulares que describiremos a continuación.

4.3.1 Enfoque Servidor

En cuanto a la arquitectura utilizada para el desarrollo de backend se optó por una arquitectura de capas, la cual consiste en dividir el sistema en diversas capas independientes entre sí y en donde cada una debe exponer en forma clara las operaciones que puede realizar. Su elección se debe a que en caso de necesitar incrementar la complejidad de la aplicación, resulta más sencillo administrar y dividirla según sus obligaciones y especificaciones.

Entre las ventajas, se destaca que este enfoque sigue el principio de separación de responsabilidad y permite mantener el código base bien organizado, facilitando a los desarrolladores la búsqueda de código donde se implementa una funcionalidad en específica. Otras de las ventajas que ofrece este tipo de arquitectura, es que la funcionalidad de bajo nivel puede ser reutilizada en diferentes partes de la aplicación. Esta reutilización es muy importante y beneficiosa ya que permite escribir menor cantidad de código siguiendo el principio DRY (Don't repeat yourself) favoreciendo la claridad del código y evitando posibles problemas de consistencias. Dicho principio

evita la duplicación de código, que provoca una cantidad de trabajo mayor requerido para extender y mantener el software en el futuro.

Con este tipo de arquitectura en diversos niveles, se pueden establecer ciertas restricciones con respecto a la comunicación entre las capas, logrando así un efecto de encapsulación en donde cada capa es independiente de las demás. Esto significa que al realizar cualquier cambio sobre una capa en particular, solo se encuentran afectadas aquellas que funcionan e interactúan con ella. Por lo tanto, al limitar la dependencia entre capas, se reduce el impacto de cualquier modificación en los diferentes módulos que conforman la aplicación.

Cabe destacar que se facilita considerablemente cualquier cambio de funcionalidad dentro de la aplicación. Por ejemplo, es posible que una aplicación utilice inicialmente una base de datos de SQL para realizar la persistencia, pero en el futuro podría optar por usar una estrategia de persistencia basada en la nube. En caso de que la aplicación haya encapsulado correctamente su implementación de persistencia dentro de una capa lógica en particular, esa capa específica de SQL se podría reemplazar por una nueva que implementará la misma interfaz pública sin afectar las capas restantes ni la funcionalidad provista.

Para este tipo de arquitectura, las capas se disponen de manera horizontal de manera que cada una de ellas sólo puede establecer comunicación con aquella capa que se encuentra por debajo. Si una capa desea comunicarse con otra que no está inmediatamente deberá hacerlo a través de las capas intermedias. A modo de ejemplo, si la capa que posee los controladores necesita realizar una consulta a la base de datos mediante un repositorio en concreto, no podrá directamente sino que deberá hacerlo a través de un servicio presente en la capa de aplicación.

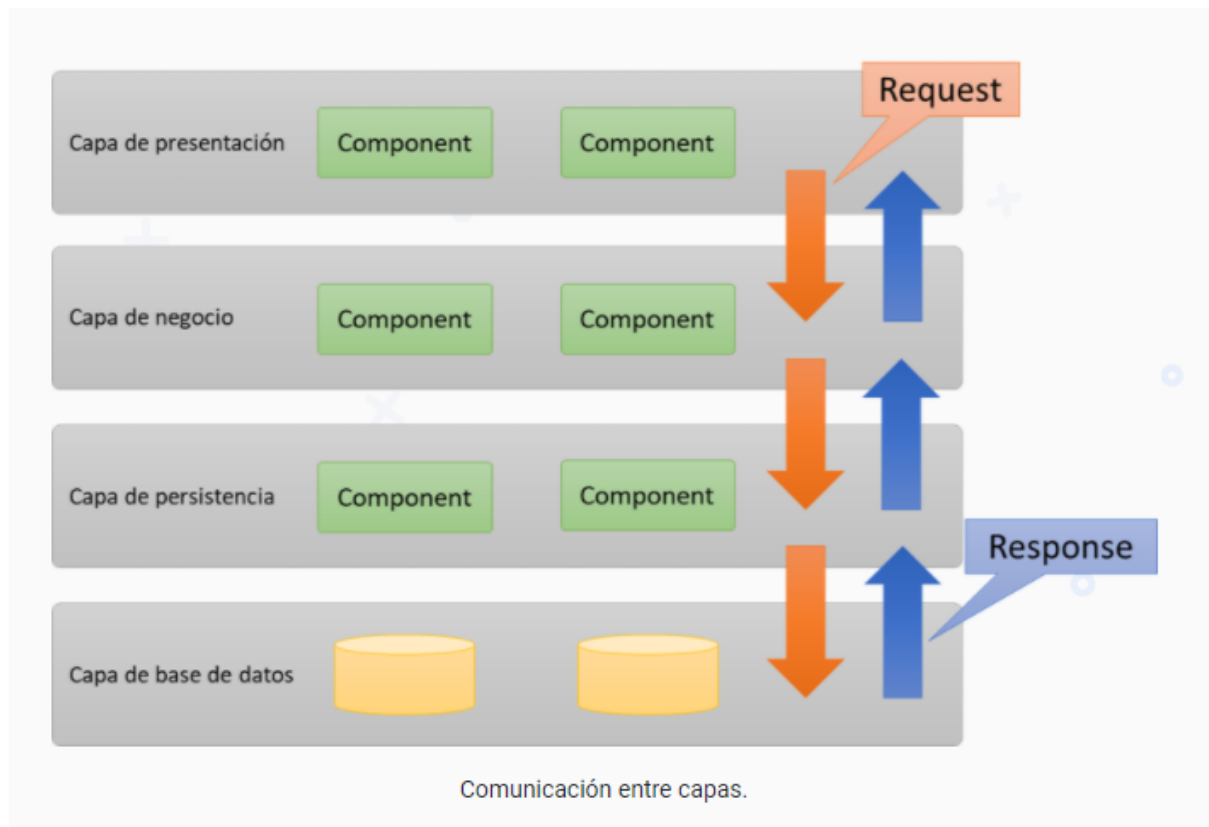


Figura 4.2 Descripción de la capas que componen el lado del servidor

Respetar el orden que poseen las capas es importante para evitar un posible desorden en cualquier flujo de comunicación, que implicaría resultados erróneos o inaccesibles.

Una de las desventajas que posee esta estructura es que hay una menor tolerancia a fallos, debido a que si una capa en particular falla, las demás capas superiores no van a poseer la funcionalidad deseada. Para evitar esto, se hizo foco en el manejo de excepciones a lo largo del desarrollo implementando codificación especial en caso de fallas.

También se busca aplicar el principio Open/Closed el cual indica que cualquier entidad debe estar abierta para la expansión pero cerrada a la modificación. Esto implica que cualquier funcionalidad nueva que se incorpora, no debe afectar al código ya existente. Para lograr esto se requieren incorporar nuevas clases y métodos, pero sin modificar lo ya presente en la herramienta. Para poner en práctica este principio se hizo uso de interfaces y para así delegar la funcionalidad a los objetos que la implementan.

4.3.2 Enfoque Cliente

Para el desarrollo del front-end, fue elegida la **arquitectura CLEAN**¹⁶. Una arquitectura de base sólida es extremadamente importante para que una aplicación se escale y cumpla con las expectativas de la base de usuarios. Además permite desacoplar diferentes unidades de código de manera organizada. De esta forma el código se hace más fácil de entender, modificar y testear. Esta arquitectura fue propuesta en 2012 por Robert C. Martin (Uncle Bob) en un blog de código limpio siendo en la actualidad la arquitectura base más elegida para el desarrollo móvil.

Basados en los principios SOLID¹⁷ para hacer que los diseños de software sean más comprensibles, más fáciles de mantener y más fáciles de ampliar; los puntos RUDT (Read, T pdate, Debug y Test) y los lineamientos ofrecidos por la guía de arquitectura de apps de Android developers, que describe la experiencia de los usuario de apps para dispositivos móviles, nos permiten la implementación de una arquitectura capaz de reducir el tiempo de desarrollo, facilitar la comprensión, escribir código limpio y de calidad.

La arquitectura Clean, mejor conocida por su arquitectura de capas, es un conjunto de principios cuya finalidad principal es ocultar los detalles de implementación a la lógica de dominio de la aplicación. De esta manera se mantiene aislada la lógica, consiguiendo que ésta sea más mantenible y escalable en el tiempo.

Estas características hacen referencia a que cada capa solo va a tener conocimiento de la capa interior, y no hacia afuera, lo que nos definirá las dependencias de capas en el proyecto. Según nuestro requisito la arquitectura no especifica el número de capas, ya que se puede enfocar de maneras diferentes.

¹⁶ "Clean Architecture of Uncle Bob - Clean Coder Blog." 13 ago. 2012, <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>.

¹⁷ "Principios SOLID: Qué son, cuáles, y qué beneficios aporta usarlos." <https://devexperto.com/principios-solid/>.

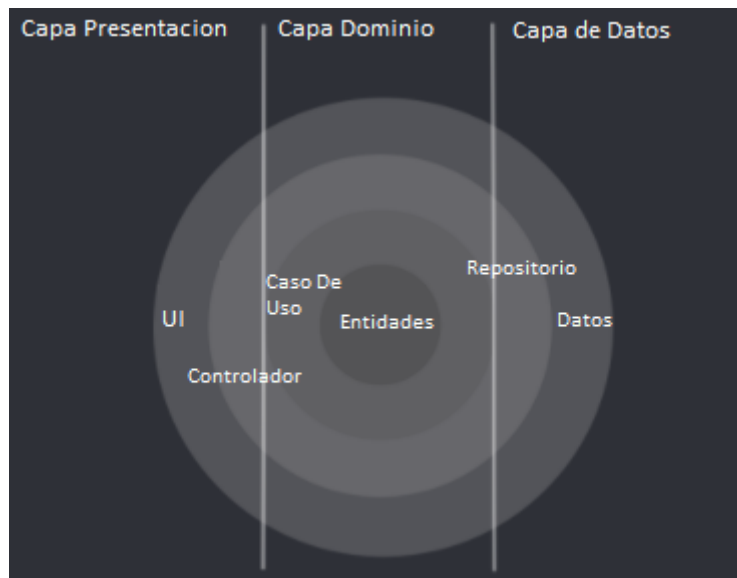


Figura 4.3 Descripción de la arquitectura del cliente

- **Capa de presentación (Presentation Layer):** Es la capa que interactúa con la interfaz de usuario y generalmente contiene UI (actividades y fragmentos) que están coordinados por presentadores / modelos de vista que ejecutan un o varios casos de uso, dependiendo del patrón de presentación que se elija. La capa de presentación depende de la capa de dominio, es decir, tiene una dependencia con dicha capa.
- **Capa de dominio:** es la parte más interior de la arquitectura (sin dependencias con otras capas) donde se encuentra la lógica o reglas de negocio. Contiene Entidades, casos de uso e interfaces de repositorio. Los casos de uso combinan datos de una o varias interfaces de repositorio.
- **Capa de datos:** se encuentra una definición abstracta de las diferentes fuentes de datos, y la forma en que se debe utilizar. Contiene implementaciones de repositorios y una o varias fuentes de datos. Los repositorios son responsables de coordinar los datos de las diferentes fuentes de datos. La capa de datos depende de la capa de dominio.

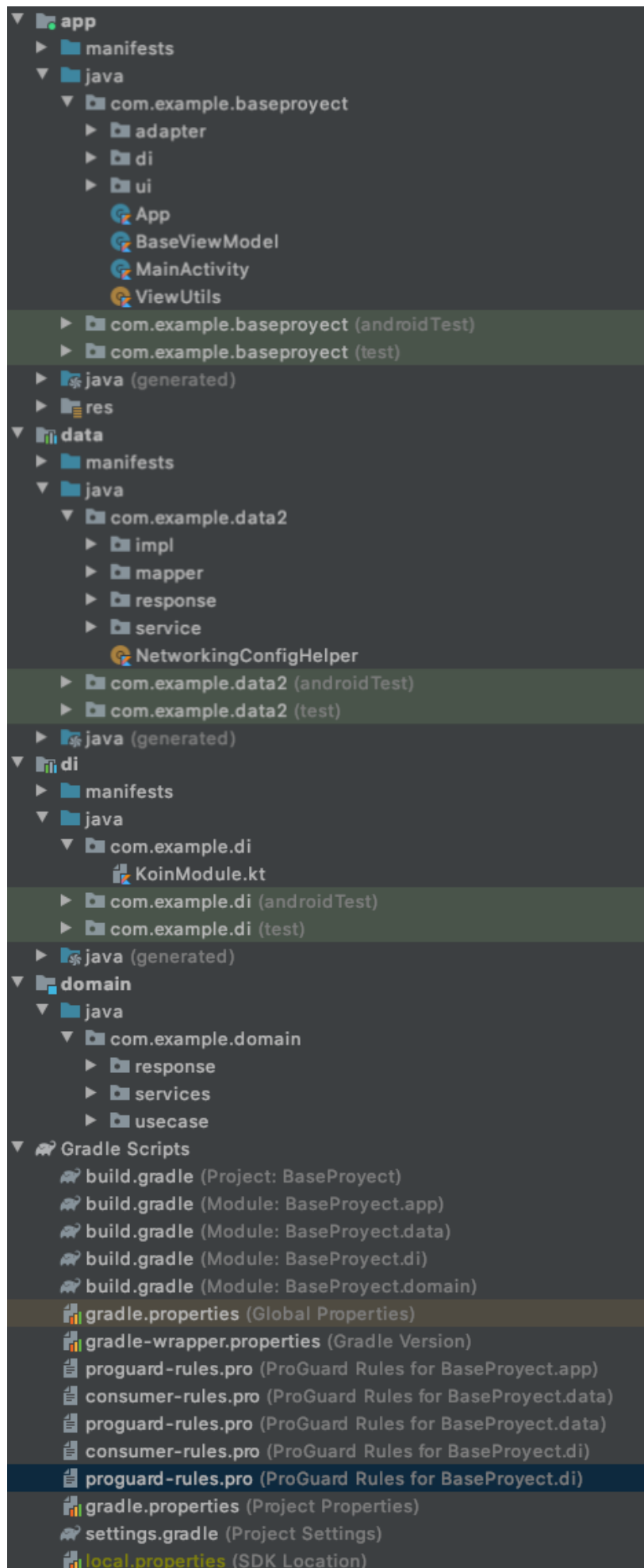


Figura 4.4: Vista con la implementación de la arquitectura de la aplicación separada por módulos.

Como podemos observar en la figura 4.4, el proyecto cuenta con una correcta implementación de una arquitectura CLEAN con módulos, conectados a través de su correspondiente módulo de inyección de dependencias.

La capa de Dominio es el módulo central de esta arquitectura, siendo una de las principales razones por las que no debería tener dependencias con otras capas. En esta capa, los casos de uso ayudarán a crear buenos Presenters / ViewModels, ya que la capa de presentación solo ejecutará los casos de uso y notificará la vista (separación de preocupaciones + principio de responsabilidad única)

La capa de Data implementa la Interfaz del Repositorio del Dominio y se encarga de combinar una o múltiples Fuentes de Datos. Este módulo se extrae de las interfaces remotas (cada fuente de datos se une a una implementación). Este desacoplamiento hace que la implementación del repositorio sea independiente de sus fuentes de datos, evitando cambios al cambiar una implementación de fuente de datos.

Los repositorios ya esperan de las fuentes de datos los modelos de dominio, por lo que empuja la responsabilidad de mapear de datos a dominio a cada implementación de fuente de datos individual (o de dominio a datos en caso de que esté enviando algo a la fuente de datos).

Tener módulos con las reglas de dependencia correctas significa que nuestro dominio no tiene ninguna dependencia en ninguna otra capa. Debido a que no hay dependencias con ninguna biblioteca de Android, la capa de dominio debería ser un módulo Kotlin. Este es un límite adicional que evitará contaminar nuestra capa más valiosa con clases relacionadas con el framework. También promueve la reutilización en todas las plataformas en caso de que cambiemos el Framework, ya que nuestra capa de dominio es completamente agnóstica.

La inyección de dependencias (DI) está estrechamente relacionada con dos conceptos SOLID: inversión de dependencias, que establece que los módulos de alto nivel no deben depender de módulos de bajo nivel, ambos deben depender de abstracciones; y el principio de responsabilidad única, que establece que cada clase o módulo es responsable de una sola pieza de funcionalidad.

DI apoya estos objetivos al desacoplar la creación y el uso de un objeto. Le permite reemplazar dependencias sin cambiar la clase que las usa y también reduce el riesgo de modificar una clase porque una de sus dependencias cambió.

En nuestro caso se optó por el uso de Koin¹⁸ como biblioteca de inyección de dependencia.

Uno de los principales objetivos es mejorar la reutilización de su código y reducir la frecuencia con la que es necesario realizar un cambio en alguna clase en particular. La inyección de dependencia respalda estos objetivos al desacoplar el proceso de creación de la utilización propiamente dicha de un objeto. Eso le permite reemplazar dependencias sin cambiar la clase que hace uso de ellas. También reduce el riesgo de que tenga que cambiar una clase solo porque una de sus dependencias cambió.

Como mencionamos, la capa de presentación generalmente posee la utilización de diferentes patrones que ayudan a la separación de responsabilidades, dividiendo la interfaz de la lógica detrás. Es por eso que se decidió la implementación del patrón Model View ViewModel¹⁹ (MVVM) basado principalmente en el apoyo que le ha estado brindando Google. No solo han creado una clase ViewModel para usar como padre de los modelos de vista, también hay un gran uso del patrón en presentaciones y muestras oficiales de Android. Además, MVVM se utiliza ampliamente en el desarrollo de Android actual y se combina muy bien con los componentes de la arquitectura de Android como Live Data y DataBindings. De manera similar a otros patrones como MVP, MVC y otros, este patrón se concentra en la conservación de información entre los distintos cambios de estados de la aplicación, donde Un objeto ViewModel proporciona los datos para un componente de IU específico, como un fragmento o una actividad, y también incluye lógica empresarial de manejo de datos para comunicarse con el modelo.

- La **Vista**: informa al ViewModel sobre las acciones del usuario
- El **ViewModel**: expone flujos de datos relevantes para la Vista.
- El **Modelo**: abstrae la fuente de datos. ViewModel trabaja con el modelo para obtener y guardar los datos.

¹⁸ "Koin - The Kotlin Injection Framework | Koin." <https://insert-koin.io/>. Se consultó el 19 junio 2021.

¹⁹ "The Model-View-ViewModel Pattern - Xamarin | Microsoft Docs." 7 ago. 2017, <https://docs.microsoft.com/en-us/xamarin/xamarin-forms/enterprise-application-patterns/mvvm>. Se consultó el 19 junio 2021.

```

private val predictorViewModel by viewModel<FragmentTravelPredictionViewModel>()

override fun onCreateView(view: View, savedInstanceState: Bundle?) {
    super.onCreateView(view, savedInstanceState)

    predictorViewModel.liveData.observe(::getLifecycle, ::updateUI)

    predictorViewModel.getCurrentPrediction(
        listaPuntos, algoritmo
    )
}

private fun updateUI(data: Event<FragmentTravelPredictionViewModel.Data>) {

    cardLoader.visibility = View.GONE
    recyclerView.visibility = View.GONE
    val cardDetailData = data.getContentIfNotHandled()
    when (cardDetailData?.status) {
        FragmentTravelPredictionViewModel.Status.LOADING -> {
            setLoader()
        }

        FragmentTravelPredictionViewModel.Status.CONTENT_DATA -> {
            setUIValues(data.peekContent().data)
        }

        |
        FragmentTravelPredictionViewModel.Status.ERROR -> {
            invokeAlertDialog(
                activity = requireActivity(),
                message = data.peekContent().data.toString(),
                positiveButtons = data.peekContent().dataAlternativa.toString()
            )
        }
    }
}
}

```

Figura 4.5: Ejemplo comunicación viewModel-Actividad para observar y ejecutar los eventos de la lógica en la vista.

4.4 Requerimientos Funcionales

La identificación, análisis y gestión de los requerimientos funcionales es una actividad crítica en la ingeniería del software, que bajo una gestión adecuada es fundamental para asegurar el éxito de los proyectos. Es por eso que estos, nos permitirán tener la capacidad de expresar en términos de cuál debe ser el comportamiento de la solución, la información y descripción suficiente que debe

manejar y la influencia con la claridad necesaria para satisfacer los requerimientos de los interesados en el proyecto.

En el siguiente apartado se efectúa un listado correspondiente a la funcionalidad que debe proveer la herramienta desarrollada, mencionando también aquellas cuestiones básicas que motivan la especificación de cada ítem del listado.

- La herramienta deberá ser capaz de clasificar los datos históricos de uso, para ser pre procesados y consumidos por los servicios del servidor.
- La herramienta deberá ofrecer la comparación entre distintos algoritmos para cálculo de datos.
- La aplicación deberá calcular cuánto tiempo conlleva desplazarse desde un punto de origen hacia un destino especificado por el usuario dependiendo de la línea, el horario y entre otros factores.
- Los resultados obtenidos a partir de los algoritmos, deben ser mostrados de forma clara e intuitiva.
- La herramienta debe ser capaz de obtener información específica desde una base de datos basada en SQL, con una estructura de tablas específica.
- La herramienta debe proveer un conjunto de servicios que agilicen la transformación y el consumo de los algoritmos.
- La aplicación debe estar orientada a brindar una interfaz centrada en la usabilidad y accesibilidad de los usuarios.
- La aplicación debe brindar diferentes opciones para que el usuario seleccione un punto o lugar de origen y destino.
- La aplicación debe ser capaz de visualizar un mapa completo de la ciudad incluyendo el nombre de las calles y avenidas.
- Brindar de forma clara e intuitiva los distintos recorridos de las unidades de colectivos disponibles pudiendo visualizar por cuales calles circula.
- La aplicación debe brindar la ubicación del lugar seleccionado así como también la ubicación de las paradas más cercanas a ese punto.
- La aplicación debe ser capaz de obtener los cálculos de tiempo del servidor clasificados por unidad y distinguir las paradas de colectivo más cercanas.

- La aplicación debe brindar una información detallada de la ubicación de cada punto, marcador seleccionado en el mapa y marcador de las unidades disponibles.
- La aplicación permitirá la configuración de los diferentes algoritmos para ofrecer una variedad de resultados a elección.
- La aplicación configura por defecto una línea base de colectivos y un algoritmo listo para ser usado en la realización de los cálculos de tiempos.

Brindar a los usuarios un mapa de la ciudad, incluyendo los nombres de las calles y avenidas complementando con la posibilidad de seleccionar y visualizar los recorridos que disponen las líneas de colectivos fue el requerimiento inicial con el que debió disponer la herramienta. También el usuario debe poder ingresar dos puntos específicos de la ciudad y calcular el tiempo que involucra movilizarse entre ellos, incluyendo en qué esquina debe subirse y cuál bajarse de la unidad, siendo dichas esquinas las más cercanas a los objetivos establecidos por el cliente. Cabe destacar que para el ingreso de una coordenada en particular puede realizarse de dos maneras distintas, por un lado simplemente haciendo contacto en la pantalla en la ubicación del mapa deseada, o simplemente considerando la ubicación actual de la persona, para lo cual se requiere habilitar la ubicación al dispositivo. Al seleccionar un punto, en lugar de mostrar las coordenadas correspondientes, se optó por proveer el nombre de la calle o avenida y su número aproximado ya que provee más flexibilidad y sencillez a la hora de su lectura.

Un detalle no menor, que es importante mencionar que nunca se accede a los contactos, a las demás aplicaciones ni cualquier otro contenido o aplicación del móvil que se encuentra corriendo la herramienta, evitando así la utilización o manejo de información propia y privada del usuario evitando ataques intencionados.

Debido a que la ciudad dispone de varias unidades que circulan por diferentes sectores, se buscó brindar al usuario la posibilidad de establecer una línea cómo por defecto, adaptándose a su uso cotidiano, y poder cambiarla a gusto.

4.5 Escenarios de Calidad

Los escenarios de calidad pueden ser considerados como requerimientos específicos que permiten realizar una medición acerca de cómo el sistema debe proveer ciertas características o propiedades en una combinación deseada. Permitiendo considerar y evaluar características de calidad de manera concreta, mejorando considerablemente el diseño y por lo tanto la implementación de la herramienta.

Un escenario de calidad es una forma de captar un requerimiento específico de un atributo de calidad. El formato escogido e implementado es fuente-estímulo-respuesta.

Está conformado por las siguientes partes:

- Fuente de estímulo.
- Respuesta.
- Estímulo.
- Artefacto.
- Entorno.
- Medición de la respuesta.

La fuente de estímulo corresponde a una entidad, la cual puede tratarse de algún tipo en particular, es decir, una persona, un sistema de ordenador o cualquier otro dispositivo. Dicha entidad es responsable de producir un estímulo determinado, que es una condición que debe ser evaluada cuando entra a un sistema. El artefacto es aquel elemento que está siendo afectado por el estímulo, definido previamente y puede incluir algunas partes en particular o simplemente todo el sistema completo. El estímulo se produce bajo ciertas condiciones, las cuales son conocidas como entorno. La respuesta es la salida que se produce luego de la llegada del estímulo, la misma debe ser medible y comparable de alguna manera en particular para que el requisito pueda ser probado y validado.

La selección de los escenarios fueron definidos a partir de los requerimientos no funcionales que se detectaron como prioritarios en la confección de la arquitectura y diseño del software. Dentro de ellos podemos mencionar: la modificabilidad que busca la integración entre el sistema de algoritmos y api's dentro de la aplicación, la escalabilidad que implica extender los distintos cálculos y flujos para el usuario dentro de la aplicación, y la usabilidad que se define a partir de una aplicación sencilla, intuitiva y práctica para los distintos usuarios.

- Escenario de calidad 1: Modificabilidad. El administrador del sistema desea modificar una de las direcciones de la API. El cambio debe afectar a un solo módulo, en el cual las llamadas a la API son manejadas, manteniendo la misma estructura y los cambios no deben afectar a más de una clase. El tipo de arquitectura usada, más el framework, permiten hacerlo de forma rápida, eficiente y afectando sólo a la porción de código correspondiente.
- Escenario de calidad 2: Escalabilidad. El administrador del sistema desea agregar una complejidad o un nuevo algoritmo de cálculo de tiempo, manteniendo la aplicación móvil sin modificaciones. El cambio sólo afecta a un archivo de configuración de la aplicación, siendo necesarios cambios mínimos en el servidor (agregar a la base de datos la nueva entrada, realizar el preprocesamiento y determinación de resultados para ese algoritmo afectado).
- Escenario de calidad 3: Escalabilidad. La arquitectura del sistema permite una rápida integración para un nuevo caso de uso o regla del negocio gracias a la guía de conceptos de una arquitectura CLEAN. Agilizando la organización de los módulos de desarrollo, permite que el mismo sea fácilmente consumido.
- Escenario de calidad 4: Usabilidad. Un usuario final desea localizar una ubicación y toda su información de recorrido. El sistema provee diferentes alternativas para satisfacer de diferentes maneras la geolocalización de lugares de acuerdo a su necesidad.
- Escenario de calidad 5: Usabilidad. Un usuario final desea localizar una ubicación realizando el mínimo de movimientos dentro de la aplicación. La aplicación contará con toda la facilidad para la ubicación dentro del mapa de forma intuitiva y sencilla

- Escenario de calidad 6: Portabilidad. Se busca que la herramienta pueda funcionar de manera óptima independientemente de en qué plataforma android sea utilizada.

4.6 Desarrollo del servidor

4.6.1 Arquitectura del servidor

La estructura correspondiente al sistema del servidor se encuentra conformada a grandes rasgos por 4 elementos principales, los cuales interactúan entre sí para asegurar el correcto funcionamiento. Dichos elementos son los Controladores, Servicios, Repositorios y el Dominio, los cuales se complementan con un conjunto de capas que detallaremos más adelante.

Inicialmente al realizar un pedido proveniente de un servicio desde la interfaz del usuario, se llama a su respectivo controlador. Dicho controlador puede recibir ciertos parámetros de entrada, los cuales son procesados y a continuación llamará a uno o más servicios necesarios para realizar la lógica de negocios para retornar la respuesta correspondiente. En caso de requerir información procedente de la base de datos, se hace uso de los repositorios específicos necesarios. El dominio representa y se codifica como una biblioteca de clases con las entidades de dominio que capturan datos y comportamiento. Haciendo uso de los principios de omisión de persistencia, el dominio no debe incluir detalles relacionados con la persistencia de la información. Esto significa que de ninguna manera se pueden incluir dependencias (al menos no de manera directa) en la infraestructura. Las clases de entidad que se encuentran del dominio deben ser POCO (del inglés Plain Old CLR Object) lo que implica no tener dependencias con algún framework particular, independientemente de las librerías utilizadas para acceder o persistir la información en la base de datos.

Durante el inicio del proyecto, surgió la incertidumbre acerca de la inclusión o no de una capa de servicios. Pero a pesar de que implica un desarrollo más extenso, nos permite

tener una mayor granularidad a la hora de distribuir las diferentes tareas, evitando así incluir lógica de negocio en los controladores o en los repositorios.

Cómo se mencionó en la sección previa, se optó por implementar una arquitectura por capas debido a que el sistema se adapta de la mejor manera a este tipo de aplicaciones. En donde cada una de las capas se considera independiente del resto, posibilitando a través de dependencias, la comunicación interna y el flujo de información. Buscando limitar las dependencias cíclicas y solo estableciendo vínculos con los niveles superiores.

A lo largo del desarrollo surgieron incertidumbres particulares a resolver con respecto a las siguientes cuestiones. En primer lugar, la información correspondiente al dataset se encontraba en formato JSON, distribuido en varios archivos de gran longitud. Pero la información relacionada a los cálculos matemáticos y la lógica de negocios se decidió almacenarla en base de datos SQL, principalmente por razones de rendimientos y extensibilidad ante futuras modificaciones. Es por eso que se optó por realizar dos capas diferentes para manejar dicha situación, esto significa realizar una implementación de una capa para manejar la lectura de datos a través de archivos, y otra capa independiente relacionada con la base de datos. Esta decisión se fundamenta en que ambas capas si bien poseen una función similar, la cual consiste en la lectura de información de un archivo (aunque de diferente extensión y formato), éstas hacen uso de metodologías y bibliotecas totalmente diferentes para operar con los datos. Si bien se podría haber realizado todo dentro de una misma capa, esta separación nos asegura mayor granularidad entre las tareas.

La segunda incertidumbre corresponde con los servicios y la lógica correspondiente a la regresión lineal, una de las metodologías empleadas para realizar la predicción de tiempo. Similar a la medida explicada anteriormente se optó por realizar una capa para regresores independiente de los servicios de la aplicación. Para realizar la implementación de los regresores se utilizó la librería ML.Net, y es por eso que se decidió abstraer toda la lógica matemática correspondiente dentro de una capa para que ante cualquier cambio, ya sea de adhesión o eliminación de lógica, no impacte en los servicios de negocios que no disponen lógica de machine learning.

Las capas o proyecto que dispone la aplicación desde el apartado del servidor, se puede apreciar en la figura 4.6.

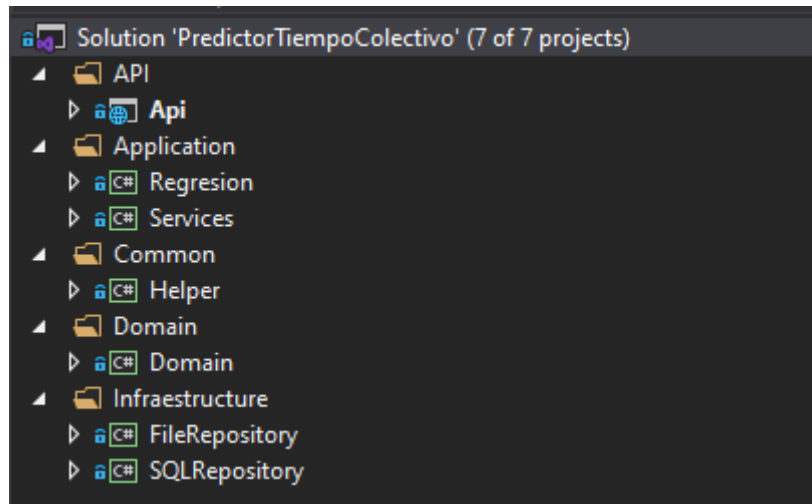


Figura 4.6: Descripción de las capas que componen el lado del servidor.

Las capas principales disponen las siguientes características y funcionalidades:

- **Capa de Domain (dominio):** Las capas de dominio (capa central) se implementan en el centro y nunca dependen de ninguna otra capa. Por lo tanto, lo que hacemos es que creamos interfaces para la capa de persistencia y estas interfaces se implementan en las capas externas. Esto también se conoce como Principio de Inversión de Dependencia o DIP. Es el principal responsable de representar conceptos del negocio, información sobre la situación del negocio y reglas de negocios.
- **Capa de Infraestructure (infraestructura):** Es la capa que se encarga de la persistencia. En la aplicación desarrollada, hemos utilizado e implementado Entity Framework que ya implementa un patrón de diseño de repositorio. DbContext será la unidad de trabajo y cada DbSet es el repositorio. Esto interactúa con nuestra base de datos utilizando los diferentes repositorios creados. También se dispone de un proyecto independiente para realizar la escritura de datos que pueden ser almacenados en archivos físicos.
- **Capa de Application (aplicación):** Capa de servicio donde podemos implementar la lógica de los diferentes algoritmos y los correspondientes cálculos de tiempo. Dispone de la lógica de negocio en donde a partir de los parámetros de entrada, se realizan operaciones necesarias para retornar los resultados correspondientes en el menor tiempo posible. Se divide en dos

proyectos, uno correspondiente a la lógica de negocios con algoritmos propios desarrollados y el segundo encargado de realizar la implementación de la regresión lineal, usando la librerías previamente mencionadas.

- **Capa Common:** En esta capa, agregamos diferentes funciones y métodos particulares que pueden ser utilizadas por alguna de las capas restantes y que no dispone de lógica de negocio.
- **Capa de Api (presentación):** Api Rest que dispone una serie de controladores en la cual cada uno posee una serie de endpoints a ser utilizados por el frontend. Dichos endpoints reciben una petición HTTP con cierta estructura y parámetros de entrada predefinidos y retornan los resultados pertinentes en caso de ser válidos. La información intercambiada se encuentra representada en formato JSON (del inglés, JavaScript Object Notation).



Figura 4.7: Descripción de la capas y su comunicación interna a nivel del servidor

Cómo puede observarse en la figura 4.7, la capa de presentación posee dependencias directas con aquellos servicios que determinan la lógica de negocios pero no con la

sección relacionada al acceso a la información almacenada. Esto significa, por ejemplo, que ante un posible cambio en la base de datos, no se requieren cambios en los endpoints existentes, ya que no hay referencias entre sí. Esto puede visualizarse a nivel implementación en la figura 4.8, observando las referencias que dispone el proyecto de API, donde solo se aprecian los elementos de los servicios con la lógica de negocios, pero no los repositorios.

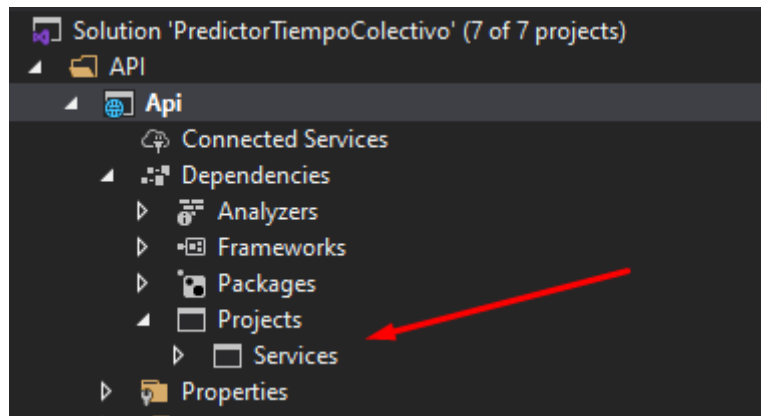


Figura 4.8: Descripción de las referencias de uno de los proyectos

Además se llevó a cabo una implementación de la técnica de inyección de dependencias la cual hace que una clase sea independiente de sus dependencias. Logrando desacople en el uso de un objeto con respecto a su creación. Esto le ayuda a seguir los principios de inversión de dependencia y única responsabilidad de SOLID.

4.6.2 Implementación de Modelos, Repositorios y Controladores

Cómo se mencionó previamente los controladores, repositorios y el dominio se encuentran en proyectos separados e independientes que se comunican a través de referencias.

El paso inicial para llevar a cabo la implementación del back-end, se corresponde a la creación de las diversas entidades utilizadas, especificando sus propiedades que la componen y sus relación con las demás. Cómo se mencionó previamente, dichas clases

son POCO lo que implica no tener dependencia alguna con otra librería externa e implementan el dominio de negocios de la aplicación.

Una vez finalizado el paso anterior, se procedió con la creación de los repositorios y sus interfaces que permiten trabajar las diversas operaciones sobre la base de datos permitiendo accionar sobre las tablas. Dichas acciones se corresponden a consultas bajo ciertos parámetros de búsqueda, cambios de valores, inserción de nueva información o eliminación de valores específicos. La utilización del ORM, permite realizar estas acciones de manera sencilla al hacer uso de las claves primarias y los atributos particulares de cada tabla.

Al implementar Entity Framework Core, se hizo uso de las migraciones para mantener la consistencia entre las tablas que conforman la base de datos y las entidades presentes en la capa de dominio. Las migraciones posibilitan trabajar con un sistema versionado de la base de datos. Es decir, que en lugar de realizar un script específico de base de datos para replicar cambios sobre el dominio, se realiza una migración con Entity Framework, actualizando automáticamente la última versión al realizar la ejecución de un comando específico en dicha migración. A si mismo, estas proporcionan la alternativa más recomendable para controlar los cambios de esquema en la base de datos, sin necesidad de crear una base de datos totalmente nueva.

Por ejemplo, en la figura 4.9 se incluye una migración efectuada para incorporar la tabla en la cual se almacena la información correspondiente a las paradas de los recorridos base, especificando el nombre de la tabla y las columnas que la componen.

```

1      using Microsoft.EntityFrameworkCore.Migrations;
2
3      namespace SQLRepository.Migrations
4      {
5          1 reference
6          public partial class AgregarTablaParadas : Migration
7          {
8              1 reference
9              protected override void Up(MigrationBuilder migrationBuilder)
10             {
11                 migrationBuilder.CreateTable(
12                     name: "Paradas",
13                     columns: table => new
14                     {
15                         Id = table.Column<int>(nullable: false)
16                             .Annotation("Sqlite:Autoincrement", true),
17                         Latitud = table.Column<double>(nullable: false),
18                         Longitud = table.Column<double>(nullable: false),
19                         RecorridoId = table.Column<int>(nullable: false),
20                         Orden = table.Column<int>(nullable: false)
21                     },
22                 constraints: table =>
23                 {
24                     table.PrimaryKey("PK_Paradas", x => x.Id);
25                 });
26             }
27
28             1 reference
29             protected override void Down(MigrationBuilder migrationBuilder)
30             {
31                 migrationBuilder.DropTable(
32                     name: "Paradas");
33             }
34         }
35     }

```

Figura 4.9: Ejemplo de una de las migraciones realizadas

Posteriormente se procedió con el desarrollo de los servicios en donde se implementa la lógica de negocios y los algoritmos correspondientes para que la aplicación funcione correctamente. Haciendo uso de los repositorios previamente mencionados, se accede a la información de la base de datos para realizar los cálculos pertinentes y retornar a la parte visual los resultados en el menor tiempo posible.

Luego, se continuó con los controladores, siendo estos los responsables de proveer los resultados al sector del cliente. Los controladores se encuentran conformados por una serie de endpoints mediante los cuales el cliente accede a cierta funcionalidad específica. Para lograr la comunicación se hace uso del protocolo de comunicación HTTP

(del inglés Hypertext Transfer Protocol) que permite realizar la petición de datos y recursos.

Cabe destacar que en la arquitectura REST, cada solicitud realizada incluye un método de petición asociado. Por ejemplo, un método del tipo PUT refleja una actualización en la información, de manera idempotente lo que significa que producirá el mismo resultado si se ejecuta múltiples veces. Un método GET es utilizado para obtener o consultar información. El método POST se utiliza para crear una nueva instancia, pero a diferencia del PUT, no es idempotente, ya que si se ejecuta varias veces procede con la creación de varios elementos. Además es requerido para cada método realizar una definición de la ruta a través de la cual se accede a dicho endpoint y los parámetros de entrada en caso de ser requeridos. Esto puede visualizarse de manera clara en la interfaz de Swagger, en donde además se pueden realizar las pruebas pertinentes con diversos parámetros de entrada.

Se hizo uso de Swagger como herramienta para realizar las pruebas correspondientes de los endpoints.

A lo largo del desarrollo, se encontró una limitación importante en cuanto a los pilares teóricos que conforman la arquitectura Rest al utilizar HTTP. La funcionalidad encargada de calcular el tiempo correspondiente a dos ubicaciones se diseñó originalmente como un método de tipo GET, pero como requiere un número de parámetros de entrada bastante alto y complejo se optó por realizar un método de tipo POST y enviando la información a través de un cuerpo (conocido como body), aceptando que no es teóricamente correcto.

Para llevar a cabo esta capa, fue crucial mantener la comunicación con el desarrollo de la parte visual, para poder establecer un estándar y definir el formato de los parámetros de entrada y las respuestas retornadas.

4.7 Enfoque aplicación móvil

4.7.1 Introducción

En esta sección, se incluirán las medidas adoptadas en la parte del cliente, también conocido como front-end. Se realiza una descripción general de la arquitectura y como se llevó a cabo la implementación, justificando las decisiones tomadas y detallando el contenido de los componentes que conforman la parte visual de la solución.

4.7.2 Definición de la arquitectura

El desarrollo de una herramienta capaz de cumplir con los principios y lineamiento de las buenas prácticas de desarrollo de software, los estándares de modelado y un desarrollo sustentable, nos desafiaron a tomar una solución concisa y bien aplicada para la implementación de cara al usuario.

Basados en las premisas de los paradigmas de programación, los principios de diseño y componentes y canalizando la estructura de la arquitectura del sistema, pudimos encontrar la mejor combinación de conceptos y técnicas para hacer un sistema completo y de gran adaptabilidad al problema propuesto.

El desarrollo de software es “recursivo” o “fractal”, por lo que los principios que se aplican a funciones o clases, también pueden aplicarse a más alto nivel como por ejemplo módulos de una arquitectura. Este concepto permite definir la arquitectura elegida acorde para nuestro proyecto: Clean Architecture.

Aunque todas estas arquitecturas varían algo en sus implementaciones, son muy similares. Todos tienen el mismo objetivo, que es la separación y delegación de responsabilidades. Todos logran esta separación dividiendo el software en capas. Cada uno tiene al menos una capa para las reglas comerciales y otra para las interfaces. Su nombre popularizado por Robert Cecil Martin, conocido como “Uncle Bob” que se basa en la premisa de estructurar el código en capas contiguas, es decir, que solo tienen comunicación con las capas que están inmediatamente a sus lados.

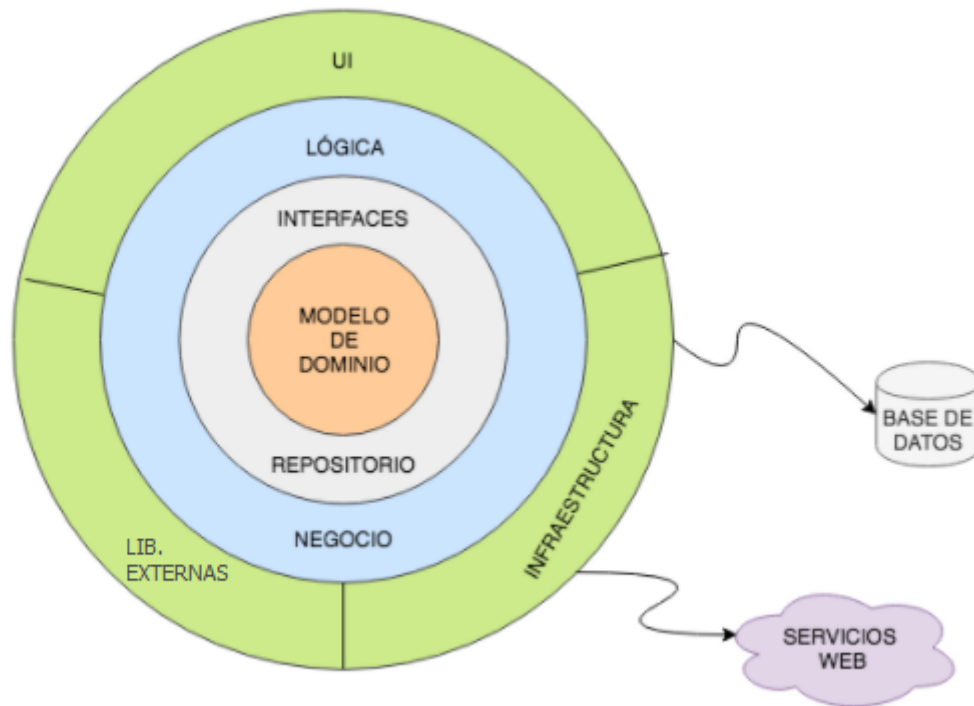


Figura 4.10: Estructura arquitectura clean

La regla primordial que hace que esta arquitectura funcione es la regla de dependencia. Esta regla dice que las dependencias del código fuente sólo pueden apuntar hacia adentro. Nada en un círculo interno puede saber nada sobre algo en un círculo externo. En particular, el nombre de algo declarado en un círculo exterior no debe ser mencionado por el código en un círculo interior. Eso incluye funciones, clases, variables, o cualquier otra entidad de software nombrada.

Del mismo modo, los formatos de datos utilizados en un círculo exterior no deben ser utilizados por un círculo interior, especialmente si esos formatos son generados por un marco en un círculo exterior. Esto quiere decir que las capas exteriores no deben depender de sus implementaciones, sino que la implementación depende de la abstracción. Por lo mencionado anteriormente y lo descrito en la sección del servidor, se puede concluir que ambas arquitecturas disponen de comportamientos similares a la hora de estructurar el código.

Por otro lado, basados en uno de los cinco principios S.O.L.I.D., la Inyección de Dependencias es un concepto que hoy en día se ha convertido en uno de los principios más usados en el mundo del desarrollo de aplicaciones. Es igual de importante para el desarrollo de componentes back-end (donde se implementó inicialmente), como de

front-end. Como todo patrón de diseño, DI tiene como finalidad solucionar un problema común que los programadores encuentran en la construcción de aplicaciones. Esto es, mantener los componentes o capas de una aplicación lo más desacopladas posible. Busca que sea mucho más sencillo reemplazar la implementación de un componente por otro. Así, evitar un gran cambio o impacto en la aplicación podría originar que deje de funcionar por completo.

Para cumplir con dicho objetivo, DI nos permite inyectar comportamientos a componentes haciendo que nuestras piezas de software sean independientes y se comuniquen únicamente a través de una interfaz. Esto extrae responsabilidades a un componente para delegarlas en otro, estableciendo un mecanismo a través del cual el nuevo componente puede ser cambiado en tiempo de ejecución.

4.7.3 Modelado de la arquitectura

Aplicando los principios de diseño y componentes, definidos por la arquitectura elegida podemos observar la separación en capas del proyecto en la figura 4.11.

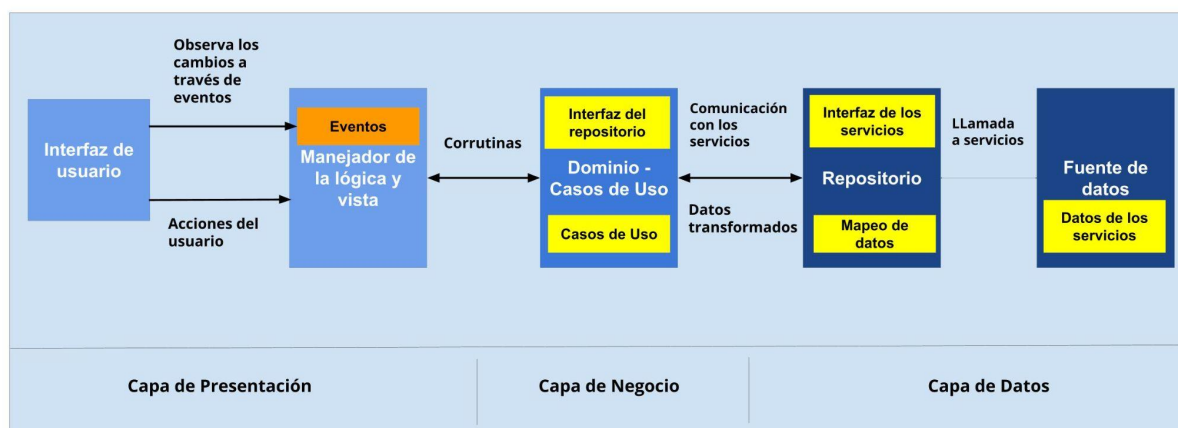


Figura 4.11: Estructura de la arquitectura estructurada en capas.

Para el proyecto presentado se decidió crear una arquitectura limpia que consta de tres capas que se subdividen en cinco:

- La capa de presentación (app)
- La capa de la lógica de negocio (domain)

- La capa de datos (data)
- La capa de inyección de dependencias (di)

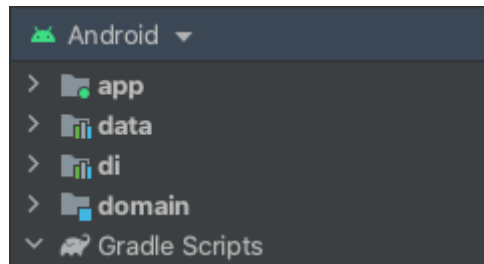


Figura 4.12: Estructura de módulos de la aplicación.

4.7.3.1 La capa de presentación

La capa de presentación es la capa de usuario, la interfaz gráfica que captura los eventos del usuario y que muestra los resultados. También realiza operaciones, como verificar que no hay errores de formato en la entrada de datos del usuario y formatea datos para que se muestren de una manera determinada.

En nuestro ejemplo, estas operaciones se reparten entre la capa de UI y de ViewModel:

- La capa de UI contiene las actividades y los fragmentos, capturando eventos del usuario y mostrando datos.
- La capa de ViewModel, responsable del manejo de la lógica de UI, formatea los datos para que la UI los muestre de una determinada manera y verifica que las entradas del usuario tengan el formato correcto. Así mismo es el encargado de comunicarse con las interfaces de los Casos de Uso, estableciendo el enlace con la capa de Dominio ò Negocio.

4.7.3.2 La capa de la lógica de negocio

En esta capa se establecen todas las reglas que debe cumplir un negocio. Para ello, recibe los datos que le facilita el usuario y hace las operaciones necesarias.

Es la capa más estable y la que indica qué está ocurriendo en la arquitectura de software desarrollada. Es por eso que esta capa posee el control de las operaciones más importantes dentro de la aplicación, que es la base de la arquitectura, y no es menor pasarlo por alto. Como podemos ver en la figura nro. 4.10, la capa de dominio se

encuentra en la parte central de la arquitectura y si observamos las referencias de dependencias entre clases, podemos ver que todos conocen a la capa de dominio, pero esta no conoce a las otras capas, esto se debe a dos razones: la comunicación y navegación entre capas es hacia dentro, por lo cual ir de adentro hacia afuera viola el concepto de esta arquitectura, tal cual describe Robert Cecil Martin y como muestra la figura nro. 4.10, “La regla primordial que hace que esta arquitectura funcione es la regla de dependencia. Esta regla dice que las dependencias del código fuente sólo pueden apuntar hacia adentro”. Por otro lado, esta capa al ser un módulo Java/Kotlin a diferencia de las otras capas que son módulos Android, si dominio conociera otra capa, podría tener la posibilidad de usar sus componentes y así encargarse de tareas que esta capa no le corresponde.

Dentro de esta capa podemos encontrar los Use Cases, encargados de definir los pasos en el correcto funcionamiento de un escenario posible y por otro lado, las interfaces que permiten la comunicación con la capa de datos.

4.7.3.3 La capa de datos

En esta capa es donde están los datos y donde se puede acceder a los mismos. Estas operaciones se reparten entre la capa de Repository y Datasource:

- La capa de Repository es la que realiza la lógica del acceso a datos. Su responsabilidad es obtenerlos y comprobar dónde están, decidiendo dónde buscarlos en cada momento. Es decir, define el flujo de acceso a los datos. Por otro lado, realiza el mapeo de los objetos provenientes de los servicios a objetos tangibles para las capas superiores (capa de presentación).
- La capa de Datasource es la que realiza la implementación para poder acceder a los datos, es decir, se comunica con la librería de conexión a los servicios. En este nivel además se serializan en objetos los campos de las respuestas JSON.

4.7.3.4 Capa de inyección de dependencias

Este módulo resulta ser el comunicador principal de la aplicación, ya que permite que los módulos se conozcan y se puedan enviar y recibir datos entre ellos.

El objetivo de esta capa es recorrer transversalmente toda la aplicación, definiendo los puntos de interconexión entre módulos, y así cuando una clase quiera conocer las clases de otro módulo, pueda hacerlo sin ninguna restricción de implementación.

Como podemos observar en la figura 4.6 y 4.16 esta capa conoce o puede acceder al resto de las capas, ya que él les brindara el mecanismo para comunicarse con las restantes áreas de la aplicación.

4.7.4 Implementación de la arquitectura

En este punto se expande un poco más en las cuestiones técnicas que permitieron la implementación de una arquitectura separada por capas de responsabilidades, y cómo se establecen las interconexiones entre ellas.

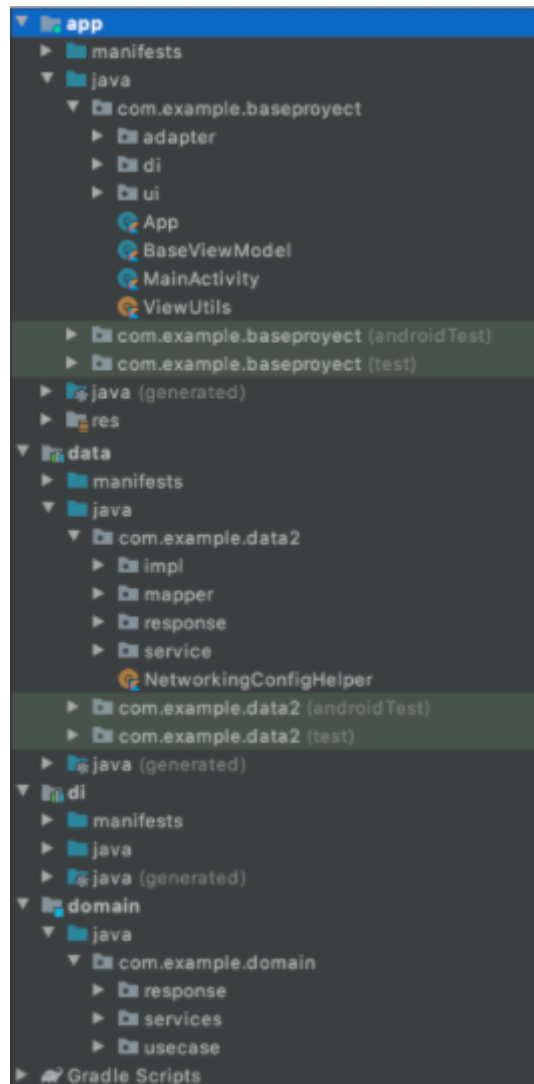


Figura 4.13: Implementación de la arquitectura en la aplicación.

El primer punto a la hora de implementar la base de la arquitectura, es definir el tipo de módulos que representarán a cada una de las capas. Esta decisión es tomada en referencia de cada patrón implementado y las dependencias que requieren el uso de los componentes dentro de ella.

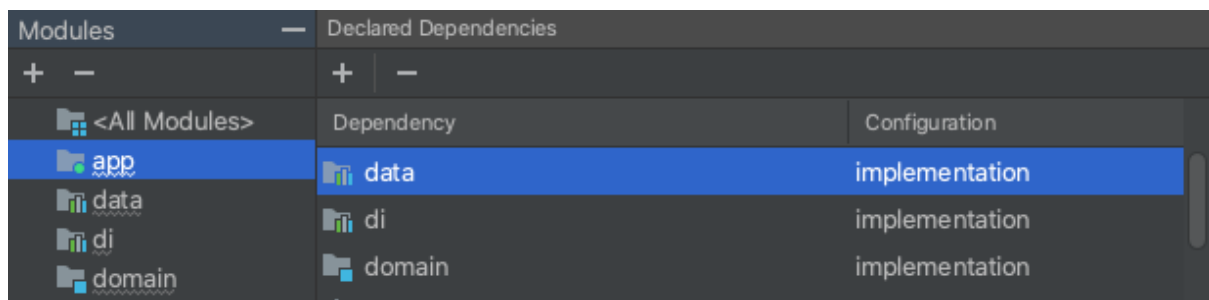
Aquí podemos observar que la capa de Datos, utiliza un módulo Android, debido a que requiere la utilización de librerías que permitan la comunicación con los servicios de la aplicación y al procesamiento de las respuestas entregadas por ellos. La capa de Dominio, a diferencia presenta un módulo Java/ Kotlin, como se mencionó anteriormente, es el módulo encargado de que la aplicación represente correctamente los pasos para ejecutar una tarea o funcionalidad, independiente de cualquier librería del sistema operativo. La capa de Dependencia de inyección, implementa un módulo

Android, necesario para el conocimiento, interconexión y funcionamiento de toda la aplicación, necesitando por supuesto del conocimiento del sistema operativo. Por último la capa de Aplicación, al manejar la interacción con el usuario, los componentes de ui, el manejo de los hilos del sistema, donde se ejecutará la lógica y operaciones de procesamiento de datos, está implementado con un módulo Android.

Otro de los puntos más importantes a definir, es la relación de dependencias de cada módulo. Esta es la puerta que le permitirá a cada módulo conectarse con el módulo que desee. Según como hemos visto, la arquitectura presenta una navegabilidad de las capas exteriores hacia las capas más internas.

A continuación definiremos cuales son estas dependencias entre capas:

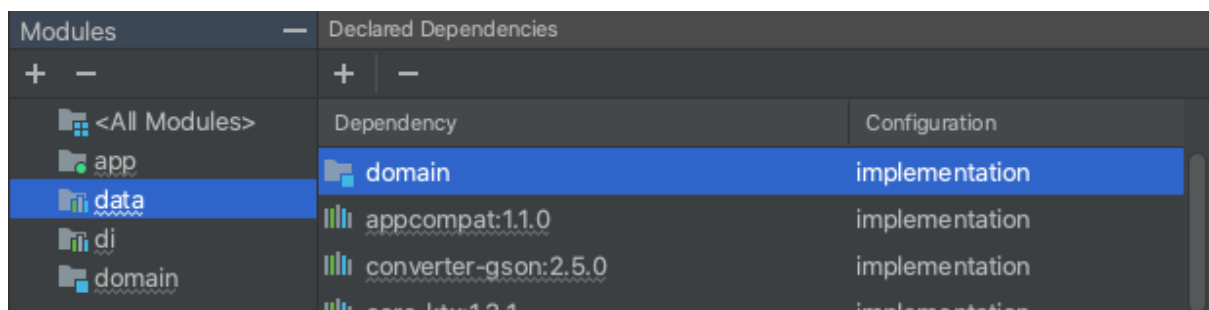
- La capa de aplicación mantiene una dependencia con la capa de dominio, datos y con la capa de inyección de dependencias.



Modules	Declared Dependencies	
+ -	+ -	
<All Modules>	Dependency	Configuration
app	data	implementation
data	di	implementation
di	domain	implementation
domain		

Figura 4.14: Árbol de dependencias para el módulo de aplicación.

- La capa de datos mantiene una dependencia con la capa de dominio.



Modules	Declared Dependencies	
+ -	+ -	
<All Modules>	Dependency	Configuration
app	domain	implementation
data	appcompat:1.1.0	implementation
di	converter-gson:2.5.0	implementation
domain	core-ktx:1.3.1	implementation

Figura 4.15: Árbol de dependencias para el módulo de datos.

- La capa de inyección de dependencias mantiene una dependencia con la capa de dominio y con la capa de datos.

Modules	Declared Dependencies	
+ -	+	-
<All Modules>	Dependency	Configuration
app	data	implementation
data	domain	implementation
di	appcompat:1.1.0	implementation
domain	converter-gson:2.5.0	implementation

Figura 4.16: Árbol de dependencias para el módulo de inyección de dependencias.

- La capa de dominio no mantiene o sujeta dependencia con otra capa.

Modules	Declared Dependencies	
+ -	+	-
<All Modules>	Dependency	Configuration
app	koin-android:2.0.1	implementation
data	koin-androidx-scope:2.0.1	implementation
di	koin-androidx-viewmodel:2.0.1	implementation
domain	kotlin-stdlib-jdk7:1.3.71	implementation

Figura 4.17: Árbol de dependencias para el módulo de dominio.

Mencionando los aspectos más importantes durante la implementación de la arquitectura de la aplicación, vamos a entrar en detalle en cada una de las capas, para ver los detalles que permiten la ejecución de cada una de las funcionalidades dentro de la aplicación presentada.

Primero comenzaremos con la implementación de la capa de inyección de dependencias. La inyección de dependencias (DI) es una técnica muy utilizada en programación y adecuada para el desarrollo de Android.

Hasta hace no mucho las soluciones acerca de poder hacer inyección de dependencias en Android se reducían prácticamente a una, Dagger²⁰, pero su curva de aprendizaje y su robusto código, dificultan la implementación en los proyectos. Es por eso que con la

²⁰ "Cómo usar Dagger en apps para Android"

<https://developer.android.com/training/dependency-injection/dagger-android?hl=es-419>.

llegada de Kotlin y las funcionalidades que nos brinda el propio lenguaje, surge Koin²¹. Koin es un framework que nos permite hacer una inyección de dependencias en Android mucho más ágil, pragmático y con mucho menos código que el necesario en Dagger. Su utilización permite simplemente declarar módulos, que incluyen dependencias potenciales, para ser utilizados en el proyecto e inyectarlos directamente en la clase de interés.

En nuestro caso, los módulos que definen las entidades que se inyectaran en algún punto de la aplicación son: el *use Cases Module*, el *repository Module* y el *networking Module*. Para iniciar el proceso de DI e indicar qué módulos estarán disponibles es necesario definir en la clase Base de la aplicación, el comando y la lista de módulos definidos en la capa de DI.

Por último, para realizar una inyección, Koin permite realizar inyecciones perezosas para un mejor funcionamiento. dentro de las clases, bajo la sentencia *by inject()*.

La capa de dominio o representante de la lógica de negocio, está subdividida en tres paquetes: Entities, Services, Use Cases. Las Entidades, representan aquellas clases que moldearán los objetos pertenecientes al dominio de cada una de las funcionalidades de la aplicación. Permitirán insertar las coordenadas en cada punto del mapa seleccionada, las líneas y colectivos disponibles en cada recorrido, los puntos de cada trayecto entre puntos seleccionados, entre otros.

El paquete de Services, incluye las interfaces que permiten a los casos de uso conectarse a la implementación de los servicios. Es decir, es la clase que permite la interconexión entre capas (la capa de negocio y la capa de datos). Para ello, distinguimos los servicios correspondientes al cálculo de métricas o algoritmos y los servicios que entregan información de los recorridos, como son la información de las líneas de colectivos disponibles, los trayectos que recorre cada línea, el trayecto que debe mostrarle a un usuario cuando seleccionado los puntos de su viaje, etc.

El paquete de casos de uso, trata principalmente de las acciones que el usuario puede desencadenar. Estos son los ejecutores de la lógica empresarial que obtienen datos de la fuente de datos, ya sea remota o local, y se los devuelven al quien los consume, que en nuestro caso sería la capa de la aplicación.

²¹ <https://github.com/InsertKoinIO/koin>

En la capa de datos, se encuentra una definición abstracta de las diferentes fuentes de datos, y la forma en que se debe utilizar. Aquí se suele usar un patrón repositorio que, para una determinada solicitud, es capaz de decidir dónde encontrar la información. Como se puede ver, esta capa está utilizando la capa de dominio, agregada en el archivo de configuración *build.gradle*.

En este módulo, podemos encontrar los paquetes de: Response, Mapper, Implementation y Service. El paquete de Response comprende las clases, que permitirán serializar la respuesta de cada servicio en particular. Estas clases a través de la librería Gson annotations, y la etiqueta *@SerializedName*, podemos mapear la clave de cada campo de la respuesta entregada en el JSON, con el correspondiente campo que tomará en nuestra clase. Una vez capturado y serializado en un objeto legible por la aplicación, los mappers o transformadores del paquete mapper, nos ayudarán a fusionar los objetos de la capa de datos (objetos de tipo Response) en objetos para la capa de dominio (objetos de tipo Entidad). Esta transformación es muy importante a la hora de querer usar los objetos en la capa de aplicación, ya que si observamos el mapa de dependencias, el módulo de aplicación conoce a Domain, pero no conoce el módulo de data.

El paquete de Service, contiene las clases íntimamente relacionadas con el establecimiento de conexión entre la aplicación y los servicios del back-end.

Primero para la inicialización del servicio de comunicación, es necesario inicializar el cliente de OkHttp. En nuestro caso, el constructor nos ayudará a inicializar el interceptor al estilo de un builder, sin la necesidad de que la clase implemente ningún tipo de builder. A su vez, para crear la instancia del objeto Retrofit, definimos la base_url a cual apunta el back-end, y el conversor de Gson para convertir las peticiones a clases.

Con todo esto definido, tenemos a nuestra disposición una extensión genérica, para poder crear la llamada al servicio, definiendo el tipo de Api al que queremos consultar y el tipo de retorno que nos dará, como podemos ver en la figura 4.18.

```
fun <S> createService(serviceClass: Class<S>): S {  
    val retrofit : Retrofit = builder.client(httpClient.build()).build()  
    return retrofit.create(serviceClass)  
}
```

Figura 4.18: Código con la función de extensión para establecer la comunicación con el cliente http

La interfaz de comunicación o Service Api, es la parte neurálgica de Retrofit. Es donde se especifica la estructura de las peticiones que tendrá que coincidir con la de la API. A grandes rasgos se identifica el tipo de la petición con la notación, y luego los parámetros de la petición como argumentos de la función. Como retorno, en Retrofit, necesitamos devolver objetos del tipo Call.

Por último, tenemos el paquete de Implementación, que cuenta con la implementación de los métodos definidos por los contratos o interfaces de servicios. Estas clases usan una instancia del *Service Generator* para establecer el enlace con la api y así consumir los servicios disponibles por el Back-end. Una vez ejecutada la llamada al servicio, controlamos qué tipo de respuesta se obtuvo. El tipo de respuesta así como el resultado que le entregara al caso de uso, fueron implementados a través de clases genéricas, para un mejor uso del paradigma de programación. Permitiendo obtener la respuesta con éxito, entregando el objeto correctamente transformado, o en caso de error, arroja una Excepción, con un dato o mensaje particular.

La capa de aplicación o presentación es en donde ocurre todo lo relacionado a cómo funcionan las vistas normalmente actividades y fragmentos los cuales contienen lógica pero únicamente enfocada en cómo funciona la vista es decir el que debo mostrarle al usuario y cuando debe hacerlo. En general, la capa de presentación es la que usa todos los casos de uso/interactuantes que se crearon en la capa de dominio.

Este módulo contiene en su estructura del directorio los siguientes paquetes: dependencia de inyección (DI), Interfaz de usuario (UI), Utilidades (utils), Adaptadores (adapter).

Para lograr un mejor manejo de vistas en esta capa se suelen usar patrones de UI, en nuestro caso usaremos MVVM (Model - View - ViewModel). La elección principal de este patrón, queda marcada en que es el patrón definido por Google como el estándar para implementar aplicaciones de Android y que ayuda a separar completamente la lógica comercial y de presentación de la interfaz de usuario, y la lógica comercial y la interfaz de usuario se pueden separar claramente para facilitar las pruebas y el mantenimiento. El paquete de UI, consta de fragmentos y viewModel para manejar la interfaz del usuario.

El orquestador de la relación entre los datos y la interfaz de usuario de la aplicación. ViewModel tiene la lógica para convertir lo que proporcionan los casos de uso en

información que la vista puede comprender y presentar. Además, tiene la lógica para reaccionar a la entrada del usuario y llamar a los casos de uso pertinentes.

La parte más útil de la clase ViewModel de Android es su conciencia del ciclo de vida. Solo se comunica con la Vista con componentes LiveData, por lo que es totalmente independiente de los contextos y actividades: puede mantener viva la información incluso frente a cambios de configuración como rotaciones de pantalla o llamadas al fondo. La vista en nuestra implementación de MVVM es en realidad un Fragmento o una Actividad. Las vistas incluyen todo lo necesario para manejar la interfaz de usuario. Observan el ViewModel, utilizando componentes LiveData, y reaccionan a sus cambios cuando lo necesitan.

La vista usa LiveData para observar cambios en ViewModel. Esto tiene varias ventajas:

- La interfaz de usuario coincide con el estado de los datos y esto mantiene los datos actualizados.
- No tener que preocuparse por las actividades detenidas y las pérdidas de memoria. Los objetos de datos en vivo son subíndices de un ciclo de vida y dejan de observar automáticamente cuando termina ese ciclo de vida.
- Maneja los cambios de configuración correctamente.
- Los mismos datos se pueden compartir entre actividades.

Cabe destacar el uso de corrutinas²², un patrón de diseño de simultaneidad muy utilizado en Android para simplificar el código que se ejecuta de forma asíncrona. Las corrutinas ayudan a administrar tareas de larga duración que, de lo contrario, podrían bloquear el subproceso principal y hacer que tu app dejara de responder, como el acceso a la red o la base de datos, en código secuencial. Usamos corrutinas para hacer tareas en un background thread. Esto va muy bien con la idea de casos de uso, acciones únicas que ViewModel llama según sus necesidades. La pauta debe ser que cada tarea ejecutada por un caso de uso debe realizarse en un hilo secundario o background thread, por lo que, en el hilo principal, podríamos mostrar una pantalla de carga o cualquier alternativa, y la IU no se bloquea.

Cada corrutina es lanzada y despachada a un Dispatcher específico para ejecutar sus funciones. Dado que esta corrutina se inicia con viewModel Scope, se ejecuta en el

²² "Coroutines | Kotlin." 27 abril 2021, <https://kotlinlang.org/docs/coroutines-overview.html>.

alcance de ViewModel. Si se destruye el ViewModel porque el usuario se aleja de la pantalla, se cancela automáticamente viewModel Scope, y todas las corrutinas en ejecución también se cancelan.

Consideramos que una función es *segura para el subproceso principal* cuando no bloquea las actualizaciones de la IU en este subproceso.

`withContext(Dispatchers.IO)` traslada la ejecución de la corrutina a un subproceso de E/S, lo que hace que nuestra función de llamada sea segura y habilite la IU según sea necesario.

```
viewModelScope.launch { this: CoroutineScope
    when (val result : UseCaseResult<List<LineBus>> =
        withContext(Dispatchers.IO) { getLinesBusesUseCase.invoke() }) {
        is UseCaseResult.Failure -> {
            mapMutableLiveData.postValue(
                Event(
                    Data(
                        status = Status.ERROR,
                        data = result.exception.message,
                        dataAlternativa = "Back"
                    )
                )
            )
        }
        is UseCaseResult.Success -> {
            mapMutableLiveData.postValue(
                Event(
                    Data(
                        status = Status.LOADING,
                        data = result.data
                    )
                )
            )
        }
    }
}
```

Figura 4.19: ejemplo estructura general del manejo de corrutinas en el código.

Capítulo 5: Interfaz Gráfica

5.1 Introducción

A lo largo de este capítulo se incluye la descripción y visualización específica de cada una de las vistas que componen a la aplicación móvil. Partiendo desde una perspectiva general de la aplicación, se busca entregar una visión simplificada de cómo el usuario puede desplazarse a través de las distintas pantallas y flujos, abocados en detallar las decisiones de diseño que definieron los distintos flujos. Estas decisiones, guiadas por los principios y recomendaciones en los estándares de calidad y navegabilidad en aplicaciones móviles ayudarán a comprender y verificar un correcto y simple uso de la aplicación.

El objetivo de esta sección, no es comparar puntualmente las características de otros sistemas, sino resaltar los beneficios que ofrece la aplicación presentada, destacando las ventajas y desventajas que dispone.

En cada pantalla se introduce cómo se cumplió con cada requerimiento funcional y el correspondiente aporte que se hizo para potenciar los no funcionales. Logrando de esta forma que el usuario haga uso de una aplicación moderna, intuitiva, simple, extensible a múltiples dispositivos, que respeta buenas prácticas de diseño y que resalta la usabilidad para el usuario final, logrando la mayor aceptación posible.

5.2 Estructura general de la Interfaz de Usuario

La aplicación móvil para la visualización de los recorridos y datos del usuario, cuenta con una interfaz nativa centrada en dispositivos Android y testeada en diferentes dispositivos tanto físicos como emulados. La gran ventaja de disponer de diferentes dispositivos para realizar la pruebas pertinentes, es la disponibilidad de diversos componentes de hardware, como son las dimensiones de tamaño de los dispositivos,

memoria de procesamiento, versiones de software, entre otros. Estas cualidades se basan respetando la mayor parte de los fundamentos de aspectos de accesibilidad, tonos de colores, organización y nomenclatura, alineamiento y espaciado, jerarquía visual y tipografía. Estos detalles son muy importantes a tener en cuenta a la hora de definir e implementar el desarrollo en Android por ser el sistema operativo más usado y también por contar con diferentes y grandes compañías de desarrollo de dispositivos.

La aplicación llevada a cabo cuenta con una interfaz dedicada no solo a los usuarios que están próximos a utilizar el servicio de transporte, sino también como una herramienta de información general de recorridos y al pronóstico de tiempos relacionados a colectivos. Es por eso, que la identificación de escenarios, funcionalidad y diseño, apunta a considerar los aspectos más importantes que debería tener todo servicio de información y de transporte público.

Durante la navegación a través de cada vista y la visualización dentro de la aplicación, el usuario tiene la disponibilidad de todos los datos provistos por los servicios y podrá seleccionar y modificar las configuraciones de datos según sus propias preferencias. Estas vistas y el desarrollo de cada flujo impulsan la simplicidad visual en pos de potenciar la usabilidad, siendo integradas por pocos elementos que tienen una función bien definida y evitan que el usuario se vea abrumado por múltiples pasos para realizar una operación.

Inicialmente cuando el usuario entra a la aplicación, la pantalla principal cuenta con la separación inicial en dos secciones, una para la navegación, desplazamiento, selección y visualización sobre un Mapa y otra sección con una pantalla simple de configuración predeterminada de las características generales del funcionamiento del sistema. Al ser la pantalla base dentro de la aplicación, se buscó un diseño moderno que promueva la rápida navegación entre las diferentes páginas o secciones. Especialmente popular en las pantallas de visualización de contenido y de incorporación, este componente actúa como un contenedor base para las siguientes pantallas. Por defecto, la primera página visualizada en pantalla va a ser la perteneciente al primer icono en la barra inferior.

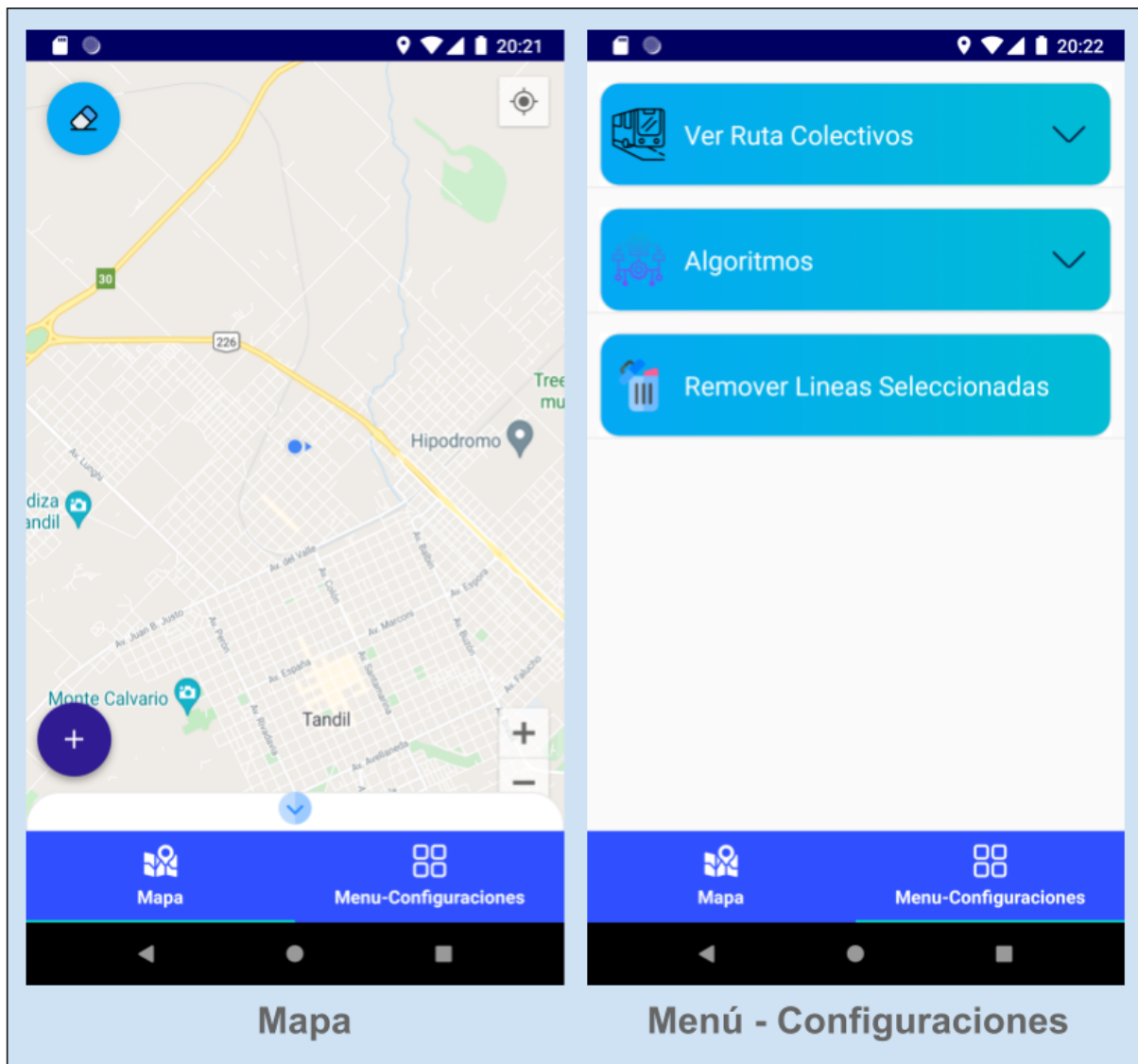


Figura 6.1: Secciones principales en la aplicación.

La primera sección principal, y donde se centra la funcionalidad principal de la aplicación corresponde al mapa con la visualización de la región. Esta es la actividad base, donde el usuario podrá navegar para seleccionar y reproducir las operaciones desde su punto de comienzo, como puede ser la selección de un punto de origen o simplemente visualizar el recorrido y ubicación de las unidades de colectivo, hasta finalizar cada uno de los flujos implementados, como puede ser la elección de que línea de colectivo tomar o ver cuán próxima está la siguiente unidad de colectivo hasta mi destino.

En esta sección, se le brinda al usuario todas las herramientas propias del componente de Google Maps para su libre interacción, como son el icono de “mi” ubicación actual,

iconos de zoom para distinguir calles y zonas, desplazamiento sobre el mapa y redirección a la aplicación de google maps para iniciar un recorrido. Sumadas a estas características propias, se le incorporan las desarrolladas para agregar información que complementen y se ajusten a la problemática de raíz, su adaptación a un sistema de transporte público de colectivos: un botón desplegable con las líneas actuales de colectivos y su simulación de recorridos, interacciones sobre el mapa, con marcadores de información propios, y una tarjeta colapsable, para la selección de puntos o lugares. Cuando el usuario presiona el botón de la izquierda inferior de la pantalla, se despliega un menú de iconos, que presentan la distinción de las líneas disponibles en ese momento. Estas opciones le permitirán al usuario, ver el recorrido base de la línea de colectivos seleccionada, y a su vez, se le presentará una simulación de las unidades disponibles, colocadas en la ubicación en que se encuentren en ese momento.

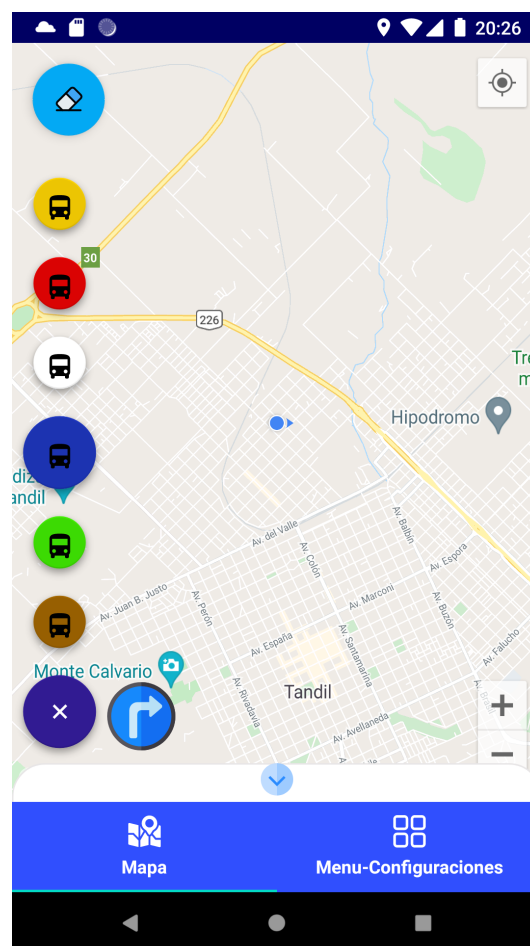


Figura 6.2: Menú desplegable con las líneas de colectivos disponibles por el sistema.

Esta funcionalidad es la más utilizada por todos los usuarios de aplicaciones para consultas sobre el sistema de transporte público, dado que permite encontrar la vía de transporte más cómoda, sea por cercanía o demora, para circular, así también como el tiempo que nos llevará ir de un lugar a otro. La mayor ventaja de este flujo es visualizar la cantidad y disponibilidad de unidades, complementadas luego por su posterior uso en la selección de dos puntos para estimar su tiempo.

Otra de las funcionalidades incorporadas para un mayor beneficio del usuario, es la presentación de una tarjeta o contenido para seleccionar dos puntos o lugares sobre el mapa. Aquí es donde el usuario podrá seleccionar manualmente los puntos del trayecto que desea ubicar sobre el mapa, para luego visualizar el trayecto y el tiempo del recorrido de un punto origen a un punto destino.

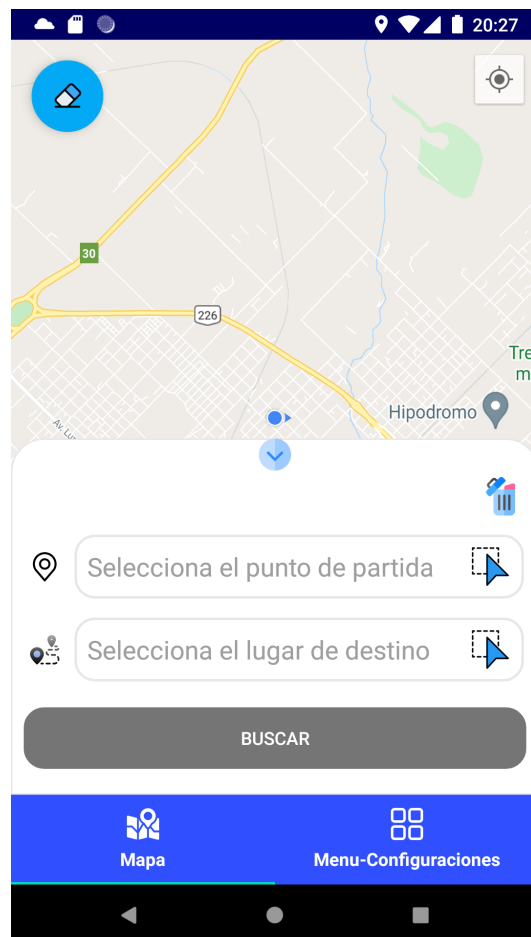


Figura 6.3: Menú desplegable para seleccionar las dos ubicaciones en el mapa.

Con el objetivo de brindarle una mejor experiencia al usuario también se le acerca un atajo sobre el icono de ubicación actual, reduciendo el número de pasos ha realizar

dentro de la aplicación. Sumado a esto, cada punto seleccionado se observará con la dirección completa, con la posibilidad de modificar o borrar cada valor incluido. Al seleccionar un lugar en particular del mapa, el campo elegido se autocompleta directamente con la dirección y la visualización de un marcador sobre el mapa, entregando una información completa al usuario.

Alineados a la idea de que el usuario pueda completar los flujos de la manera más rápida e intuitiva, y complementando la funcionalidad desarrollada para marcar y entregar la información del punto seleccionado, el usuario podrá utilizar ese punto marcado sobre el mapa y usarlo como punto de recorrido. Al presionar sobre el cartel informativo de la ubicación, automáticamente es precargada como punto de destino en la tarjeta de puntos seleccionados. Esto agiliza el proceso de búsqueda dándole un mejor uso a la funcionalidad antes mencionada.

Una vez cargados ambos puntos, se habilita al usuario la opción de Buscar para procesar la búsqueda cargada y arrojar los resultados estimados para el recorrido seleccionado, completando así la función para determinar cuánto tardará en llegar a una ubicación.

Por otro lado, la segunda sección principal cuenta con un menú desplegable, con un menú de opciones para que el usuario configure sus características de preferencia. Cuando el usuario presiona el menú de configuraciones se produce un efecto de desplazamiento lateral de izquierda a derecha de la vista actual, proyectando sobre la pantalla, el listado de opciones permitidas para el funcionamiento de la aplicación. Las dos opciones principales consisten en primer lugar, en un ítem para representar los diferentes trayectos de colectivos y en segundo lugar la distinción de los diferentes algoritmos para el cálculo de operaciones.



Figura 6.4: Vista de la pantalla con el menú de configuraciones y sus opciones desplegables.

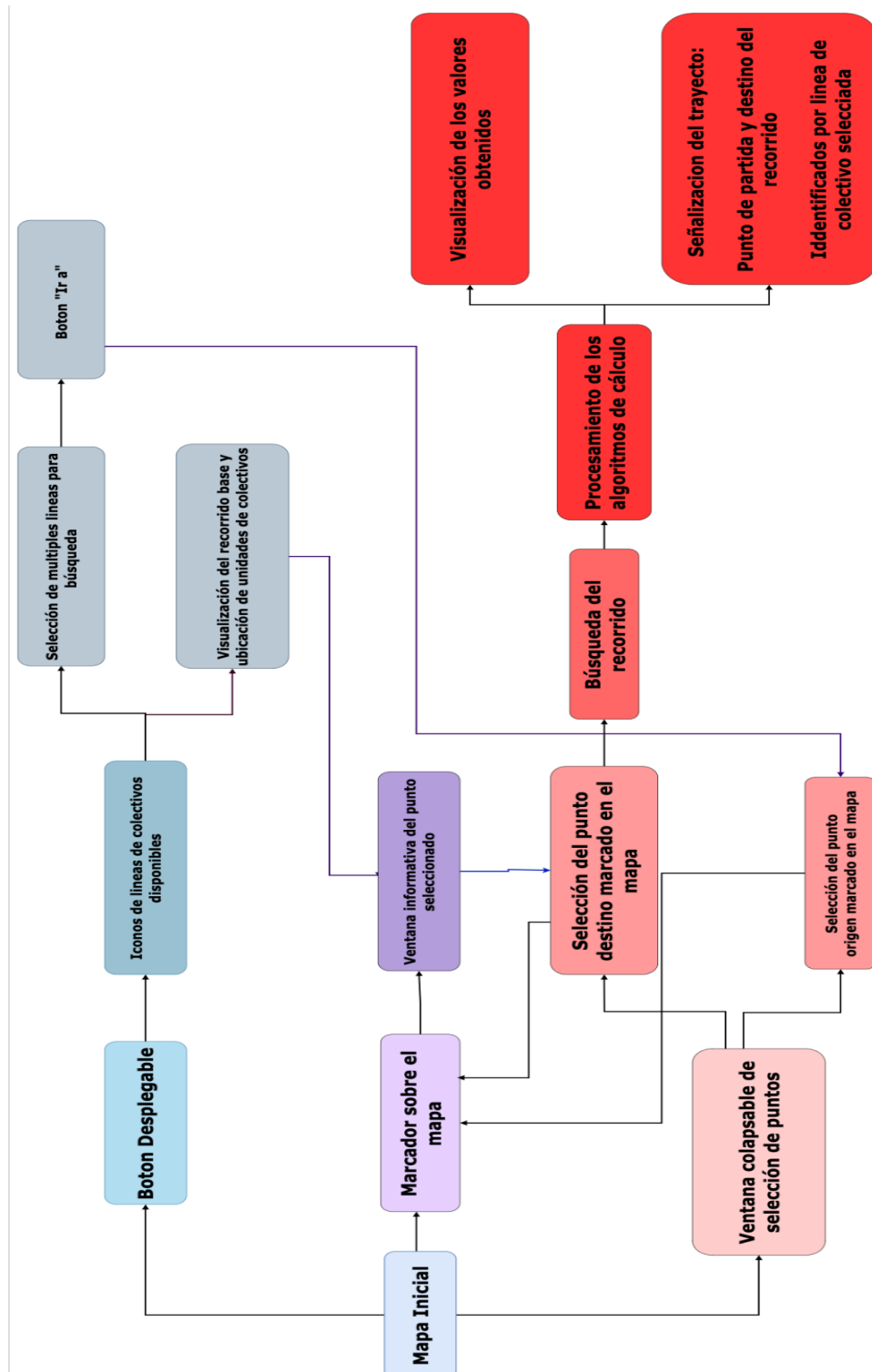
Al seleccionar el icono de Ver Rutas Colectivos, se despliega una lista con el contenido de las diferentes líneas configuradas y disponibles por el sistema. Estas están distinguidas por el número y el color de la línea de colectivo, características principales con que los usuarios diferencian a las mismas. Al seleccionar cada una de ellas, se reproduce automáticamente la navegación hacia el mapa, con la proyección del recorrido de esa línea escogida. Esta característica es una de las más utilizadas en la mayoría de los sistemas de transporte actuales.

A su vez, el icono de Algoritmos, muestra los diferentes algoritmos para el procesamiento y cálculo de métricas en la aplicación, al buscar un recorrido seleccionado. En este caso, se informará al usuario de la selección y configuración para el posterior cálculo.

Por último, se agrega la posibilidad de quitar la selección de las rutas previamente seleccionadas en el icono de Remover Líneas Seleccionadas.

Continuando con esta descripción general del flujo de navegación en la aplicación, se provee un análisis con mayor profundidad de las distintas vistas, su trazabilidad y conexión entre los distintos flujos y una descripción de cómo el usuario puede ejecutar

cada una de las funcionalidades implementadas. En el esquema 6.5 se muestra una versión simplificada de la estructura general de la aplicación y cómo se interconectan los flujos entre sí.



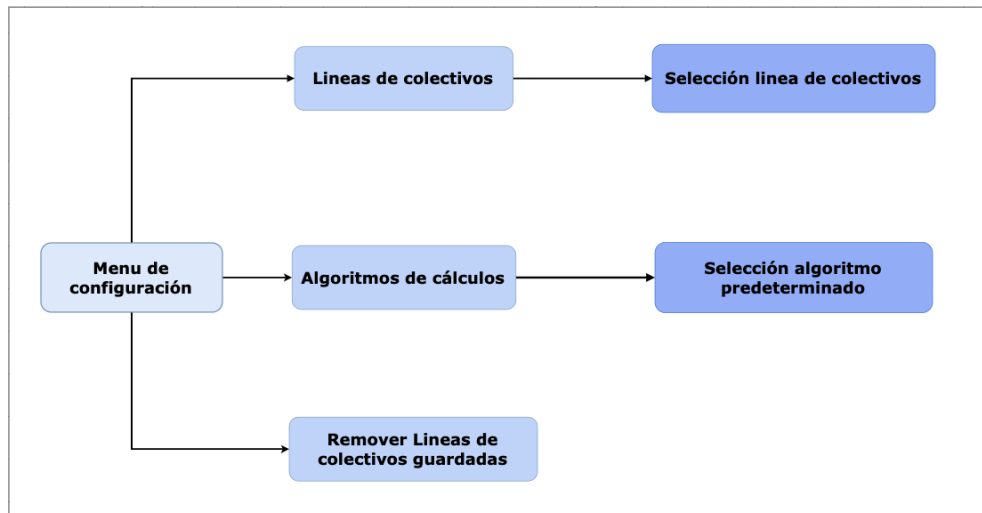


Figura 6.5: Esquema de los flujos dentro de la aplicación.

5.3 Análisis individual de las Vistas

En esta sección se profundiza individualmente cada una de las vistas que conforman la estructura general de la aplicación. Se describe su diseño y funcionamiento, las posibilidades que provee al usuario, las ventajas y desventajas desde la experiencia y el diseño y en donde se podrá establecer una comparación con los diferentes sistemas de transporte público. De esta forma se puede obtener una idea más integral de cada una de las partes que conforman la interfaz gráfica, entregando una vista unificada de los flujos en la aplicación. Asimismo, se adjunta en el Apéndice del informe, las referencias a los videos demostrativos del funcionamiento de las vistas.

5.3.1 Vista mapa principal

La pantalla principal y la funcionalidad más importante de la aplicación se centra en la utilización y el reflejo de las operaciones sobre el mapa de la región. Para brindarle una mejor experiencia, el usuario cuenta con toda la suite de herramientas de mapas para Android, complementado con las operaciones especiales implementadas para nuestra herramienta.

Este componente consta de el SDK de mapas para Android que utilizan datos, mapas y respuestas gestuales de Google Maps. Google Maps funciona tanto para ordenadores

como para dispositivos móviles (teléfonos y tabletas), pero es en los smartphones en los que se obtiene su máximo rendimiento por la facilidad para encontrar un punto en un traslado y la posibilidad de observar la ubicación del propio usuario en tiempo real.

La API de Google Maps ofrece controles de la interfaz incorporados similares a los de la aplicación de Google Maps en un teléfono Android. Cada uno de los controles se encuentran en una posición predeterminada, relativa al borde del mapa, lo cual beneficia a su rápido accionar y los hacen fáciles de encontrar a la vista del usuario. En la herramienta presentada, los componentes integrados para que el usuario pueda interactuar con el mapa, son los siguientes: Controles de zoom, Botón de mi ubicación, Barra de herramientas del mapa. Por último, el mapa responde a una gran variedad de gestos que pueden cambiar el nivel del zoom de la cámara, así como permitirle al usuario desplazarse por el mapa arrastrando el dedo sobre el mismo, inclinar el mapa y realizar movimientos de rotación.

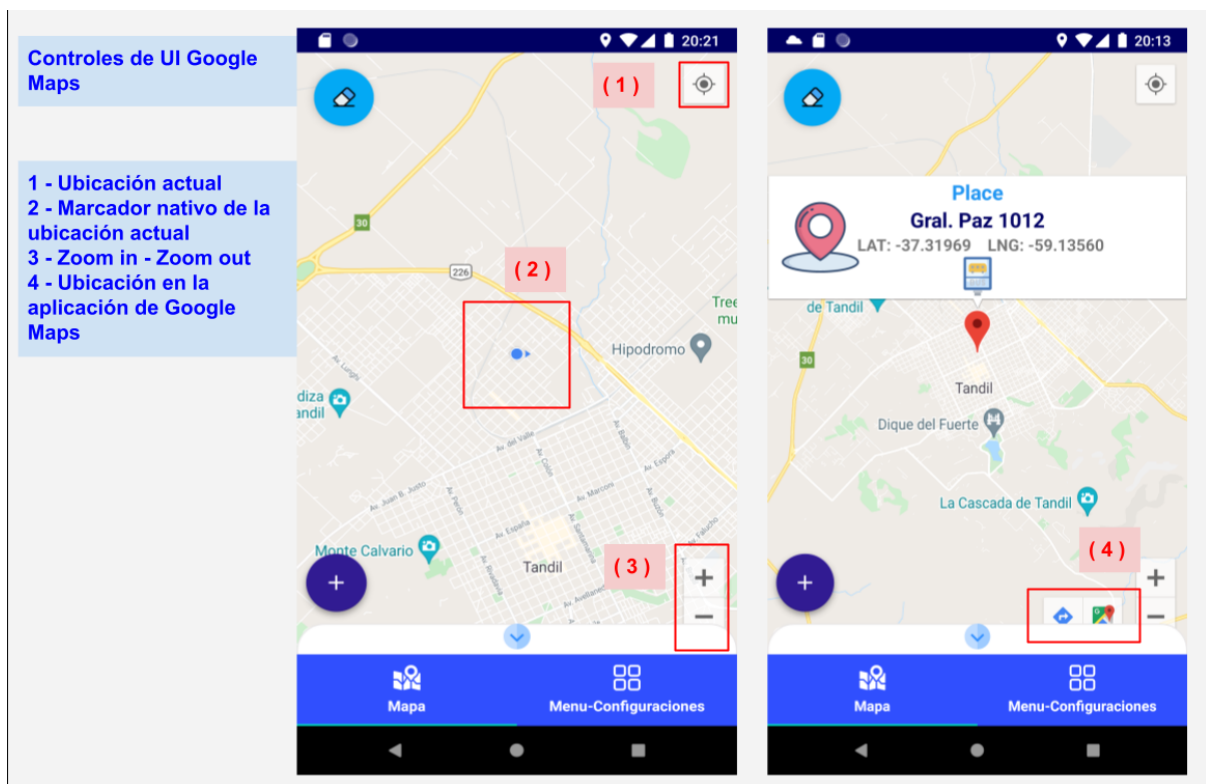


Figura 6.6: Controles nativos de google maps en la interfaz del mapa.

La ubicación de cada uno de estos componentes se encuentran posicionados de manera tal que se prioriza la visualización del mapa en su mayor proporción. Esto permite que

el usuario tenga siempre presente el componente esencial en este flujo, que es el mapa, y pueda descubrir nuevo contenido a medida que realice las diferentes interacciones sobre el mismo.

Estas herramientas permiten entregar al usuario una experiencia mejorada para la navegación sobre mapas en dispositivos móviles, y su integración para el trabajo con sistemas de transporte público. La ventaja principal de redefinir algunas funcionalidades nos permiten adaptar la solución base de este componente a nuestra problemática y entregarle una versión más simplificada al consumidor.

5.3.2 Vista de Selección líneas de colectivos

El usuario una vez dentro de la aplicación, cuando se encuentra en la vista del mapa, al presionar el icono de expandir o ver más, contará con un menú de iconos para elegir cuál línea de colectivo visualizar en la zona y así observar el recorrido de cada unidad.

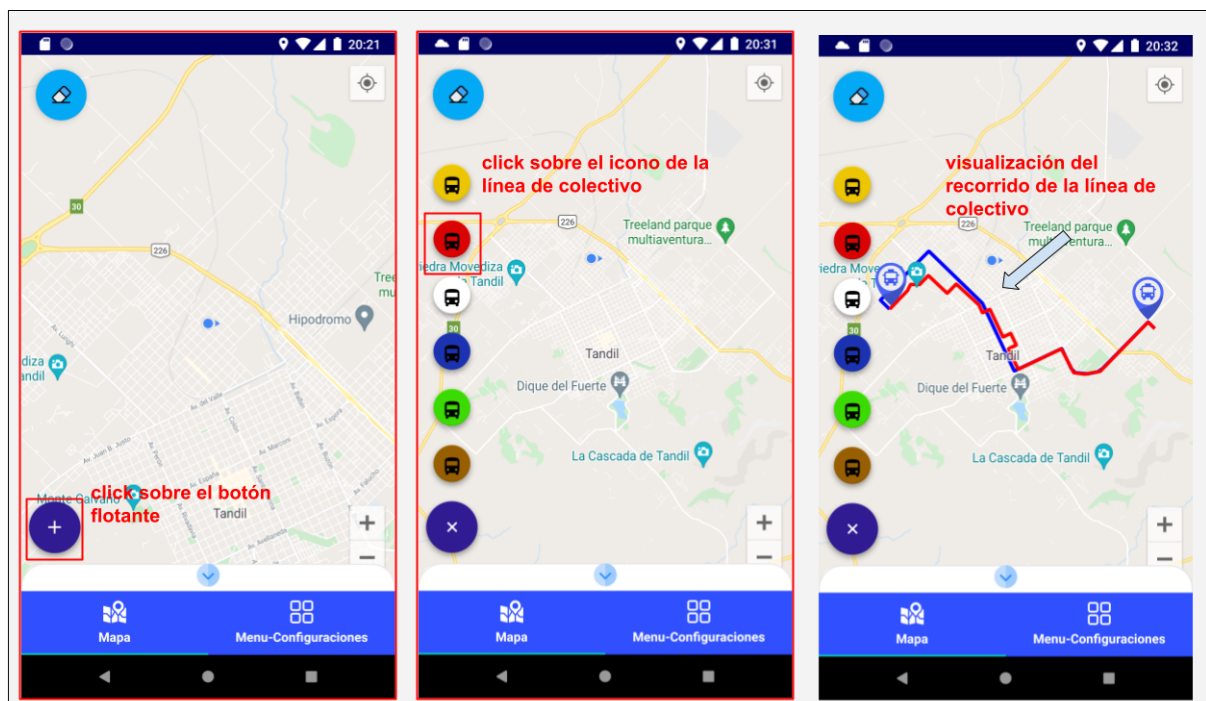


Figura 6.7: Vista con la interacción del botón de líneas de colectivos disponibles.

La vista está compuesta por un botón con el indicador de “+” para referenciar, expandir y ocultar el menú con las diferentes unidades disponibles y así una vez seleccionado una de ellas, reflejar en el mapa el recorrido base de esa línea junto con la identificación de cada unidad disponible en el recorrido actual. La simulación de las unidades funcionales representando el tráfico de colectivos y el comportamiento de esta funcionalidad se puede observar en detalle en el video adjuntado en el informe a modo de demostración, en el apartado de Anexo A2, adjuntado al informe. Este elemento está implementado con un componente de desarrollo que prioriza el estilo de diseño y la simpleza, brindando mejores detalles como los efectos visuales al presionar o darle click. Así mismo cumple con el tamaño estándar definido para seguir las reglas de accesibilidad y usabilidad en aplicaciones móviles. En cuanto a su ubicación, se encuentra alineada a los diferentes componentes que complementan la navegación e interacción sobre el mapa. El símbolo representado en el componente, es el icono predefinido en todas las aplicaciones, este símbolo sugiere que el usuario pueda expandir el contenido oculto y así reflejar una cómoda visualización de más información. La ventaja principal, es la relación entre la cantidad de elementos que se necesitan mostrar y el área disponible de pantalla. Este componente ofrece una reducida carga de contenido y a su vez permite brindarle al usuario la capacidad de elegir que vista quiere visualizar. En relación a otras aplicaciones existentes en la actualidad, la herramienta creada presenta la flexibilidad y comodidad para mostrar dicha funcionalidad, que en contraparte se refleja de forma estática y poco interactiva.

Por último se le ofrece al usuario un atajo visual para incorporar la funcionalidad de dirigirse a un lugar seleccionado, renderizando automáticamente un botón a la derecha del componente colapsable. Para añadir este atributo a la funcionalidad desarrollada, el usuario puede mantener presionado el icono del menú desplegado y así determinar múltiples rutas de selección para realizar la búsqueda.

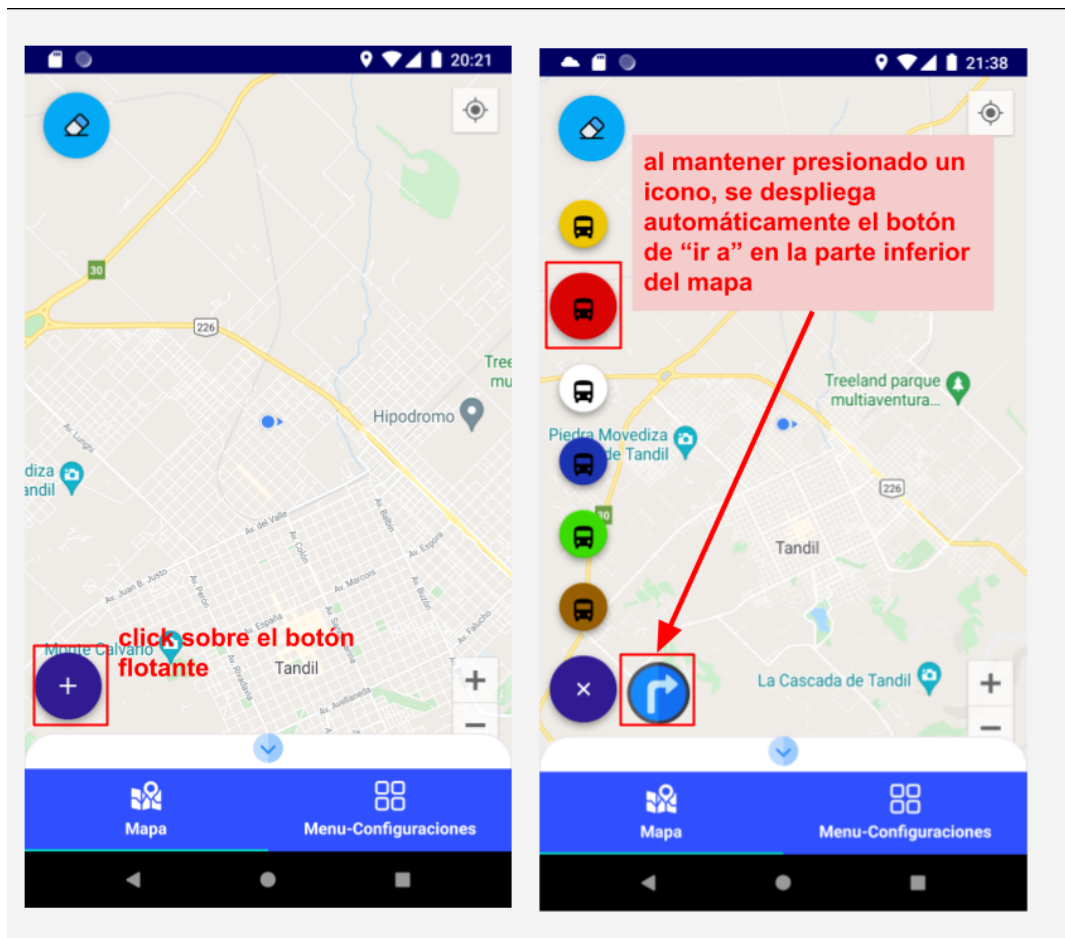


Figura 6.6: Vista con la selección de una línea de colectivo y el despliegue del botón de “ir a”.

Esta funcionalidad se pensó tomando como idea el componente nativo de google maps, y consta de un flujo conector entre la selección de la línea de colectivo que desea marcar, junto con la determinación del lugar al que desea llegar. Cuando el usuario presiona este elemento, será redirigido a la ventana informativa para la selección del punto de comienzo y punto de destino en el mapa.

Una pequeña desventaja a la hora de analizar que tan sencillo es hacer uso de estas funcionalidades adicionales al componente, es la utilización de la pulsación larga o mantener presionado el icono de cada línea para seleccionarlo. A pesar de ser una herramienta que brinda complementos a cada flujo para extender su funcionamiento, esta operación suele ser imperceptible para el usuario, pero es fácil de usar.

5.3.3 Vista del cuadro de selección de lugares

Complementariamente, otra de las utilidades que se le ofrece al usuario, es la posibilidad de seleccionar manualmente un trayecto en el mapa. Esta ventana de información pequeña le indica al usuario tomar la decisión e ingresar información desde el punto de origen al punto de destino ingresando la referencia en el mapa.

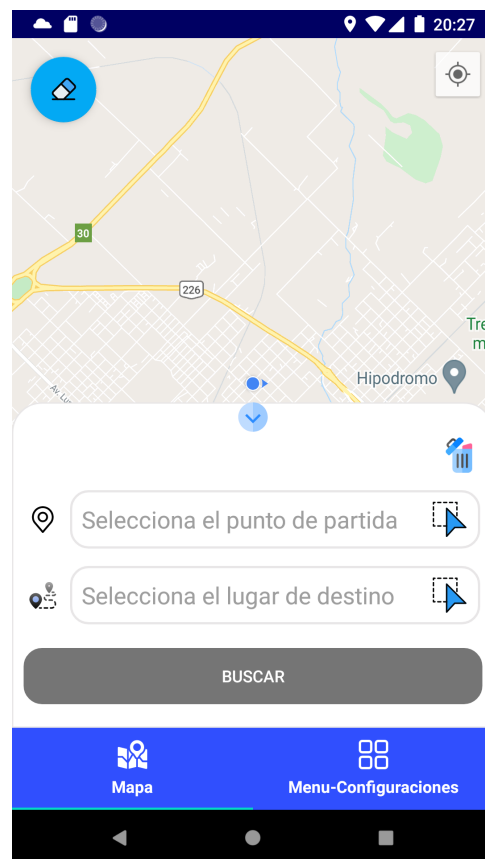


Figura 6.7: Vista cuadro de información inferior desplegable.

La renderización de este componente no ocupa toda la pantalla y, generalmente, se usa para eventos modales que requieren que los usuarios realicen alguna acción para poder visualizarla. Esta función le permite no perder de vista el componente base de la aplicación, el mapa, a la hora de marcar los lugares de selección.

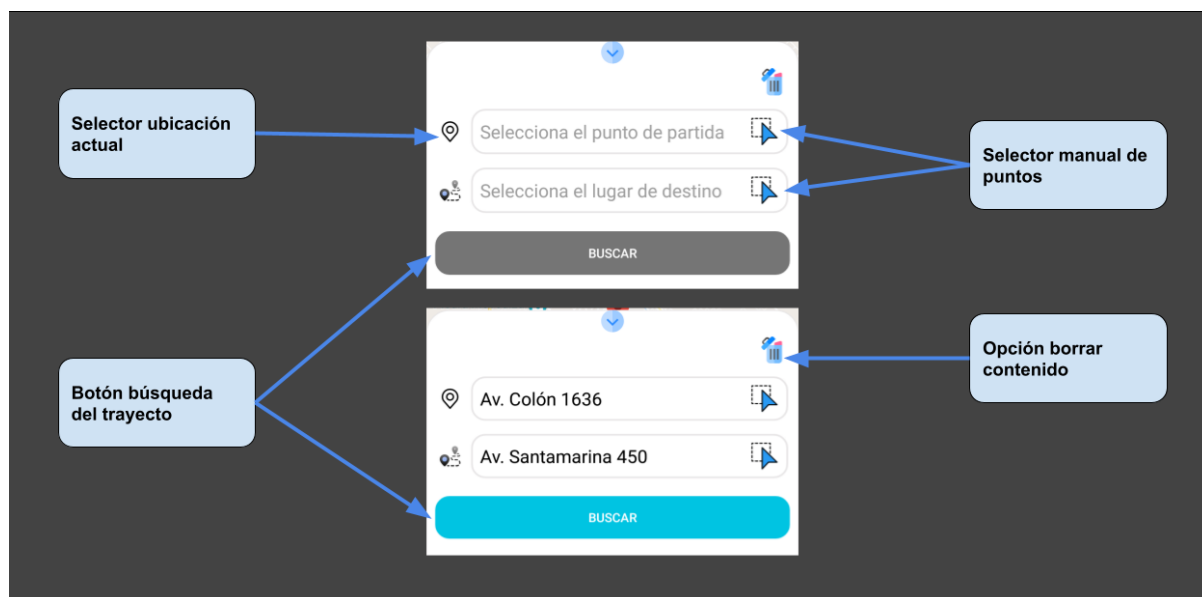


Figura 6.8: Acciones distinguibles en el cuadro de información.

La funcionalidad se centra en que el usuario presione el icono de ubicación correspondiente al campo que desea modificar, sea el casillero perteneciente al punto de origen o destino. Una vez ejecutada la acción, el cuadro de información se oculta permitiéndole al usuario marcar el punto dentro del área del mapa. Este punto se verá reflejado con un marcador, un icono distintivo de ubicación en los mapas, seguida de la carga de la dirección que representa ese punto en el mapa. Este flujo denota la rápida acción que le representa a la persona, trazar un trayecto y encontrar un lugar. Sumado a este principio, este cuadro de información cuenta con un icono especial para auto precargar la ubicación actual del usuario, ubicado a la izquierda del campo de texto que representa el punto de origen.

Una vez completos estos campos, origen y destino, el botón de búsqueda se activa, permitiéndole al usuario proceder a la búsqueda de cuál es el colectivo que más se ajusta al recorrido y agrega las paradas o punto de espera más cercanas a su ubicación.

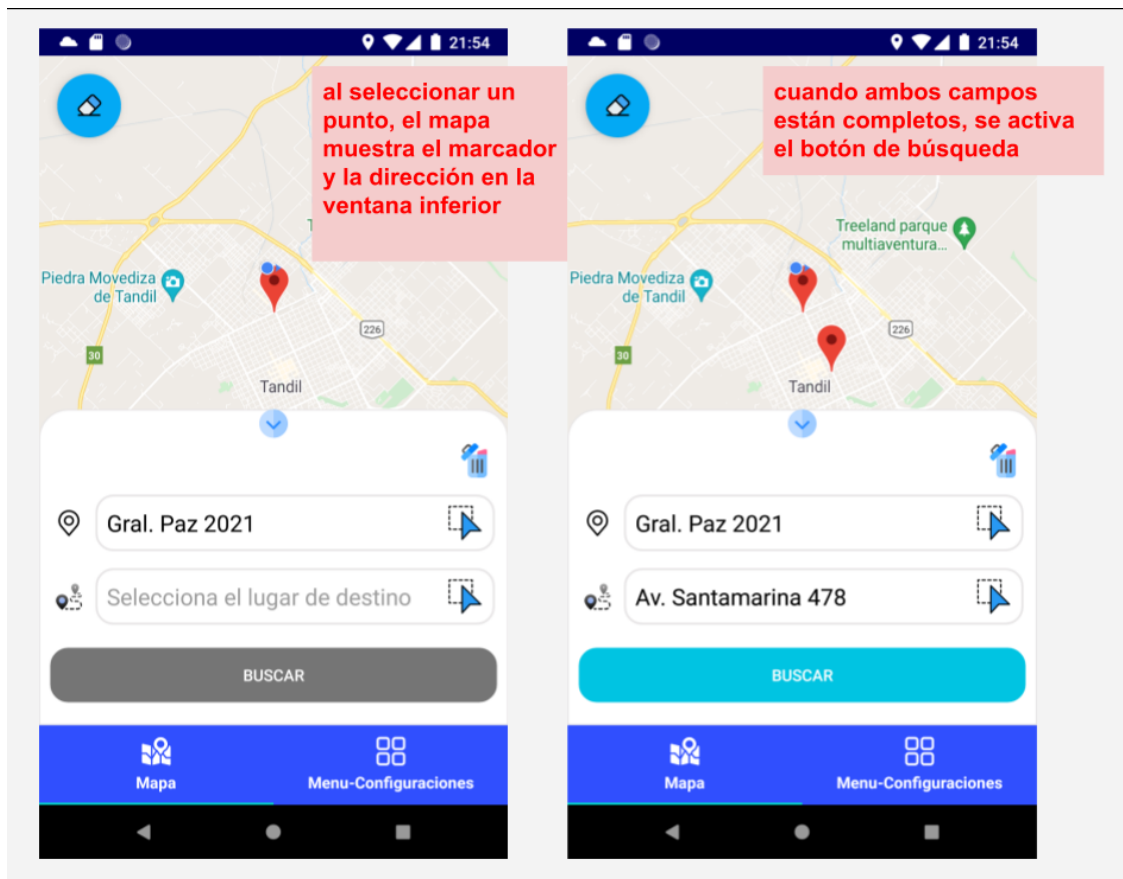


Figura 6.9: Vista con los puntos seleccionados en el mapa y cargados en el cuadro de información.

Este es otro de los beneficios que se le presenta al usuario, en relación a la cantidad de interacciones que requiere realizar el usuario y la cantidad de información que se le brinda. Por otro lado, sigue los estándares de diseño para mostrar el contenido para la búsqueda de lugares, junto con la comodidad de ser un componente deslizable y fácil de usar, al ser reconocido en la gran variedad de aplicaciones móviles. El despliegue y selección de lugares, implica la utilización del mínimo de interacciones por el usuario y una intuición sencilla para su utilización.

Este componente cuenta con la interconexión de distintos puntos mencionados anteriormente, que amplían la funcionalidad de los diferentes flujos de la aplicación. Para detallarlos se realiza una breve descripción a continuación, del inicio y fin de cada uno de ellos:

5.3.4 Selección de un lugar como referencia de punto.

Uno de los puntos más interesantes en la aplicación es poder calcular a qué distancia y cuánto tiempo tardará una unidad en llegar a la ubicación actual de una persona o simplemente estimar cuánto tardará en llegar a la ubicación destino escogida.

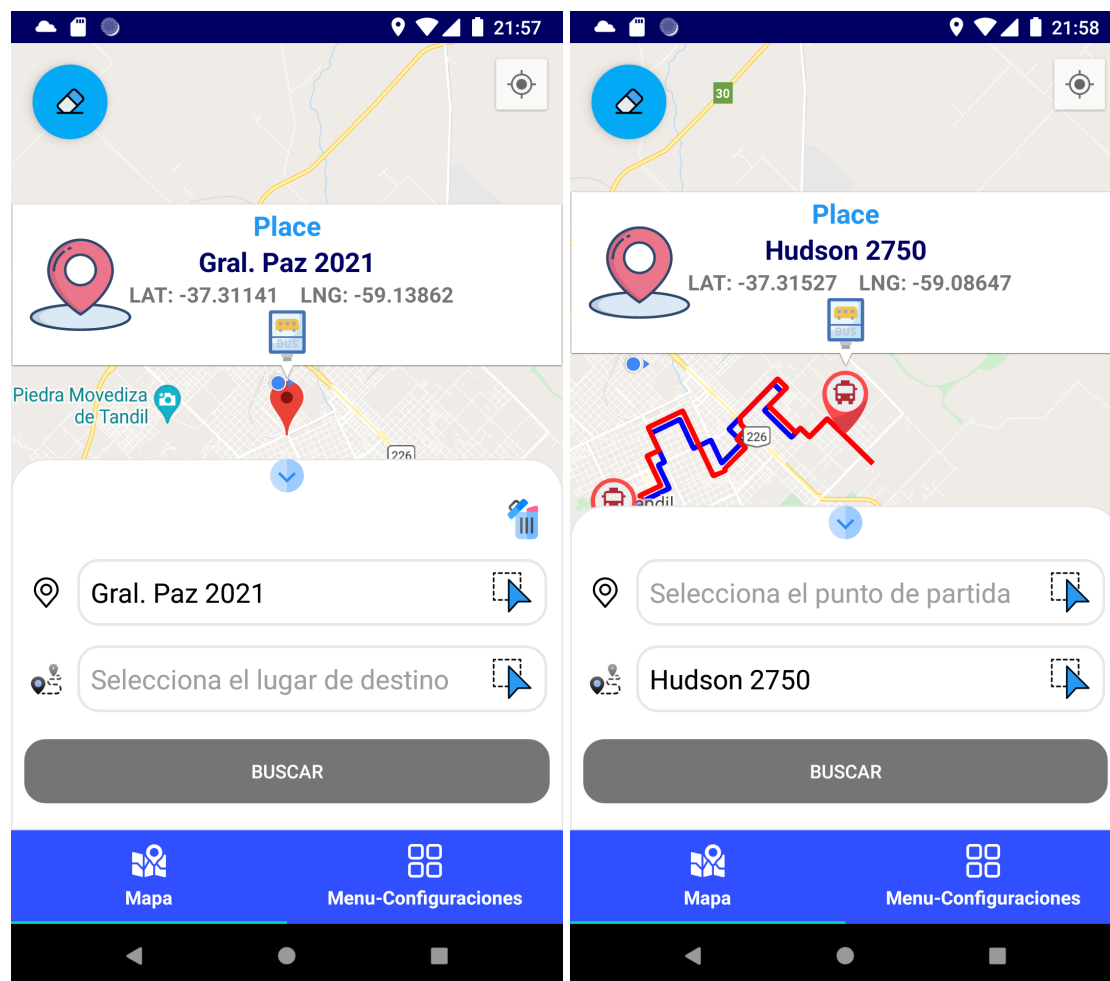


Figura 6.10: Vista de un punto en el mapa y una unidad disponible, con la información cargada en la ventana de información seleccionada.

Para ello el usuario podrá utilizar las ventanas informativas de cada marcador, ya sea un marcador definido por el usuario mismo u otro marcador, como son las unidades de los colectivos simuladas en cada trayecto.

Una vez que el usuario realiza el click en la ventana, se expande automáticamente el cuadro de información inferior, con el campo de selección del punto de destino, modificado con la dirección perteneciente a este punto marcado en el mapa.

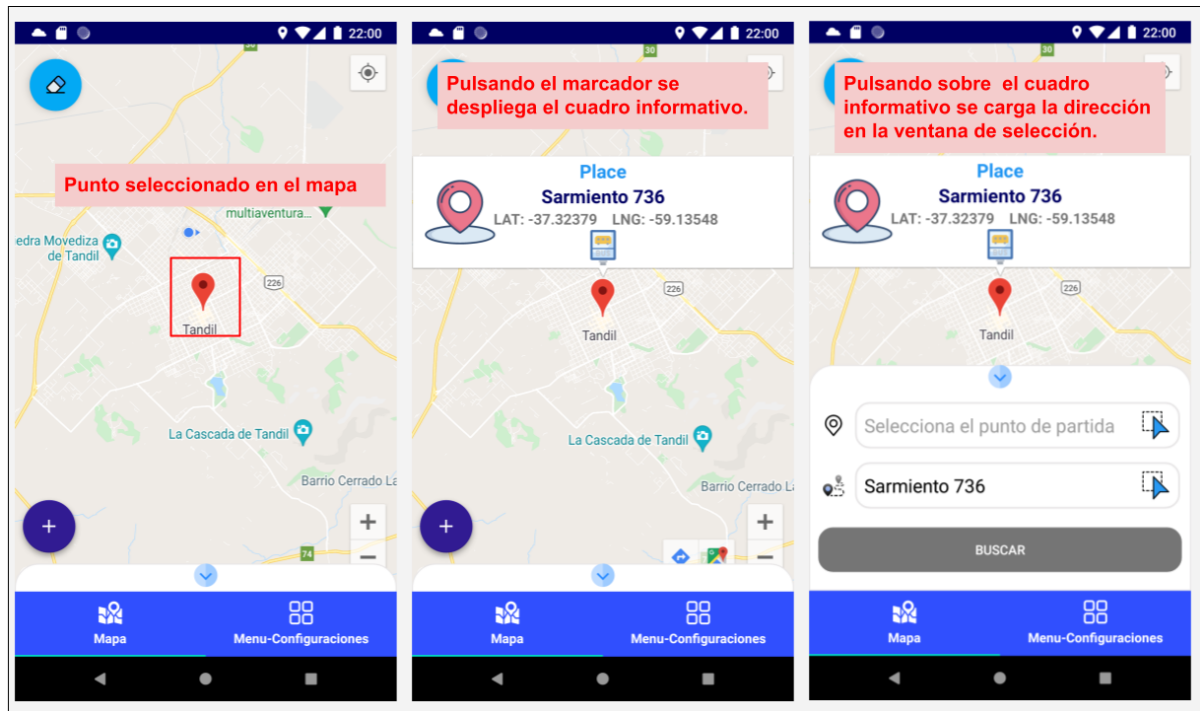


Figura 6.10: Flujo de selección de un punto en el mapa.

Este flujo resalta la automatización en los flujos y está basada en ofrecer un accionar más cómodo e interactivo para el usuario. En lugar de buscar individualmente el lugar donde se encuentra el colectivo y de ahí incluirlo como punto en la ventana de selección de puntos, esta funcionalidad minimiza este escenario con una solución óptima para el usuario.

Dicha funcionalidad es una de las más importantes y que destaca a esta herramienta de las demás aplicaciones del mercado. La estimación de tiempos y distancias es uno de los requerimientos más pretendidos por los consumidores de servicios de transportes y que esta funcionalidad se representa con una vista clara y eficiente.

5.3.5 Selección de una ruta con múltiple selección de líneas de colectivo

Para lograr una mejor navegabilidad y una mejor experiencia para el usuario, cuando requiera realizar una operación rápida y sencilla, al mantener presionado sobre alguno de los iconos de una línea de colectivo se muestra el botón con la opción de “ir a”. Esta opción resuelve el problema que presentan la mayoría de los usuarios, cuando necesitan comparar qué recorrido de transporte tarda menos, cuál línea lo dejará más cerca del punto de destino seleccionado o simplemente estimar cual va a ser la demora del próximo viaje.

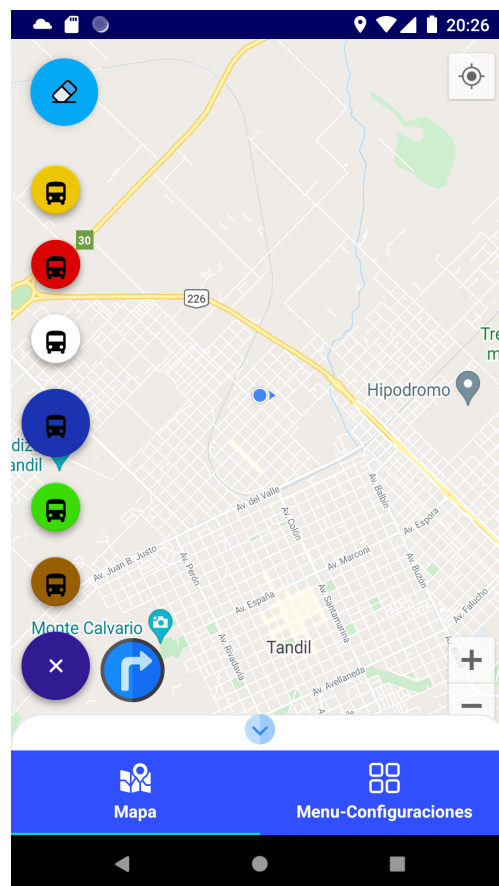


Figura 6.11: Selección individual o múltiple de líneas de colectivos.

Dicho botón, como se mencionó anteriormente, se muestra una vez que el usuario selecciona la o las líneas disponibles y determina el lugar de destino al que quiere llegar.

Una vez presionado el icono de “ir a”, captura la ubicación actual del usuario y la ingresa en el campo de punto de partida para que el usuario pueda ejecutar la búsqueda.

Esta vista permite no solo entregarle al usuario otra forma de búsqueda de lugares, sino la selección simple o múltiple por unidades de colectivos. Por otro lado, le brinda una rápida acción al cargar su ubicación actual, con el fin que el mismo solo necesite ingresar el punto de referencia a dónde quiere dirigirse. Una de las grandes incertidumbres a la hora de usar estos medios de transporte, son las demoras y las distancias que requieren en traslado a distintos puntos de una ciudad o área, que le lleva al usuario la necesidad de elegir la ruta más cómoda. Este requerimiento cumple con las expectativas de dichos consumidores, entregando una forma flexible para determinar la selección de distintas rutas y la automatización de selección de puntos.

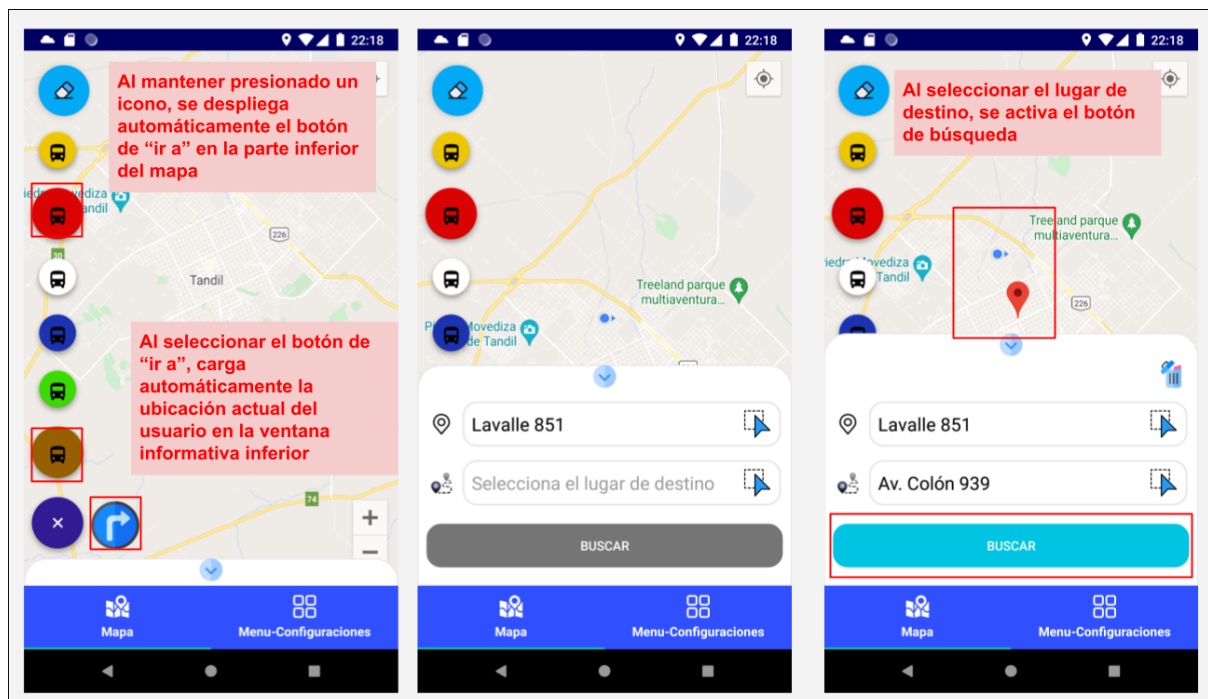


Figura 6.12: Selección individual o múltiple de líneas de colectivos.

Al igual que en las plataformas de mapas, esta funcionalidad es esencial dentro de nuestra aplicación. La interacción en este flujo, promueve que el usuario pueda realizar cualquier búsqueda rápidamente, desplegando de manera automática la ventana de selección de puntos, con su ubicación precargada, minimizando y anticipando las interacciones que realizará el mismo.

5.3.6 Vista del cálculo de tiempos e información de un recorrido

La vista con la información del recorrido, se renderiza una vez procesada la búsqueda con los puntos seleccionados en el mapa y se presenta como un carrusel de tarjetas navegables. Complementariamente, el usuario podrá visualizar dentro del mapa los recorridos involucrados en dicha búsqueda y la distinción de los puntos referentes a las paradas de colectivos más cercanas a los puntos ingresados por ellos. La búsqueda de los puntos seleccionados y el procesamiento del resultado de la información podemos observar en el video demostrativo A2.1.4.1 en el apéndice del informe.



Figura 6.13: Tarjeta con la información completa del recorrido y su trazado sobre el mapa.

Este requerimiento muestra una gran variedad de atributos útiles para los consumidores de los sistemas de transporte y que satisfacen eficientemente el hecho de proveer “información extra al sistema actual”. Dentro de estas tarjetas informativas el usuario cuenta con la asociatividad entre lo que se le muestra en el mapa y el contenido

de cada tarjeta, representada por el color que identifica a cada una de las líneas de colectivo.

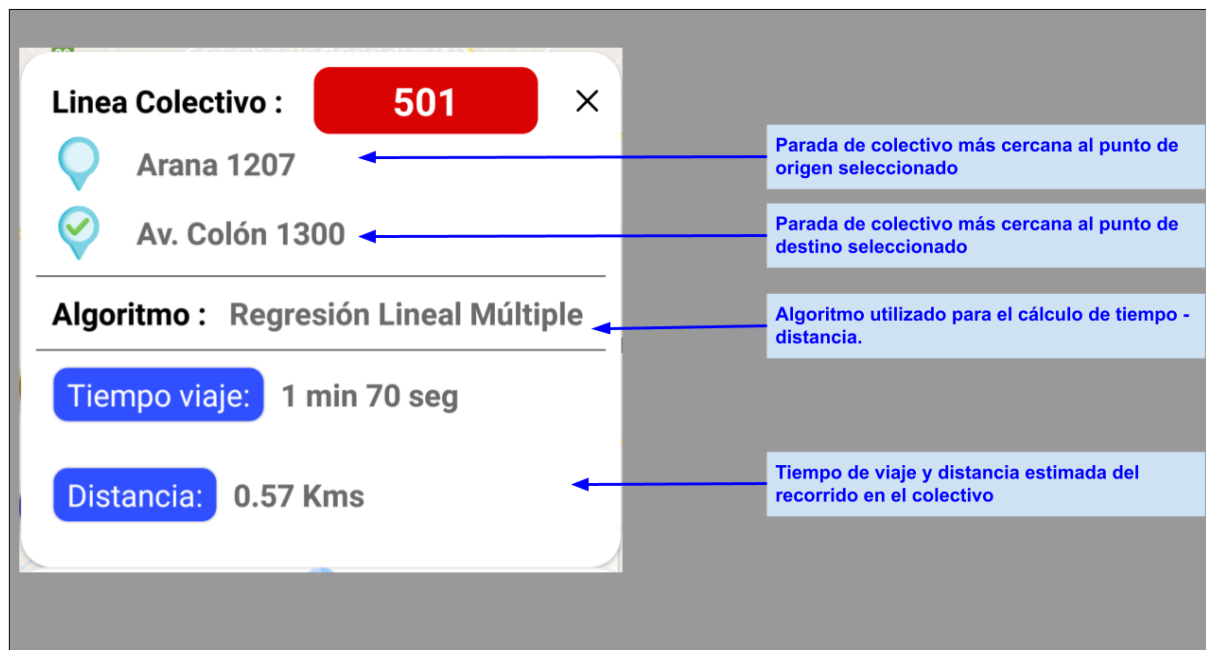


Figura 6.14: Tarjeta informativa para el usuario donde podrá encontrar toda la información del recorrido que realizará distinguido por las líneas de colectivos seleccionadas.

Cada recorrido es trazado sobre el mapa y los puntos de las paradas de colectivo más cercanas marcadas, se especifican con la información completa ubicada en el mapa. Además de estos rasgos, se le entrega al usuario el resultado obtenido por el cálculo del algoritmo ejecutado. Este cálculo entrega el tiempo total y la distancia del recorrido sobre la unidad de colectivo. Toda esta información es fácilmente accesible para el usuario al poder desplazarse por el carrusel de tarjetas informativas y de las cuales podrá comparar con los puntos trazados en el mapa. Esta versatilidad de información visual permite un uso más interactivo y preciso a la hora de que el usuario determine qué línea de colectivo elegir para su desplazamiento.

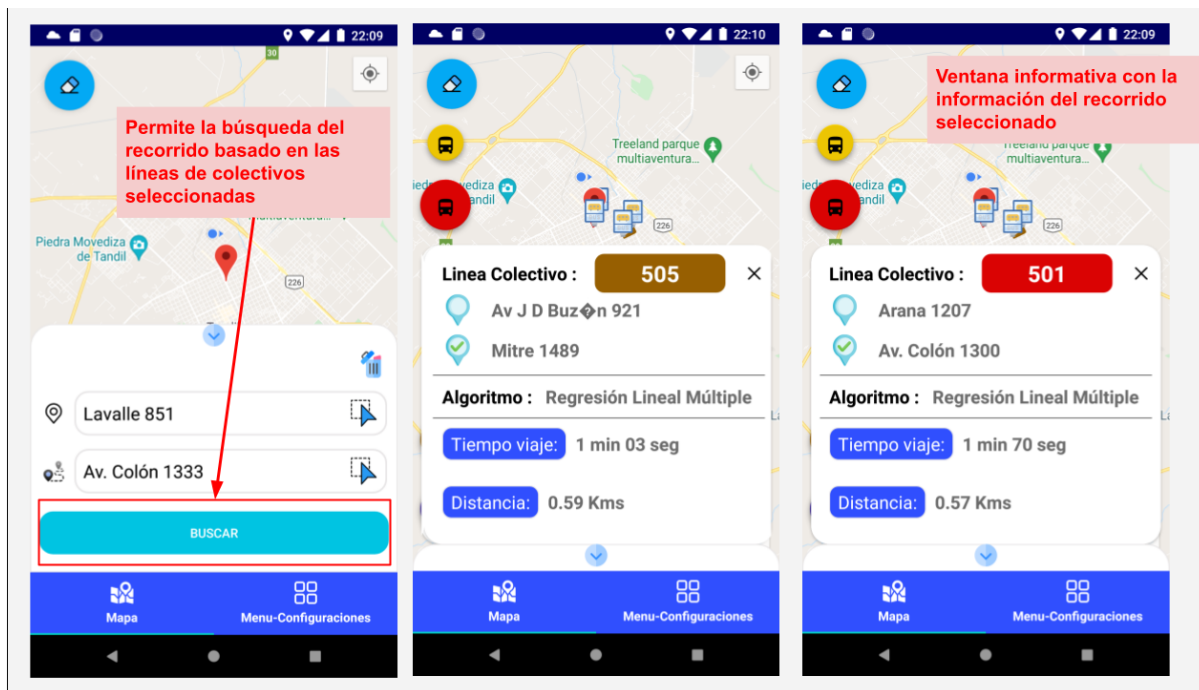


Figura 6.15: Flujo búsqueda de un recorrido sobre la selección de múltiples unidades de colectivos.

Esta vista sirve, entonces, como centro de consulta y pronóstico del trayecto seleccionado, con la información más completa e intuitiva que un usuario requiere a la hora de programar su próximo viaje o determinar cuánto tardará en llegar la próxima unidad. La pantalla presenta un diseño donde prima la simplicidad, la consistencia con el resto de la aplicación, la navegación intuitiva y la interacción del usuario con el dispositivo. Esta vista se distribuye con el objetivo de que el usuario tenga dentro del alcance de la pantalla, el trayecto marcado sobre el mapa y la disponibilidad de los datos específicos de la tarjeta informativa. Esta distribución es muy importante a la hora de analizar la usabilidad de la aplicación. En todo momento se prioriza que los nuevos contenidos descubiertos no dejen de lado el mapa, componente principal de la aplicación. Sumado a esto, la asociación entre iconos y datos, se ve perfectamente aplicada logrando una rápida visualización de los datos.

5.3.7 Vista Menú de configuraciones: Selección líneas de colectivos

Dentro de la pantalla principal encontraremos la sección de apoyo para el usuario en la navegación y configuración de las líneas de colectivos y algoritmos. Como primera opción, el usuario podrá ver el conjunto de líneas disponibles en el sistema, con el objetivo de seleccionar a cada una de ellas dentro del conjunto de líneas involucradas en la búsqueda y cálculos de recorridos.

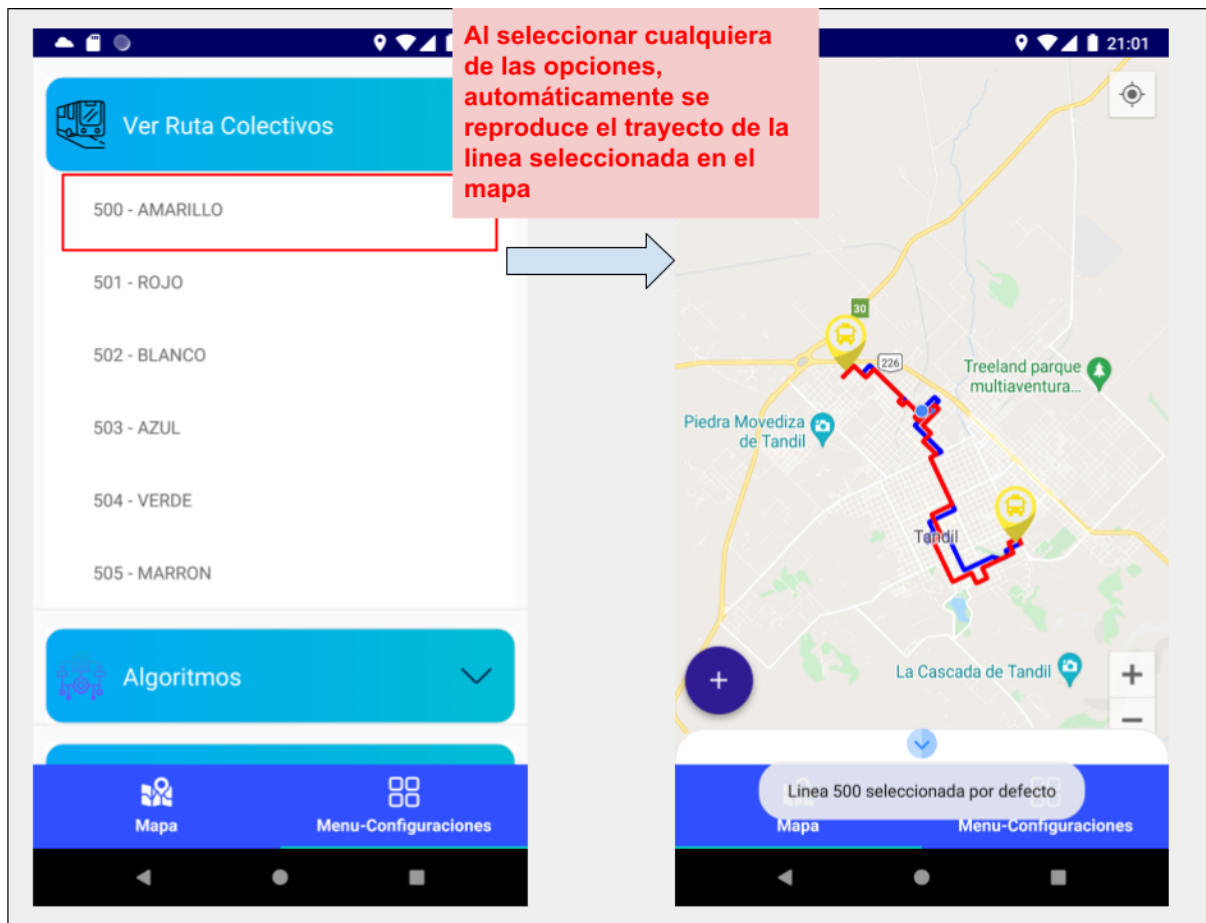


Figura 6.16: Flujo para el guardado de una línea de colectivos para el cálculo de un recorrido.

Cuando el usuario desea elegir una de las líneas disponibles, se conectan las dos vistas principales de la aplicación, logrando una navegación automática y despliegue del recorrido seleccionado. Complementariamente esta opción es guardada por defecto para ser incluida en el cálculo del trayecto que el usuario desee seleccionar. Con esta vista, se logra la unificación y combinación de los flujos para marcar una línea y ser utilizada en el posterior cálculo y la visualización del recorrido base de la línea seleccionada.

5.3.8 Vista Menú de configuraciones: Remover todas las líneas guardadas

Complementariamente al guardado de líneas de colectivos en el sistema, el usuario contará con la posibilidad de remover el contenido guardado, para restablecer los valores de búsqueda.

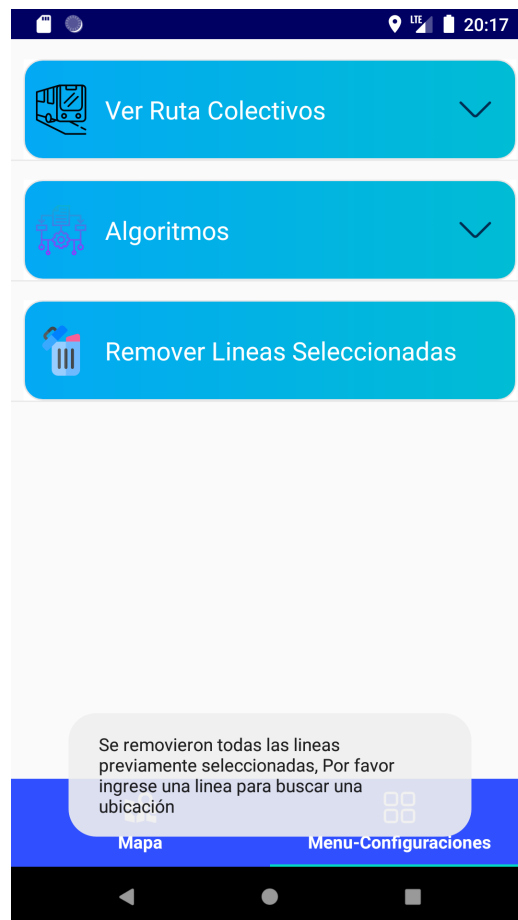


Figura 6.17: Vista con el borrado de las líneas de colectivos guardadas en el sistema.

Con esta opción el usuario puede restablecer y vaciar el contenido de las líneas de colectivos utilizadas en la búsqueda de recorridos. Una vez eliminados estos valores, se le informa al usuario mediante un mensaje visual, que se han removido los valores guardados, advirtiéndolo la necesidad de seleccionar nuevamente alguna de las líneas disponibles para poder efectuar una búsqueda en el sistema.

5.3.9 Vista Menú de configuraciones: Selección de algoritmos.

Dentro de la aplicación uno de los objetivos planteados para el cálculo de las operaciones de búsquedas, fue la disponibilidad de diferentes estrategias para encontrar las métricas y valores de los recorridos que un usuario desee seleccionar. En esta vista, se encuentra a disposición la opción para elegir cual de los algoritmos y técnicas implementados se utilizara para el procesamiento del trayecto marcado en el mapa. Estos algoritmos permiten comparar los diferentes valores de las estimaciones entre los puntos del mapa seleccionado.

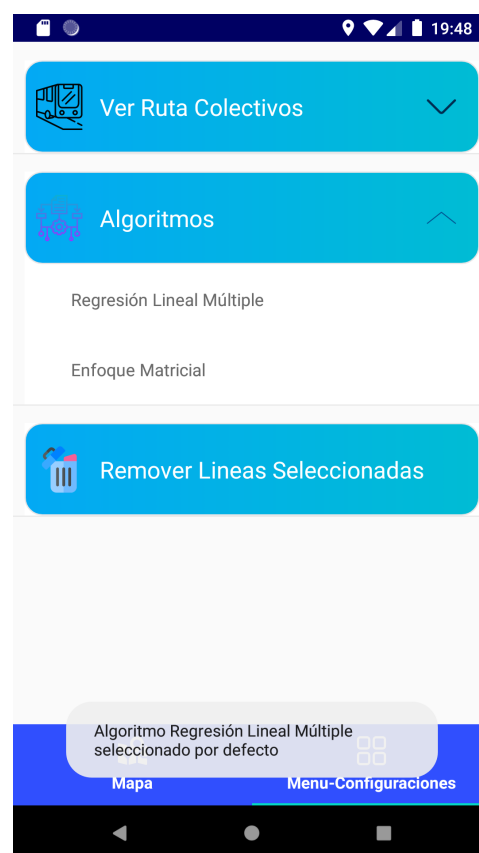


Figura 6.17: Vista selección de algoritmos.

Al optar por cualquiera de los dos algoritmos implementados, se guardará ese valor en el sistema, siendo utilizado cada vez que se realice una consulta sobre los recorridos seleccionados por el usuario. A pesar de tener definido por defecto uno de los valores al abrir la aplicación, esta sección será el punto de partida para realizar el cambio del algoritmo utilizado en cada cálculo de métricas.

Capítulo 6: Conclusiones y trabajos futuros

6.1 Conclusiones

A lo largo del informe se han detallado los aspectos de la herramienta presentada, siendo un complemento para la aplicación de transporte público de colectivos de la ciudad, incorporando características nuevas hoy ausentes en GPS Sumo y en la gran mayoría de sistemas informáticos de este rubro. Dicha aplicación se encuentra dividida en servidor y cliente, siendo independientes y contruidos con arquitecturas que permiten disponer de una estructura mantenible y extensible en el tiempo, logrando que cualquier cambio tanto de funcionalidad como la actualización de librerías sea de manera rápida y sencilla, transversal a toda la aplicación. La elección de una herramienta móvil se encuentra en la disponibilidad y facilidad para su uso durante la circulación de los usuarios. Complementariamente, junto con la creación de una interfaz intuitiva, clara y fácil de usar, similar a las aplicaciones más populares de localización, se obtiene una mejor experiencia en el entorno móvil.

Este sistema tiene como objetivo facilitar el día a día de que aquellas personas que hacen uso, ya sea rutinario o casual, del sistema de transporte público, siendo especialmente pensada para las personas que buscan hacer uso más eficiente de su tiempo y evitar esperas innecesarias o simplemente buscan tener una idea aproximada de cuánto va a demorar un viaje en específico. Como se mencionó previamente, el objetivo no consiste en sustituir ni GPS Sumo ni cualquier sistema existente, sino que se busca contribuir y colaborar con ellos para lograr un producto lo más completo posible.

La aplicación presentada dispone de varias funcionalidades adicionales a las propuestas originalmente. Además de poder retornar el tiempo aproximado que conlleva trasladarse desde un punto a otro de la ciudad haciendo uso de los colectivos, permite visualizar el recorrido de las calles y avenidas por donde circula cada línea. Por otro lado, permite seleccionar tanto puntos aleatorios por fuera de los recorridos base como los iconos de los colectivos simulados en el recorrido para calcular y visualizar cuál de

estos recorridos le conviene más a los usuarios, indicando la parada donde subirse y donde descender y los tiempos y distancias de cada trayecto seleccionado.

A partir del análisis de los tres enfoques planteados, el primero con regresión lineal simple, el segundo con la matriz de puntos y el tercero con regresión lineal múltiple, se puede deducir que la problemática puede ser abordada desde múltiples perspectivas donde cada uno presenta sus ventajas y desventajas. El primer caso destaca por su sencillez de cálculo luego de realizar las operaciones sobre el dataset pero cómo se analizó en las métricas, los resultados no fueron los esperados. En el segundo y tercer caso de pruebas presentado, la complejidad aumenta pero los resultados se ajustan más a la realidad, además en los cálculos se consideran nuevas variables previamente no usadas, cómo el sector de la ciudad o el horario a tener en cuenta.

En conclusión la aplicación desarrollada cumple con los objetivos propuestos inicialmente, brindando al usuario una aplicación móvil sencilla e intuitiva de utilizar la que hace uso de algoritmos de cálculos previamente explicados. A lo largo de este trabajo se detalla cómo fue el proceso de trabajo en todas sus etapas y cómo se pudo aplicar y profundizar en diversos conceptos estudiados a lo largo de la carrera logrando un resultado positivo.

6.2 Trabajos Futuros

Para poder determinar de qué manera la herramienta puede mejorar o detectar aquellos aspectos donde no es óptimo, es importante realizar pruebas durante el transcurso y desarrollo del servicio real de colectivos, que permitan una captura del nivel de aceptación de los usuarios y del descubrimiento de nuevas necesidades.

Para poder estimar el rendimiento de la aplicación es importante disponer de un ambiente con cierta cantidad de usuarios y poder obtener así estadísticas y evaluar el funcionamiento presentado. Es recomendable evaluar los tiempos obtenidos con diferentes líneas y recorridos e inclusive en diferentes franjas horarias. También es importante realizar la carga constante de información histórica del transporte en el formato específico para que los algoritmos utilizados puedan tener en cuenta valores nuevos.

Otras funcionalidades que no se tuvieron en cuenta dentro del alcance del proyecto y que podrían llevarse a cabo para enriquecer aún más la herramienta, se destacan:

- Dar la posibilidad de crear cuenta para poder almacenar información específica.
- Añadir gráficos y métricas más específicas donde se muestran los resultados obtenidos.
- Mejorar las opciones de configuración para el usuario.
- Incluir información en tiempo real de los estados de los recorridos.
- Poder guardar recorridos utilizados regularmente.
- Dar soporte en dispositivos móviles iOS u otro sistema operativo.
- Implementar sistema de alarmas y notificaciones.
- Implementar testeo de comportamiento en la aplicación.
- Incluir un complemento que vincule la herramienta presentada, con la aplicación que se encuentra hoy en día en uso (GPSumo).
- Ofrecer un asistente que ayude a personas con grandes limitaciones para navegar dentro de la aplicación.
- Utilizar diferentes técnicas para la estimación y cálculo de resultados.

Apéndice 1: Tecnologías empleadas

Tecnologías Empleadas

A continuación, se describe cómo se diseñó e implementó la arquitectura de la herramienta, comenzando con una explicación de la estructura desde el enfoque visual, siendo la perspectiva del usuario y posteriormente de manera similar desde el lado de backend. Para cada una, se proporciona una breve descripción de cuáles son sus características más importantes y cuál es el objetivo de su uso.

1.1 Tecnología y bibliotecas utilizadas en el cliente

1.1.1 Kotlin

Kotlin es un lenguaje de programación de tipado estático que corre sobre la máquina virtual de Java y que también puede ser compilado a código fuente de JavaScript. En la Google IO 2017, recibe un importante impulso al ser nombrado por Google como lenguaje oficial para Android al mismo nivel que Java.

Con Kotlin para el desarrollo de Android, se beneficia del soporte de Android Jetpack y diversas librerías. Las extensiones de Android KTX incorporan características particulares como corrutinas, funciones de extensión, lambdas y parámetros con nombre, a las bibliotecas de Android existentes. Dispone de una gran comunidad y según Google, más del 60% de las 1000 aplicaciones principales en Play Store usan dicho lenguaje. Complementariamente provee soporte para el desarrollo no solamente para el entorno Android, sino que también para iOS y aplicaciones web.

Desde su creación en 2011, Kotlin ha mejorado continuamente, no solo como un lenguaje sino como un ecosistema completo con herramientas robustas. En la actualidad

se encuentra perfectamente integrado en Android Studio y muchas empresas lo utilizan activamente para desarrollar aplicaciones de Android.

1.1.2 Postman

Postman es una plataforma de colaboración para el desarrollo de API. Comenzó como un cliente REST y se ha convertido en la actual plataforma API Postman, que principalmente permite crear peticiones sobre APIs de una forma muy sencilla y poder, de esta manera, probar las APIs. Basado en una extensión de Google Chrome, un usuario de Postman puede ser un desarrollador que esté comprobando el funcionamiento de una API para desarrollar sobre ella o un operador el cual esté realizando tareas de monitorización sobre un API. Las funciones de Postman simplifican cada paso de la creación de una API y agilizan la colaboración para que pueda crear mejores API, más rápido.

En esta herramienta se crea y envía peticiones Http a servicios REST mediante una interfaz gráfica, las cuales pueden ser guardadas y utilizadas en el futuro. Permite definir colecciones en donde se agrupan múltiples pedidos y también posibilita la generación de la documentación pertinente de los endpoint que dispone una API Rest. Provee componentes para la creación de variables locales y globales que son utilizadas dentro de las distintas invocaciones o pruebas.

1.1.3 Retrofit

Retrofit es un cliente HTTP de tipo seguro para Android y Java. Retrofit es una herramienta que permite conectarnos a un servicio web REST traduciendo la API a interfaces Java.

Esta librería facilita el consumo de datos JSON o XML, que después son analizados en Objetos Java (Plain Old Java Objects, POJOs). Como la mayoría del software de código abierto, fue construido encima de algunas otras librerías y herramientas. Por un lado hace uso de OkHttp (del mismo desarrollador) para manejar peticiones de red. Retrofit no tiene un convertidor JSON integrado para convertir de objetos JSON a Java. En su lugar, cuenta con soporte para las librerías Gson - Jackson - Moshi para realizar esta

tarea. Cada método debe tener una anotación HTTP que proporcione el método de solicitud y la URL relativa. Hay ocho anotaciones integradas: HTTP, GET, POST, PUT, PATCH, DELETE, OPTIONS y HEAD. La URL relativa del recurso se especifica en la anotación.

Dentro de las funciones principales de esta librería podemos destacar el manejo de conexiones a servicio, cachear respuestas, reintentar peticiones fallidas, ilación, análisis de respuesta, manejo de errores, y más. Retrofit, por otro lado, es una librería bien planeada, documentada y probada que te ahorrará mucho tiempo valioso y dolores de cabeza.

1.1.4 Plataforma de Mapas (Google Maps Platform). SDK de Mapas para Android

Google Maps proporciona un servicio de cartografía que podremos utilizar en nuestras aplicaciones Android, para agregar mapas basados en datos de Google Maps a una aplicación. Esta API, forma parte de Google Play Services, un conjunto de librerías de Google que brindan a los desarrolladores las funcionalidades de las aplicaciones de Google como Gmail, Analytics, Google Fit, Google Maps, etc.

Este SDK maneja automáticamente el acceso a los servidores de Google Maps, la descarga de datos, la visualización de mapas y la respuesta a los gestos del mapa. Dentro de las interacciones disponibles para el usuario, esta herramienta facilita las llamadas a la API para agregar marcadores, polígonos y superposiciones a un mapa básico y para cambiar la vista del usuario de un área de mapa en particular. Estos objetos proporcionan información adicional para las ubicaciones del mapa y permiten la interacción del usuario con el mapa. La API le permite agregar los siguientes tipos de gráficos a un mapa:

- Iconos anclados a posiciones específicas en el mapa (Marcadores).
- Conjuntos de segmentos de línea (Polilíneas).
- Segmentos cerrados (Polígonos).
- Gráficos de mapa de bits anclados a posiciones específicas en el mapa (Superposiciones de suelo).

- Conjuntos de imágenes que se muestran en la parte superior de los mosaicos del mapa base (Tile Overlays).

El SDK de Maps para Android incluye compatibilidad integrada para la accesibilidad. Cuando los usuarios habilitan la función de accesibilidad de TalkBack en sus dispositivos móviles, cada deslizamiento por la pantalla mueve el foco de un elemento de la interfaz de usuario al siguiente. (Una alternativa al deslizamiento simple es explorar los elementos de la interfaz de usuario arrastrando un dedo sobre la interfaz). Cuando un elemento de la interfaz de usuario se enfoca, TalkBack lee el nombre del elemento. Si el usuario toca dos veces en cualquier lugar de la pantalla, se realiza la acción enfocada.

1.1.5 Material Design, librería para Android

Material es un sistema adaptable de pautas, componentes y herramientas que respaldan las mejores prácticas de diseño de interfaces de usuario. Respaldado por código de fuente abierta, Material agiliza la colaboración entre diseñadores y desarrolladores, y ayuda a los equipos a construir rápidamente componentes con una amplia variedad de funcionalidades predefinidas y con un diseño moderno e innovador.

Siendo utilizado principalmente en este proyecto, esta herramienta ofrece soporte para los principales sistemas de front-end, tanto Web, Android y iOS.

Android ofrece las siguientes funciones para facilitar la creación de aplicaciones de material design:

- Un tema de app de material design para diseñar todos los widgets para la interfaz de usuario.
- Widgets para vistas complejas, como listas y tarjetas
- Nuevas API para sombras y animaciones personalizadas

1.2 Tecnología y bibliotecas utilizadas en el servidor

1.2.1 .Net 5

Generalmente en trabajos de escala similar, se suele utilizar un framework para facilitar el desarrollo. Un framework consiste en un conjunto de clases y módulos, que puede servir de base para la organización y desarrollo de software. Para la herramienta presentada se optó por la utilización de .NET 5, que consiste en un framework gratuito y de código abierto utilizado principalmente para el desarrollo web, API para la nube, IoT, microservicios, interfaz de usuario de cliente y aprendizaje automático. Se trata de un framework open source, lo que significa que está disponible de forma gratuita y la comunidad es alentada para contribuir con detección y corrección de errores y complemento de nuevas características. Entre las ventajas se destacan una seguridad más estricta debido a un menor intercambio de información y rendimiento mejorado, ya que está formado por paquetes NuGet, lo que permite una modularidad total, de esta forma solo se añaden los paquetes con funcionalidad que se requieran. También provee soporte para la inyección de dependencias, lo cual es muy importante para asegurar la extensibilidad. Permite crear y ejecutar aplicaciones multiplataforma en Windows, Mac y Linux.

Este framework, se ejecuta sobre el entorno de ejecución .NET de Microsoft, similar a la Máquina Virtual de Java (JVM) o el intérprete de Ruby, con el que es posible crear aplicaciones en un diferentes lenguajes (C#, Visual Basic y F#).

1.2.2 SQLite

SQLite consiste en una librería para gestionar bases de datos de tipo relacional, utilizadas bajo esquemas de tamaños reducidos, por lo que la hace ideal para aplicaciones móviles o web donde el consumo de datos es bajo. Se utiliza en forma de base de datos embebida, por lo que los datos se encuentran almacenados en un archivo cuya extensión es '.db'. Por lo que cualquier consulta consiste en una solicitud al archivo directamente, lo que significa una velocidad mayor. Esto difiere de otros sistemas de

base de datos, donde se manejan como procesos independientes con su propio servidor. SQLite es una librería compacta y muy liviana por lo que su utilización no impacta fuertemente en el tamaño final de la solución. Adicionalmente, otra ventaja que proporciona es su sencilla configuración. Esto se debe a que no es requerido realizar la conexión a una dirección mediante autenticación con usuario y contraseña o instalación del servidor. Tanto el proceso de creación del esquema, con tablas y columnas pertinentes, y las operaciones básicas de inserción, borrado y la ejecución de consultas pueden realizarse de manera simple sin ningún problema a través del código de back-end. Complementariamente se obtiene un mejor rendimiento si comparamos con un sistema de archivos ya que solo carga los datos que se necesitan, en lugar de leer todo el archivo y mantenerlo en la memoria, así como también si se edita una parte pequeña, solo sobrescribirá las partes del archivo que se modificó y no en su totalidad.

1.2.3 LINQ

Language Integrated Query (LINQ) consiste en un conjunto de extensiones que se encuentran integradas al lenguaje de programación C# utilizado para desarrollar la funcionalidad de back-end. Dicha extensión nos posibilita trabajar de manera rápida y cómoda con colecciones de datos, como por ejemplo listas, como si se tratase de una base de datos. Es decir, podemos llevar a cabo inserciones, selecciones y borrados, así como operaciones sobre sus elementos de cualquier colección, independientemente del tipo de datos que disponga. Todas estas operaciones se pueden realizar de manera sencilla gracias a los métodos de extensión para colecciones que nos ofrece Linq y a las expresiones lambda. Entre las principales ventajas se destaca la realización de un código más compacto y legible para que otros desarrolladores puedan entenderlo y mantenerlo fácilmente. Además brinda una forma estandarizada de consultar múltiples fuentes de datos ya que la misma sintaxis LINQ se puede utilizar para consultar múltiples fuentes de datos.

1.2.4 Docker

Docker consiste en una tecnología que nos permite crear una imagen que describa todo lo que su aplicación necesita para ejecutar sobre un contenedor en cualquier Docker. En otras palabras, permite una independencia de plataforma, lo que

significa que si la aplicación funciona en docker, también funcionará de igual manera en cualquier otra plataforma que tenga soporte de docker. Esto nos facilita centrarse en desarrollar su código sin preocuparse de si dicho código funcionará en la máquina en la que se ejecutará, automatizando la parte de deployment. Inicialmente, se debe realizar la construcción de la imagen, en donde se proporciona información específica cómo puertos a utilizar, entre otras variables. Dichas configuraciones se almacenan en un archivo de texto con el nombre de Dockerfile (sin extensión), el cual dispone de una sintaxis en particular. Al momento de realizar la construcción de la imagen, se utiliza como base la configuración detallada en el archivo presente en el directorio donde se ejecuta el comando específico.

1.2.5 Entity Framework Core

Se trata de una tecnología de acceso a datos para .NET. Consiste en un mapeador objeto-relacional o ORM (por sus siglas en inglés Object Relational Mapping). Su función principal es servir como intérprete entre el código desarrollado desde el lado de back-end y la base de datos relacional.

Las estructuras de la base de datos relacional quedan vinculadas con las entidades lógicas o base de datos virtual definida a través del ORM utilizado, de tal modo que las acciones básicas CRUD (Create, Read, Update, Delete) a ejecutar sobre la base de datos física se realizan de forma indirecta por medio del ORM. Por lo tanto, los objetos o entidades de la base de datos creada podrán ser manipulados sencillamente por la extensión de LINQ explicada anteriormente.

1.2.6 Newtonsoft.Json

Newtonsoft.Json es una librería que proporciona diferentes funcionalidades para el manejo de JSON a través de objetos de tipo "Dictionary" como también permite la manipulación de JSON con los objetos JObject, JArray y JToken, convirtiéndose en los últimos años en uno de los estándares para usar y manipular texto y objetos con formato JSON.

Los dos métodos más utilizados de esta librería son los siguientes:

- **Deserialize:** intenta deserializar una cadena de JSON válida en un objeto del tipo especificado <T>, utilizando las asignaciones predeterminadas admitidas de forma nativa por JavaScriptSerializer.
- **SerializeObject:** consiste en un proceso de codificación de un objeto en un medio de almacenamiento (como puede ser un archivo, o un buffer de memoria) con el fin de transmitirlo a través de un formato más legible JSON.

1.2.7 ML.NET

ML.NET consiste en un framework de machine learning de tipo open-source y cross-platform con el cual se puede desarrollar nuevas funcionalidades en lenguaje C# o F# capaces de resolver problemas de diferente tipo, ya sea clasificación, detección de anomalías, visión de computadora, sistemas de recomendación, entre otros. Dicho framework permite a los desarrolladores implementar y entrenar sus propios modelos e infundir aprendizaje automático personalizado en sus aplicaciones utilizando .NET. Además proporciona carga de datos desde archivos y bases de datos, permite transformaciones de datos e incluye diversos algoritmos ML.

1.2.8 Swagger

Swagger²³ es un conjunto de herramientas de software de código abierto para diseñar, construir, documentar, y utilizar servicios web RESTful. Esta herramienta simplifica el desarrollo de API para usuarios, equipos y empresas con el conjunto de herramientas profesionales.

Swagger UI utiliza un documento JSON o YAML existente y crea una plataforma que ordena cada uno de nuestros métodos (get, put, post, delete) y categoriza cada operación. Cada uno de los métodos es expandible, y en ellos se puede encontrar un listado completo de los parámetros con sus respectivos ejemplos, permitiendo probar valores de llamada.

²³ "Swagger." <https://swagger.io/>.

1.3 Tecnología utilizadas general

1.3.1 Entorno de Desarrollo Integrado

Actualmente, gran porcentaje de los programadores hacen uso de un Entorno de Desarrollo Integrado (conocido cómo IDE) para realizar el proceso de compilación y ejecución del código de manera sencilla y veloz. Estos entornos permiten realizar pruebas sobre la aplicación y debuggear los distintos métodos que la componen, haciendo uso de una interfaz agradable e intuitiva para el uso constante. Otra de las ventajas, es el soporte sobre los lenguajes de programación específicos utilizados, lo cual implica una ventaja muy importante debido a la complejidad y extensión que poseen la sintaxis de dichos lenguajes de programación.

Para el desarrollo del front-end, se optó por la utilización del framework de desarrollo más oficial para el desarrollo de aplicaciones móviles para android, Android Studio²⁴. Está basado en IntelliJ IDEA de la compañía JetBrains, que soporta varios lenguajes basados en JVM, como Java, Clojure, Groovy, Kotlin y Scala. Como herramientas integradas para el desarrollo o construcción de programas en Android, contiene una interfaz de usuario que es construida o diseñada previamente, con variados modelos de pantalla, donde en ella los elementos existentes pueden ser desplazados. Adicionalmente se incluyen depuradores para emuladores y la posibilidad de trabajo con Logcat, más un soporte para Maven y Gradle, y archivos de configuración.



²⁴ "Download Android Studio and SDK tools" <https://developer.android.com/studio>.

Figura A1.1 Android Studio

En el apartado de back-end, si bien existen varios IDEs de programación disponibles, se optó por hacer uso de Visual Studio (versión 2019). Dispone de una versión comercial y otra dedicada a la comunidad de manera gratuita. Se encuentra desarrollado por Microsoft con actualizaciones constantes y provee un conjunto de herramientas que permiten desarrollar aplicaciones de escritorio, aplicaciones móviles, aplicaciones web ASP .NET, entre otras. Siendo compatible con el lenguaje de programación utilizado, C#. Entre las principales características que presenta, se destacan las herramientas de navegación para desplazarse, buscar y modificar código de manera rápida. Además se permite funcionalidad para depurar mediante perfiles y además se provee manejo e integración con git, permitiendo el control de versiones de manera ágil y eficiente.



Figura A1.2 Visual Studio

1.3.2 GIT y GitHub: administración de código

Al trabajar con un proyecto de tamaño moderado y con múltiples desarrolladores en simultáneo surgen algunos inconvenientes que se deben superar para poder completar el desarrollo del software. En ese caso, se vuelven complejas de manejar múltiples situaciones como lo es el trabajo colaborativo, el manejo de las distintas versiones de los archivos, el guardado del proyecto en un repositorio en la nube y la distribución de los permisos sobre los archivos, entre otros. Para solucionar estos inconvenientes y algunos otros se puede utilizar la herramienta llamada GIT²⁵.

²⁵ <https://git-scm.com/>

GIT es un sistema distribuido de control de versiones que rastrea cambios en el código fuente durante el desarrollo de software. Fue creado para facilitar el trabajo colaborativo (ediciones múltiples sobre archivos), reducir considerablemente los tiempos de despliegue (sube sólo los cambios), permitir regresar a versiones anteriores de forma sencilla y rápida, generar flujos de trabajo y proveer un ecosistema con múltiples herramientas (mediante una interfaz gráfica o una consola).

Al trabajar con GIT es importante seleccionar un cliente que provea la interacción con esta herramienta y la posibilidad de almacenar el código fuente correspondiente en la nube. Un servicio de uso gratuito para un límite de 6 desarrolladores es GitHub.

GitHub es una plataforma de desarrollo colaborativo para la gestión de repositorios Git diseñada para equipos profesionales y proyectos particulares, que opera bajo el nombre de GitHub²⁶, Inc. Proporciona un lugar central para administrar los repositorios de GIT, colaborar en el código fuente y seguir un flujo de desarrollo. Cuenta con algunas características que son el control de acceso, el control del flujo de trabajo, facilidades a la hora de hacer revisiones de código, la integración con Jira para la trazabilidad completa del desarrollo, y que puede ser utilizada bajo la consola de terminal de los distintos frameworks de desarrollo.

Concluyendo, con configuraciones de este tipo se beneficia mucho el trabajo remoto y colaborativo, se aceleran los tiempos de desarrollo y se evitan posibles errores. Es por esto que en la actualidad la gran mayoría de los proyectos lo integran para poder, así, cumplir de forma exitosa con el desarrollo de su sistema.



Figura A1.3 Github

²⁶ <https://github.com/>

1.4 Tecnologías utilizadas para la regresión lineal simple

Para llevar a cabo la implementación de la regresión lineal y realizar las pruebas de rendimiento correspondientes se hizo uso de ciertas herramientas especializadas en el área con el objetivo de realizar el proceso de manera sencilla y eficiente.

1.4.1 Scikit-Learn

Scikit-Learn²⁷ Es una biblioteca gratuita utilizada para el aprendizaje de software automático para el lenguaje de programación Python. Esta librería dispone de una gran variedad de algoritmos entre los cuales destacan:

- Regresión, cómo por ejemplo regresión lineal y logística.
- Clasificación, cómo K-NN.
- Clustering, incluyendo K-Means and K-Means++.
- Preprocesamiento, incluyendo Min-Max Normalization

Al poseer una gran variedad de algoritmos y herramientas a disposición, hacen de Scikit-Learn la biblioteca ideal para estructurar el sistema de análisis y el modelo planteado. Dichos algoritmos pueden utilizarse e integrarse con estructuras de datos externas provistas por librerías externas cómo Pandas. Esto significa que la adaptación e implementación de cualquier algoritmo sea realizada de manera sencilla y eficiente, proporcionando además componentes para la visualización de la información de manera intuitiva y personalizable.

Complementariamente, posee compatibilidad con diversas librerías de Python que han sido utilizadas para la realización de este trabajo:

- **Matplotlib**²⁸: Consiste en una librería gráfica utilizada para la generación de diversos tipos de gráficos cómo histogramas, diagramas de barras, etc. Siendo altamente personalizables, configurando cada aspecto del gráfico desde las etiquetas o los colores para cada componente hasta el tamaño final.

²⁷ <https://scikit-learn.org/stable/>

²⁸ <https://matplotlib.org/>

- **Pandas**²⁹: Se trata de un paquete de Python que provee estructuras de datos flexibles y rápidas diseñadas para que al trabajar con datos estructurados sea sencillo e intuitivo. Dispone de la capacidad para manejar los valores faltantes y de funciones específicas relacionadas a la alineación de datos para realizar agrupaciones bajo ciertas condiciones. Permite la mutabilidad de tamaño, permitiendo insertar columnas o eliminarlas del DataFrame sin problemas.
- **Numpy**³⁰: Es una biblioteca que proporciona soporte para la creación de vectores y matrices de gran tamaño y dimensiones, proporcionando además una colección de rutinas y funciones para procesar dichas estructuras de manera performante.
- **Seaborn**³¹: Es una librería de visualización, utilizada para la representación estadística, debido a que muestra sencillamente la relación que guardan los datos permitiendo detectar posibles tendencias y patrones. Posee una galería de gráficos muy amplia, creada para representar los análisis estadísticos de forma práctica y sencilla. Brinda herramientas para la personalización de los gráficos para ser utilizada de manera práctica e intuitiva.

1.4.2 Jupyter Notebook

Jupyter Notebook³² consiste en una aplicación web de código abierto que posibilita la creación e implementación de documentos con ecuaciones, gráficos y textos explicativos dentro de un mismo archivo. Proporciona un entorno de desarrollo sencillo e interactivo para configurar y utilizar. Es interactivo y flexible debido a que los usuarios escriben y ejecutan un código en concreto, observa el resultado y pueden realizar modificaciones concretas y repiten el proceso.

Puede ser empleado para tareas de ciencia de información, pudiendo realizar limpieza de datos, re estructuración, análisis, visualización modelado, machine learning entre otras.

²⁹ <https://pandas.pydata.org/>

³⁰ <https://numpy.org/>

³¹ <https://seaborn.pydata.org/>

³² <https://jupyter.org/>

Dispone de un funcionamiento intuitivo, en primer lugar, el usuario ingresa el código de programación en "filas" rectangulares en una página web. El navegador luego pasa el código a un "kernel" de back-end que ejecuta el código y devuelve los resultados.

Apéndice 2: Videos demostrativos de los flujos de la aplicación

A continuación, se incluyen las referencias a los videos demostrativos de la ejecución de la herramienta, a lo largo de los diferentes vistas y flujos dentro de la aplicación explicadas en el capítulo 5. Estos se encuentran disponibles dentro de la carpeta [Videos Demostrativos](#).

2.1 Demostración sobre el Mapa

2.1.1 Visualización de los controles nativos de los mapas de google utilizados

Referencia video: [video controles nativos sobre el mapa.webm](#)

2.1.2 Simulación del recorrido base de las diferentes unidades de colectivos

Referencia video: [video simulación del recorrido de los colectivos.webm](#)

2.1.3 Ventana para la selección de lugares sobre el mapa

2.1.3.1 Ubicación y selección de un sitio desde un marcador aleatorio en el mapa o marcador representativo de la unidad activa en el recorrido

Referencia video: [video selección ubicación desde un marcador aleatorio o colectivo activo.webm](#)

2.1.3.2 Atajo visual para obtener ubicación actual del usuario

Referencia video: [video obtener ubicación actual del usuario en la ventana de lugares.webm](#)

2.1.3.3 Selección manual lugar de origen y destino sobre el mapa

Referencia video: [video selección manual de los lugares marcados en el mapa.webm](#)

2.1.4 Flujos búsqueda de lugares

2.1.4.1 Búsqueda de recorrido con una sola línea de colectivo seleccionada

Referencia video: [video búsqueda de recorrido por una sola línea seleccionada.webp](#)

2.1.4.2 Búsqueda de recorrido con multiples líneas de colectivos seleccionadas

Referencia video: [video búsqueda de recorrido según múltiples líneas seleccionadas.webp](#)

2.1.4.2 Búsqueda de recorrido con los distintos algoritmos

Referencia video: [video búsqueda por comparación algoritmos.webm](#)

2.2 Demostración sobre el menú de configuración

2.2.1 Selección y visualización de rutas disponibles

Referencia video: [video menú selección de rutas colectivos.webm](#)

2.2.2 Selección algoritmo para cálculo de búsquedas

Referencia video: [video menú selección de algoritmos.webm](#)

2.2.3 Selección para remover las líneas de colectivos guardadas

Referencia video: [video menú selección remover líneas guardadas .webm](#)

Bibliografía

[1] Usos y aplicaciones SIG. Antoni Bosch Editor. Facultad de Ciencias Económicas - Unicen. 2013.

[2] Millonig, A., and M. Sleszynski. Sitting, Waiting, Wishing: Waiting Time Perception in Public Transport. In Proceedings of 15th International IEEE Conference on Intelligent Transportation Systems, IEEE, Piscataway, N.J., 2012.

[3] Lucas, J.L., and R.B. Heady. 2002. Flexitime commuters and their driver stress, feelings of time urgency, and commute satisfaction, Journal of Business and Psychology.

[4] Droit-Volet Sylvie and Sandrine Gil. The Time-Emotion Paradox. Philosophical Transactions of the Royal Society, Vol. 364, No. 1525, 2009.

[5] Taylor, Shirley. Waiting for the Service: The Relationship Between Delays and Evaluation of the Service. Journal of Marketing, Vol. 58, 1994.

[6] Hornik, Jacob. Time Estimation and Orientation Mediated by Transient Mood. Journal of Socio-Economics, Vol. 21, No. 3, 1992.

[7] Pruyn Ad, and Ale Smidts. Effects of Waiting on the Satisfaction with the Service: Beyond Objective Time Measures. International Journal of Research in Marketing, Vol. 15, 1998.

[8] M. Lagune-Reutler, A. Guthrie, Y. Fan, D. Levinson. Transit riders' perception of waiting time and stops' surrounding environments. University of Minnesota. 2015.

[9] ISO/IEC 25000:2014. International Standards named Systems and software Quality Requirements and Evaluation (SQuaRE). 2014.

[10] Osvaldo Simeone. A Brief Introduction to Machine Learning for Engineers. Department of Informatics. 2018.

[11] Antonio Pardo Merino, Miguel Ángel Ruiz Díaz. Análisis de datos con SPSS 13 Base. McGraw-Hill/Interamericana. 2005.

[12] M. Victoria Esteban, M. Paz Moral, Susan Orbe. Análisis de Regresión con Gretl. Facultad de Ciencias Económicas y Empresariales Universidad del País Vasco.

[13] Montero Granados. R. Modelos de regresión lineal múltiple. Documentos de Trabajo en Economía Aplicada. Universidad de Granada. España. 2016.

[14] James, G., Witten, D., Hastie, T., Tibshirani, R.. An Introduction to Statistical Learning with Applications in R. Springer. 2013.

[15] Dean Abbott. Applied Predictive Analytics: Principles and Techniques for the Professional Data Analyst. John Wiley & Sons, Inc. Wiley. 2014.

[16] John E. Hanke, Dean W. Wichern. Pronósticos en los negocios, Novena Edición. Pearson, Prentice Hall. 2010.

[17] A.K. Sharma. Text Book of Correlations and Regression. Discovery Publishing House. 2005.

[18] David S. Moore. Estadística aplicada básica, 2a ed. Antoni Bosch Editor. Purdue University. 1998.